

V2V Communication Security Simulation

Complete Beginner Study Pack

Sections S1 to S9 — Theory, Code, Practical and Presentation

Information Security Project — Beginner Study Notes

Combined edition — all nine per-section study files in one document

Contents

How to Use This Study Pack	4
1 S1 — Big Picture and Threat Model	5
1.1 Title and “What You’ll Learn”	5
1.2 Where This Fits	6
1.3 The Theory You Must Study	6
1.4 Code Walkthrough	9
1.5 Hands-On / Practical	11
1.6 Attack Relevance	12
1.7 Illustrations	12
1.8 Presentation Pack	13
1.9 Glossary	15
1.10 References and Further Study	16
2 S2 — Public Key Infrastructure and the Certificate Authority	18
2.1 Title and “What You’ll Learn”	18
2.2 Where This Fits	19
2.3 The Theory You Must Study	19
2.4 Code Walkthrough	22
2.5 Hands-On / Practical	28
2.6 Attack Relevance	29
2.7 Illustrations	30
2.8 Presentation Pack	30
2.9 Glossary	33
2.10 References and Further Study	34
3 S3 — The Cryptographic Core	36
3.1 Title and “What You’ll Learn”	36
3.2 Where This Fits	36
3.3 The Theory You Must Study	37
3.4 Code Walkthrough	40
3.5 Hands-On / Practical	44
3.6 Attack Relevance	45
3.7 Illustrations	46
3.8 Presentation Pack	47
3.9 Glossary	49
3.10 References and Further Study	50
4 S4 — The 4-Step Authentication Handshake	52
4.1 Title and “What You’ll Learn”	52
4.2 Where This Fits	52
4.3 The Theory You Must Study	53

4.4	Code Walkthrough	55
4.5	Hands-On / Practical	61
4.6	Attack Relevance	63
4.7	Illustrations	63
4.8	Presentation Pack	64
4.9	Glossary	67
4.10	References and Further Study	68
5	S5 — Secure Messaging	70
5.1	Title and “What You’ll Learn”	70
5.2	Where This Fits	70
5.3	The Theory You Must Study	71
5.4	Code Walkthrough	73
5.5	Hands-On / Practical	77
5.6	Attack Relevance	78
5.7	Illustrations	78
5.8	Presentation Pack	79
5.9	Glossary	81
5.10	References and Further Study	82
6	S6 — The Vehicle Node and TCP Networking	84
6.1	Title and “What You’ll Learn”	84
6.2	Where This Fits	84
6.3	The Theory You Must Study	85
6.4	Code Walkthrough	86
6.5	Hands-On / Practical	90
6.6	Attack Relevance	91
6.7	Illustrations	92
6.8	Presentation Pack	93
6.9	Glossary	95
6.10	References and Further Study	96
7	S7 — The Monitoring Dashboard	97
7.1	Title and “What You’ll Learn”	97
7.2	Where This Fits	97
7.3	The Theory You Must Study	98
7.4	Code Walkthrough	99
7.5	Hands-On / Practical	102
7.6	Attack Relevance	103
7.7	Illustrations	104
7.8	Presentation Pack	104
7.9	Glossary	106
7.10	References and Further Study	108
8	S8 — Attack Demonstrations	109
8.1	Title and “What You’ll Learn”	109
8.2	Where This Fits	109
8.3	The Theory You Must Study	110
8.4	Code Walkthrough	111
8.5	Hands-On / Practical	114
8.6	Attack Relevance	116
8.7	Illustrations	117
8.8	Presentation Pack	117

8.9	Glossary	119
8.10	References and Further Study	120
9	S9 — Security Evaluation	122
9.1	Title and “What You’ll Learn”	122
9.2	Where This Fits	122
9.3	The Theory You Must Study	123
9.4	Code Walkthrough	124
9.5	Hands-On / Practical	127
9.6	Attack Relevance	128
9.7	Illustrations	129
9.8	Presentation Pack	129
9.9	Glossary	131
9.10	References and Further Study	132

How to Use This Study Pack

This single document combines all nine section study guides for the **V2V Communication Security Simulation**. Each **chapter** is one section (S1–S9). Within every chapter the same ten parts appear in the same order:

1. Title and “What You’ll Learn”
2. Where This Fits
3. The Theory You Must Study
4. Code Walkthrough
5. Hands-On / Practical
6. Attack Relevance
7. Illustrations
8. Presentation Pack (slides, talking points, demo script, examiner Q&A)
9. Glossary
10. References and Further Study

Suggested reading order. Start with S1 for the big picture, then follow $S2 \rightarrow S3 \rightarrow S4 \rightarrow S5$ to trace the Identity $\rightarrow$ Authentication $\rightarrow$ Secure-Messaging pipeline. Then read S6 (networking), S7 (dashboard), S8 (attacks) and S9 (evaluation). Each chapter is deliberately self-contained, so a few core ideas are re-explained briefly where they recur — this is intentional, so any chapter can also be studied on its own.

Source of truth. Every explanation is based on the real code in the repository (`ca/`, `crypto/`, `protocol/`, `node/`, `dashboard/`, `attacks/`, `tests/` and `config.py`). Where the simulation is simplified compared with a real-world V2V system, the text says so explicitly.

Chapter 1

S1 — Big Picture and Threat Model

1.1 Title and “What You’ll Learn”

Section Title

S1 — The Big Picture and Threat Model of the V2V Communication Security Simulation.

Plain-English Summary (read this first)

This project is a small computer program, written in Python, that pretends to be two cars talking to each other on a road. The cars send tiny “I am here, this is my speed” safety messages. The catch is that the radio link between them is *untrusted* — a stranger could listen in, change the words, or pretend to be one of the cars. Section S1 does not teach a single piece of code in depth; instead it gives you the *map of the whole project*: what problem it solves, who the attacker is, what the four attacks are, and what six “security properties” the project promises to deliver. Once you understand S1, every later section (the certificates, the encryption maths, the handshake) will have a place to fit.

Learning Outcomes

After studying this section you will be able to:

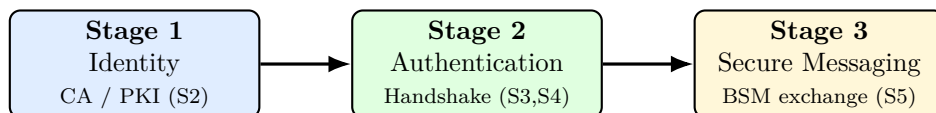
- Explain in one sentence what “Vehicle-to-Vehicle (V2V) communication” is and why it needs protecting.
- Define a **Basic Safety Message (BSM)** and say how often it is sent.
- Describe the **threat model**: who the attacker is and what powers they have over the network.
- Name and describe the **four attacks** the project defends against: replay, tamper, man-in-the-middle (MITM), and impersonation.
- Name and explain the **six security properties**: confidentiality, integrity, authentication, non-repudiation, forward secrecy, and replay prevention.
- Describe the three-stage pipeline — Identity, Authentication, Secure Messaging — and which project files belong to each stage.
- Read `config.py` and explain what every shared constant controls.

1.2 Where This Fits

S1 is the “cover page” of the whole system. Every other section is one component; S1 is the picture on the jigsaw box that tells you what the finished puzzle looks like. The project is built as a **pipeline of three stages**, and the readme describes them in exactly this order:

1. **Identity** — “Who are you?” Each vehicle is given a digital passport (a *certificate*) by a trusted authority. Files: `ca/ca.py`, `ca/issue_cert.py`, `ca/verify_cert.py` (this is Section S2).
2. **Authentication** — “Prove it!” Before talking, the two vehicles run a four-step handshake to check each other’s passport and agree on a secret key. File: `protocol/auth_protocol.py` (Section S4), built on the crypto core `crypto/crypto.py` (Section S3).
3. **Secure Messaging** — “Sign, then encrypt.” Now the vehicles exchange safety messages, each one stamped and sealed. File: `protocol/v2v_protocol.py` (Section S5).

The remaining files support these stages: `node/vehicle_node.py` runs the networking, `dashboard/app.py` shows a live monitoring screen, the `attacks/` scripts try to break the system, and `tests/evaluate_security.py` measures how well it holds up. `config.py` is the small file of shared settings that every stage reads from.



This study pack covers S1 (the map) plus S2, S3 and S4 (Stages 1 and 2).

1.3 The Theory You Must Study

1.3.1 Vehicle-to-Vehicle (V2V) Communication

Plain definition. V2V communication means cars wirelessly broadcasting short messages to nearby cars about their position, speed and movement, so the cars (or their drivers) can react to danger faster than a human could.

Real-life analogy. Imagine every car has a loudspeaker on its roof constantly calling out “I am here, going 80, turning left”. Other cars hear this and brake or steer in time.

Why it exists / what problem it solves. Human reaction time is slow and a driver cannot see around a blind corner or through the truck in front. V2V lets a car “see” a sudden brake three vehicles ahead and react in milliseconds, preventing pile-ups.

How it works at a beginner level. Each car has a small radio. Several times every second it sends a tiny message. Nearby cars receive it and update their picture of the traffic around them. *In this project the radio is simulated by an ordinary TCP network connection between two programs.*

Security property it relates to. None on its own — V2V is just the *communication*. The danger is precisely that, by default, this communication has *no* security at all. Everything else in the project exists to add security on top.

1.3.2 Basic Safety Message (BSM)

Plain definition. A BSM is the small data packet a vehicle broadcasts. It contains facts such as speed, heading (which way the car points), location, and whether the brakes are on.

Real-life analogy. A BSM is like a quick status shout: “Speed 80, heading 45 degrees, not braking.” It is short, said often, and instantly forgotten once newer information arrives.

Why it exists. Cars need a *standard, tiny* message so the radio channel is not flooded and every car understands every other car. Real systems (the SAE J2735 standard) broadcast a BSM about ten times per second.

How it works here. In this project the readme states a BSM is sent *once per second* — controlled by the constant `BSM_SEND_INTERVAL = 1.0` in `config.py`. A BSM carries fields like speed, heading and a braking flag. (Real V2V sends BSMs about ten times a second; the project slows this to once a second so a human watching the dashboard can follow along. This is a deliberate simplification.)

Security property it relates to. A BSM is the *thing being protected*. All six security properties below are promises about what happens to a BSM as it crosses the network.

1.3.3 The Untrusted Channel and the Threat Model

Plain definition. A *threat model* is a clear written statement of *who might attack you* and *what they are able to do*. An *untrusted channel* is a communication link you must assume the enemy controls.

Real-life analogy. Posting a letter through a postal service run by nosy, dishonest staff: they can read your letter, rewrite it, throw it away, copy it and post the copy again next week, or even write a brand-new letter and sign your name.

Why it exists. You cannot defend against an attacker you have not described. The threat model forces the designer to be honest: “assume the worst about the network.”

How it works here. The readme’s threat model assumes an attacker who fully controls the channel between Vehicle A and Vehicle B. That attacker can:

- **Eavesdrop** — read every message that passes.
- **Tamper** — change the contents of a message in transit.
- **Impersonate** — pretend to be a legitimate vehicle.
- **Replay** — record a real message and send it again later.

The project does *not* try to defend against an attacker who has stolen a vehicle’s secret private key files from its hard disk, or against denial of service (jamming the radio so no message gets through). Those are out of scope — a normal and honest limitation for a student project.

Security property it relates to. The threat model is the question; the six security properties are the answer.

1.3.4 The Four Attacks

(a) Replay Attack

Definition. The attacker copies a genuine, valid message and re-sends the exact same bytes later.

Analogy. Recording someone saying “open the door” and playing the recording back tomorrow to get in.

The danger. Re-sending an old “I am braking hard!” message could make cars behind brake for no reason and cause a crash.

Project defence. Every message carries a *timestamp*; messages older than `TIMESTAMP_MAX_AGE = 5.0` seconds are rejected. Every message also carries a one-time random number (a *nonce*); once a nonce has been seen it is remembered for `REPLAY_WINDOW_SECONDS = 300` seconds and any repeat is rejected. Demo script: `attacks/replay_attack.py`.

(b) Tamper Attack

Definition. The attacker intercepts a message and changes part of it, e.g. flips “speed: 60” to “speed: 0”.

Analogy. Steaming open a sealed envelope, editing the letter inside, and re-sealing it — except here the seal is mathematical and cannot be faked.

The danger. False speed or position data makes other cars make wrong, dangerous decisions.

Project defence. The encryption used, AES-256-GCM, produces a 128-bit *authentication tag* computed over the whole ciphertext. Changing even one bit makes the tag mismatch and decryption fails immediately. Demo: `attacks/tamper_attack.py`.

(c) Man-in-the-Middle (MITM) Attack

Definition. The attacker secretly sits between A and B, relaying (and possibly altering) every message, while each side believes it is talking directly to the other.

Analogy. A dishonest interpreter standing between two people who do not share a language: each trusts the interpreter completely, so the interpreter can quietly change every sentence.

The danger. The attacker could read or rewrite the entire conversation.

Project defence. Two layers. (Scenario A) If the attacker offers a *fake certificate*, it is rejected because it is not signed by the trusted CA. (Scenario B) If the attacker tries to *forge a signature*, it fails because they do not own the vehicle’s private key. Demo: `attacks/mitm_attack.py`.

(d) Impersonation / Spoofing

Definition. The attacker pretends to be Vehicle A without being Vehicle A.

Analogy. A stranger walking up to a border using a passport with someone else’s name on it.

The danger. A fake car could inject false safety messages and direct real cars into danger.

Project defence. Each real vehicle has a certificate signed by the CA *and* a private key that only it owns. During the handshake a vehicle must sign a fresh random challenge; only the true owner of the private key can produce a signature that verifies. Impersonation overlaps with MITM Scenario B and is handled by the same handshake (Section S4).

1.3.5 The Six Security Properties

A “security property” is a promise the system keeps. Table 1.1 lists the six the project claims, exactly as written in the readme.

Property	Plain meaning	How the project achieves it
Confidentiality	Only the intended car can read the message.	AES-256-GCM encryption with a shared session key.
Integrity	The message cannot be changed without it being noticed.	The 128-bit AES-GCM authentication tag.
Authentication	Each car proves who it really is.	X.509 certificates plus the 4-step handshake.
Non-repudiation	A sender cannot later deny it sent the message.	An ECDSA digital signature on every BSM.
Forward secrecy	Stealing a key tomorrow does not unlock today’s messages.	Fresh (ephemeral) ECDH keys generated per session.
Replay prevention	An old message cannot be reused.	Nonce cache plus a 5-second timestamp window.

Table 1.1: The six security properties promised by the V2V project.

One honest clarification about “5 seconds”. The readme’s property table summarises replay prevention as a “5s window”, but the code actually uses *two* separate windows, and it helps to keep them apart:

- `TIMESTAMP_MAX_AGE = 5.0` — a message whose timestamp is more than 5 seconds away from “now” is rejected as *stale*.
- `REPLAY_WINDOW_SECONDS = 300` — each nonce that has been seen is *remembered* for 300 seconds (5 minutes), so an exact repeat within that window is caught even if its timestamp were somehow fresh.

Together they form a belt-and-braces defence: the timestamp catches *old* messages cheaply, and the nonce cache catches *exact duplicates*.

1.4 Code Walkthrough

Section S1 has no large algorithm to trace. Its “code” is the project’s shared settings file, `config.py`. Every other module imports its numbers from here so that there is a single, easy-to-find place to change behaviour. Below is the real file, followed by a step-by-step explanation of every group.

```

1  """Shared configuration constants for the V2V security simulation."""
2
3  # -- Network Configuration --
4  VEHICLE_A_PORT = 9001          # TCP port Vehicle A listens on
5  VEHICLE_B_PORT = 9002          # TCP port Vehicle B listens on
6  DASHBOARD_A_PORT = 5001        # Flask dashboard for Vehicle A
7  DASHBOARD_B_PORT = 5002        # Flask dashboard for Vehicle B
8
9  # -- Certificate Paths --
10 CA_CERT_PATH = "ca/certs/ca_cert.pem"
11 CA_KEY_PATH = "ca/certs/ca_key.pem"
12 VEHICLE_A_CERT = "ca/certs/vehicle-a_cert.pem"
13 VEHICLE_A_KEY = "ca/certs/vehicle-a_key.pem"
14 VEHICLE_B_CERT = "ca/certs/vehicle-b_cert.pem"
15 VEHICLE_B_KEY = "ca/certs/vehicle-b_key.pem"
16
17 # -- Security Parameters --
18 REPLAY_WINDOW_SECONDS = 300     # Nonces remembered for 5 minutes
19 TIMESTAMP_MAX_AGE = 5.0        # Messages older than 5 seconds are rejected
20 BSM_SEND_INTERVAL = 1.0        # Send a BSM every 1 second
21
22 # -- Crypto Settings --
23 AES_KEY_LENGTH = 32             # 256 bits (AES-256)
24 NONCE_LENGTH = 12              # 96 bits (AES-GCM standard nonce size)
25 EC_CURVE = "secp256r1"         # NIST P-256 curve (same as TLS 1.3)
26 SESSION_KEY_INFO = b"v2v-session-key-v1" # HKDF info string
27
28 # -- Simulation Settings --
29 SIMULATED_LATITUDE = 6.9271     # Colombo, Sri Lanka
30 SIMULATED_LONGITUDE = 79.8612
31 SPEED_RANGE = (60.0, 100.0)     # km/h for simulated vehicle

```

Listing 1.1: `config.py` — shared constants for the whole simulation (decorative dashes in the original comments shortened to plain ASCII for printing).

1.4.1 Group 1 — Network Configuration

Purpose: tells each program which “door numbers” (TCP ports) to use.

1. A *port* is a numbered door on a computer; a program listens behind one door so messages can find it.
2. `VEHICLE_A_PORT = 9001` and `VEHICLE_B_PORT = 9002`: the two cars listen on ports 9001 and 9002.

3. `DASHBOARD_A_PORT = 5001` and `DASHBOARD_B_PORT = 5002`: each car also runs a small web page (the dashboard) on its own port so you can watch it in a browser.

Analogy: ports are flat numbers in an apartment block — the building (computer) has one street address (IP), but the postman needs the flat number to deliver to the right resident.

1.4.2 Group 2 — Certificate Paths

Purpose: the file locations of every digital passport and key.

1. `CA_CERT_PATH` / `CA_KEY_PATH`: the trusted authority’s public certificate and its secret signing key.
2. `VEHICLE_A_CERT` / `VEHICLE_A_KEY` (and the B pair): each vehicle’s own passport and matching private key.
3. All files end in `.pem`. PEM (Privacy-Enhanced Mail) is just a text format for storing keys and certificates — you will meet it in detail in Section S2.

Tie to theory: this group is the “Identity” stage on paper — it records where the passports live.

1.4.3 Group 3 — Security Parameters

Purpose: the timing rules that stop replay attacks.

1. `REPLAY_WINDOW_SECONDS = 300`: how long a used nonce is remembered. Implements the *nonce cache* part of replay prevention.
2. `TIMESTAMP_MAX_AGE = 5.0`: how stale a message may be before it is thrown away. Implements the *freshness* part of replay prevention.
3. `BSM_SEND_INTERVAL = 1.0`: send one safety message per second.

Tie to theory: this group is the numeric heart of the **replay prevention** property.

1.4.4 Group 4 — Crypto Settings

Purpose: the sizes and names of the cryptographic building blocks.

1. `AES_KEY_LENGTH = 32`: the secret key is 32 bytes = 256 bits, hence “AES-256”. Implements **confidentiality**.
2. `NONCE_LENGTH = 12`: the AES-GCM nonce is 12 bytes = 96 bits, the size recommended by the standard. (Note: this is the encryption nonce. The handshake in S4 uses a separate, larger 32-byte nonce — do not confuse the two.)
3. `EC_CURVE = "secp256r1"`: the name of the elliptic curve used for keys — the same NIST P-256 curve used by TLS 1.3 (the protocol behind HTTPS).
4. `SESSION_KEY_INFO = b"v2v-session-key-v1"`: a label fed into the key-derivation step (HKDF) so the derived key is tied to this exact purpose.

Honest note on accuracy: `config.py` is meant to be the single source of truth, but a few modules hard-code the same values directly instead of importing them — for example `crypto/crypto.py` writes `os.urandom(12)` and the literal string `b"v2v-session-key-v1"` itself rather than reading `NONCE_LENGTH` and `SESSION_KEY_INFO`. The numbers agree, so behaviour is correct, but it is good practice to notice this duplication and mention it if an examiner asks “is `config.py` actually used everywhere?”

1.4.5 Group 5 — Simulation Settings

Purpose: fake but realistic data so the cars have something to “say”.

1. `SIMULATED_LATITUDE / SIMULATED_LONGITUDE`: a fixed GPS location (Colombo, Sri Lanka) used as the starting point.

2. `SPEED_RANGE = (60.0, 100.0)`: the simulated car picks a speed between 60 and 100 km/h.

Tie to theory: this is the *content of a BSM* — the very data the six security properties are protecting.

Why a config file at all? If the port number or the timeout were scattered across ten files, changing one would mean ten risky edits. Centralising them is the same idea as a fuse box: all the switches in one labelled panel. There are no function arguments, return values, or exceptions in `config.py` — it is pure data, executed once when imported.

1.5 Hands-On / Practical

S1 is the “map” section, so the practical work is to (1) confirm your tools are installed, (2) inspect the shared settings, and (3) see the whole pipeline run once end to end.

1.5.1 Step 1 — Check Python and the crypto library

Open PowerShell in the project folder and run:

```
python --version
pip install cryptography flask requests pytest
```

You need Python 3.10 or newer. The `cryptography` library supplies all the encryption maths used by S3 and S4.

1.5.2 Step 2 — Inspect the shared constants

Run a one-line Python command that imports `config.py` and prints a few values:

```
python -c "import config; print(config.AES_KEY_LENGTH, config.TIMESTAMP_MAX_AGE,
    config.BSM_SEND_INTERVAL)"
```

Expected output:

```
32 5.0 1.0
```

What each number means: 32 is the AES key length in bytes (a 256-bit key); 5.0 is the maximum age in seconds before a message is treated as stale; 1.0 is the gap in seconds between safety messages.

1.5.3 Step 3 — Watch the whole pipeline once

You can see all three stages in a single run by generating the certificates and running the handshake demo (these belong to S2–S4 but together they show the S1 big picture):

```
python ca\ca.py
python ca\issue_cert.py --vehicle-id vehicle-a --output-dir ca\certs
python ca\issue_cert.py --vehicle-id vehicle-b --output-dir ca\certs
python protocol\auth_protocol.py
```

The first three commands build Stage 1 (Identity); the last command runs Stage 2 (Authentication) and ends by printing that both cars derived the *same* session key.

1.5.4 Try This Yourself (mini-experiment)

Open `config.py` and change one line:

```
Timestamp_max_age = 5.0      ->  Timestamp_max_age = 0.001
```

Predict before you run. 0.001 seconds is one millisecond. A message almost always takes longer than a millisecond to build and travel, so by the time it is checked it is already “too old”. Prediction: the handshake or BSM exchange should now *fail* with a “stale message” error, even though nothing malicious happened. This shows that security parameters are a balance — too strict and the honest system breaks; too loose and attacks slip through. **Remember to change the value back to 5.0 afterwards.**

1.6 Attack Relevance

S1 does not itself contain a defence — it is the chapter that *names* the attacks and the defences so the rest of the project has a target. Still, it is directly relevant to all four attacks because it defines them and points to the file that demonstrates each one.

Attack	What the attacker does	Which property/section stops it
Replay	Re-sends a captured valid message.	Replay prevention — timestamp + nonce cache (S4).
Tamper	Edits a message in transit.	Integrity — AES-GCM authentication tag (S3, S5).
MITM	Relays/alters traffic from the middle.	Authentication — certificates + handshake (S2, S4).
Impersonation	Pretends to be a real vehicle.	Authentication + non-repudiation — signed challenge (S2, S4).

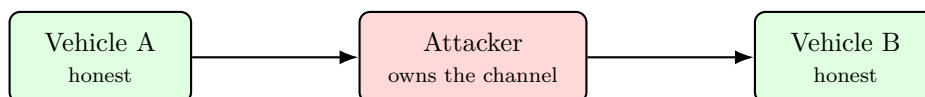
Table 1.2: S1 maps each attack to the property and section that defeats it.

The `config.py` constants `TIMESTAMP_MAX_AGE` and `REPLAY_WINDOW_SECONDS` are the only S1 items that play a *direct* defensive role: they are the dials that tune how aggressively replay attacks are rejected.

1.7 Illustrations

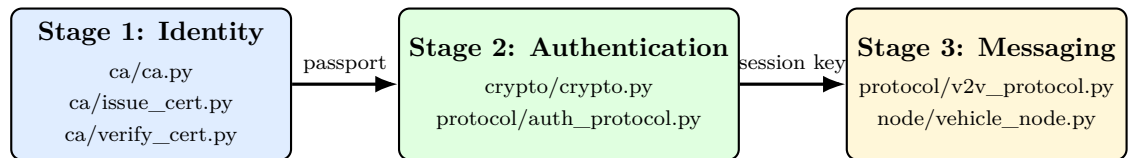
1.7.1 Diagram 1 — The Threat Model

The attacker owns the channel. Figure below shows the honest vehicles at the edges and the attacker controlling everything in between.



Attacker powers: **eavesdrop** (read), **tamper** (edit), **replay** (resend), **impersonate** (fake identity).

1.7.2 Diagram 2 — The Three-Stage Pipeline with Files



Supporting cast: dashboard/app.py (monitoring), attacks/*.py (break it), tests/evaluate_security.py (measure it), config.py (shared settings)

1.8 Presentation Pack

1.8.1 Slide Outline

Slide 1 — The Problem.

- Cars send tiny “safety messages” (BSMs) many times a second.
- The radio channel is untrusted — anyone nearby can listen.
- No security by default: a stranger can read, change, copy or fake messages.
- One false “I am braking” message could cause a crash.

Slide 2 — The Threat Model (who is the enemy?).

- We assume the attacker fully controls the channel.
- Four powers: eavesdrop, tamper, replay, impersonate.
- Out of scope: stealing key files from a car, jamming the radio.
- “Design for the worst attacker you can describe.”

Slide 3 — The Four Attacks.

- Replay — resend an old valid message.
- Tamper — edit a message in transit.
- MITM — secretly relay/alter the whole conversation.
- Impersonation — pretend to be a real vehicle.
- Each has a demo script in the **attacks/** folder.

Slide 4 — The Six Security Properties.

- Confidentiality, Integrity, Authentication.
- Non-repudiation, Forward secrecy, Replay prevention.
- Each property is a *promise* kept by a specific technique.
- Show Table 1.1 on screen.

Slide 5 — The Three-Stage Pipeline.

- Stage 1 Identity — give every car a passport (CA).
- Stage 2 Authentication — handshake to prove identity + share a key.
- Stage 3 Secure Messaging — sign then encrypt every BSM.
- **config.py** is the shared settings file feeding all three.

1.8.2 Speaker Talking Points (about 30–60 seconds each)

Slide 1. “Picture two cars on a highway. Modern cars can talk to each other by radio, sending tiny status messages called Basic Safety Messages many times a second — speed, direction, braking. This is genuinely useful: a car can warn the car behind it before a human even notices danger. But the radio link is open air. Anyone with a receiver can join in. If we send these messages with no protection, a stranger can read them, change them, or fake them — and a single false ‘I am braking hard’ message could trigger a real crash. So the project’s job is to add a security layer.”

Slide 2. “Before defending anything, a security engineer writes down the threat model — who the enemy is and what they can do. We make a deliberately pessimistic assumption: the attacker completely owns the channel between the two cars. They can listen to every message, change any message, copy a message and send it again, or pretend to be one of the cars. We are honest about limits too: we do not defend against someone physically stealing the secret key files from inside a car, or jamming the radio entirely. Naming what you do not cover is part of good security work.”

Slide 3. “Those attacker powers turn into four concrete attacks. A replay attack records a real message and plays it back later. A tamper attack edits a message as it passes. A man-in-the-middle attack puts the attacker secretly in the middle, like a dishonest interpreter relaying and changing every sentence. And impersonation is simply pretending to be a car you are not. The nice thing about this project is that each attack has its own script in the attacks folder, so we can actually run the attack and watch it fail.”

Slide 4. “For each attacker power we promise a security property. Confidentiality means only the intended car can read the message. Integrity means any change is detected. Authentication means each car proves who it is. Non-repudiation means a sender cannot later deny what it sent. Forward secrecy means that even if a key leaks tomorrow, today’s messages stay secret. And replay prevention means an old message cannot be reused. Six promises — and every one is kept by a specific, named technique we will see in later sections.”

Slide 5. “Finally, the big picture. The system is a pipeline of three stages. Stage one, Identity: a Certificate Authority gives every car a digital passport. Stage two, Authentication: the two cars run a four-step handshake to check passports and secretly agree on a shared key. Stage three, Secure Messaging: every safety message is signed and then encrypted before it goes out. One small file, `config.py`, holds the shared settings — ports, timeouts, key sizes — so all three stages read the same numbers.”

1.8.3 Live-Demo Script

1. **Show the settings.** Run `python -c "import config; print(config.TIMESTAMP_MAX_AGE)"`. Point at the 5.0 and say “this is our replay-freshness dial.”
2. **Show the project map.** Open the folder in the editor; point at the `ca/`, `crypto/`, `protocol/`, `attacks/` folders and link each to a stage.
3. **Run the pipeline.** Execute the four commands from Hands-On Step 3. Point at the final line “both vehicles share the same key” — that is the whole project working in miniature.
4. **Optional break-it moment.** Edit `TIMESTAMP_MAX_AGE` to 0.001, re-run the handshake, and point at the “stale message” failure to show parameters matter. Restore it to 5.0.

1.8.4 Examiner Q&A

1. **Q: What is a BSM and how often is it sent here?**
A: A Basic Safety Message — a small packet with speed, heading, location and braking state. In this project it is sent once per second (`BSM_SEND_INTERVAL = 1.0`); real V2V sends about ten per second, slowed here so a human can follow the dashboard.

2. **Q: What is a threat model and what is this project's?**

A: A written statement of who attacks and what they can do. Here: an attacker who fully controls the channel and can eavesdrop, tamper, replay and impersonate. Out of scope: stolen key files and jamming.

3. **Q: Name the four attacks and one defence for each.**

A: Replay (timestamp + nonce cache), Tamper (AES-GCM auth tag), MITM (CA-signed certificates), Impersonation (signed challenge proving private-key ownership).

4. **Q: Name the six security properties.**

A: Confidentiality, integrity, authentication, non-repudiation, forward secrecy, replay prevention.

5. **Q: Difference between confidentiality and integrity?**

A: Confidentiality hides the content (attacker cannot *read* it); integrity protects the content (attacker cannot *change* it undetected). AES-256-GCM gives both at once.

6. **Q: Why two replay windows — 5 seconds and 300 seconds?**

A: `TIMESTAMP_MAX_AGE` (5 s) cheaply rejects old messages; `REPLAY_WINDOW_SECONDS` (300 s) remembers each nonce so an exact duplicate is caught even within the freshness window. Two layers, belt and braces.

7. **Q: What is the purpose of `config.py`?**

A: A single, central place for shared constants (ports, timeouts, key sizes) so a setting is changed once, not in many files.

8. **Q: Is this a complete, real V2V system?**

A: No — it is a faithful *simulation*. Real V2V uses dedicated radio (DSRC/C-V2X) and broadcasts to many cars; this project uses TCP between two programs and slows the message rate. The cryptography, however, is the same industry-standard set used by TLS.

1.9 Glossary

V2V

Vehicle-to-Vehicle: cars exchanging wireless messages directly.

BSM

Basic Safety Message: the small status packet a vehicle broadcasts.

Threat model

A written description of who might attack and what they can do.

Untrusted channel

A communication link assumed to be under attacker control.

Eavesdrop

To secretly read messages in transit.

Tamper

To change a message's contents in transit.

Replay attack

Re-sending a previously captured valid message.

MITM

Man-in-the-Middle: an attacker secretly relaying/altering traffic.

Impersonation

Pretending to be a legitimate party (spoofing identity).

Confidentiality

Property: only the intended recipient can read a message.

Integrity

Property: any change to a message is detected.

Authentication

Property: each party proves who it really is.

Non-repudiation

Property: a sender cannot later deny having sent a message.

Forward secrecy

Property: past messages stay secret even if a key leaks later.

Replay prevention

Property: an old message cannot be reused.

Nonce

“Number used once”: a one-time random value that detects duplicates.

Timestamp

A recorded time, used here to reject stale messages.

CA

Certificate Authority: the trusted issuer of digital passports.

Certificate

A digital passport binding an identity to a public key.

PEM

A text file format for storing keys and certificates.

Port

A numbered “door” on a computer that a program listens behind.

TCP

Transmission Control Protocol: a reliable network connection type.

AES-256-GCM

The authenticated encryption algorithm used for confidentiality + integrity.

ECDSA

Elliptic Curve Digital Signature Algorithm: proves who wrote a message.

ECDH

Elliptic Curve Diffie-Hellman: lets two parties agree a shared secret.

HKDF

A function that turns a raw shared secret into a clean usable key.

Session key

A temporary secret key used to encrypt one conversation.

config.py

The project’s shared-settings file.

1.10 References and Further Study

- **V2V overview (NHTSA, US road-safety agency)** — <https://www.nhtsa.gov/research-data/vehicle-vehicle-communication>
Read for: why V2V exists and the real-world safety case.

- **Vehicular communication systems (Wikipedia)** — https://en.wikipedia.org/wiki/Vehicular_communication_systems
Read for: a plain-English overview of the whole field.
- **SAE J2735 message-set standard** — https://www.sae.org/standards/content/j2735_202309/
Read for: the real definition of the Basic Safety Message.
- **IEEE 1609.2 (security for V2X)** — <https://standards.ieee.org/ieee/1609.2/10258/>
Read for: how real V2V security and certificates work.
- **Information security / the CIA triad (Wikipedia)** — https://en.wikipedia.org/wiki/Information_security
Read for: confidentiality, integrity, availability explained.
- **Replay attack (Wikipedia)** — https://en.wikipedia.org/wiki/Replay_attack
Read for: how replay works and standard defences.
- **Man-in-the-middle attack (Wikipedia)** — https://en.wikipedia.org/wiki/Man-in-the-middle_attack
Read for: the MITM concept and why authentication stops it.
- **Non-repudiation (Wikipedia)** — <https://en.wikipedia.org/wiki/Non-repudiation>
Read for: why digital signatures give non-repudiation.
- **Forward secrecy (Wikipedia)** — https://en.wikipedia.org/wiki/Forward_secrecy
Read for: why ephemeral keys protect past sessions.
- **Threat modelling (OWASP)** — https://owasp.org/www-community/Threat_Modeling
Read for: how professionals build a threat model.
- **NIST Computer Security Resource Center glossary** — <https://csrc.nist.gov/glossary>
Read for: precise definitions of every security term.
- **Python cryptography library documentation** — <https://cryptography.io/en/latest/>
Read for: the library this whole project is built on (used heavily in S2–S5).

End of Section S1. Next: S2 — Public Key Infrastructure / Certificate Authority.

Chapter 2

S2 — Public Key Infrastructure and the Certificate Authority

2.1 Title and “What You’ll Learn”

Section Title

S2 — The Public Key Infrastructure (PKI): how the project creates a trusted “passport office” and issues digital passports to vehicles.

Plain-English Summary (read this first)

Before two cars can trust each other, each one needs a digital *identity* that cannot be faked. Section S2 builds that identity system. A special program, the **Certificate Authority (CA)**, acts like a passport office: it creates its own master signing key and a self-signed master certificate, and then *signs* a small certificate for each vehicle. Any car can later check another car’s certificate by verifying the CA’s signature on it — if the CA did not sign it, it is a fake. Three Python files do this work: `ca.py` creates the CA, `issue_cert.py` signs a certificate for one vehicle, and `verify_cert.py` checks a certificate.

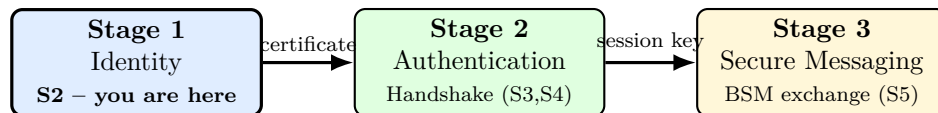
Learning Outcomes

After studying this section you will be able to:

- Explain what **public-key (asymmetric) cryptography** is and why a key comes in a public/private pair.
- Explain **elliptic-curve cryptography** and the NIST **P-256** curve at a beginner level.
- Explain **SHA-256 hashing** and **ECDSA digital signatures**.
- Describe an **X.509 certificate**, its main fields, and the **BasicConstraints** and **KeyUsage** extensions.
- Explain **PEM, DER and Base64** encodings.
- Explain **PKI**, the **chain of trust**, a **trust anchor**, and a **self-signed root**.
- Walk through every function in `ca.py`, `issue_cert.py` and `verify_cert.py` and say what each line does.
- Run the three scripts, read their output, and explain how this stage defeats a man-in-the-middle attacker using a forged certificate.

2.2 Where This Fits

S2 is **Stage 1: Identity** of the three-stage pipeline. It comes first because authentication (Stage 2) and secure messaging (Stage 3) are both built *on top of* the certificates created here. The handshake in S4 does not re-invent identity — it simply calls a verification routine that checks the certificates this section produced.



The *certificate* produced here is the bridge into Stage 2: the handshake hands a certificate to the other vehicle as proof of identity.

2.3 The Theory You Must Study

2.3.1 Symmetric vs Asymmetric (Public-Key) Cryptography

Plain definition. *Symmetric* cryptography uses *one* secret key for both locking and unlocking — both sides must already share it. *Asymmetric* (or *public-key*) cryptography uses a *pair* of keys: a **private key** kept secret, and a **public key** given to everyone. What one key does, only the other can undo.

Real-life analogy. Symmetric is one house key that both you and your flatmate carry. Asymmetric is a mailbox: anyone can drop a letter through the public slot, but only you have the private key to open the box and read it.

Why it exists. Symmetric keys have a chicken-and-egg problem: how do two strangers agree on a shared secret over an untrusted channel *without* already having a secret? Public-key cryptography solves this — you can publish your public key to the whole world and it is still safe.

How it works here. Every key in S2 is a public/private *pair*. The CA has a pair; each vehicle has a pair. The private key is used to *sign*; the public key is used to *verify*. The certificate is simply a wrapper that publishes a public key together with a name.

Security property it provides. Public-key cryptography underpins **authentication** and **non-repudiation**.

2.3.2 Elliptic-Curve Cryptography (ECC) and the P-256 Curve

Plain definition. ECC is a family of public-key algorithms whose security comes from the difficulty of a particular kind of arithmetic done on a curved mathematical shape called an *elliptic curve*.

Real-life analogy. Think of points hopping around a closed loop according to a fixed rule. Going *forwards* a known number of hops is easy; working out *how many hops* someone took just by seeing where they landed is practically impossible. The private key is “the number of hops”; the public key is “where you landed”.

Why it exists. An older algorithm, RSA, needs very large keys (2048+ bits) for strong security. ECC reaches the same strength with much smaller keys (256 bits) — so it is faster and the certificates and signatures are smaller, which matters for cars sending messages many times a second.

How it works here. The project uses the curve named SECP256R1, also called **NIST P-256**. Both `ca.py` and `issue_cert.py` call `generate_private_key(SECP256R1())` to create keys on this curve. P-256 gives a “128-bit security level” — strong enough for decades — and it is the same curve used by TLS 1.3 (the security behind HTTPS).

Security property it provides. ECC is the engine under **authentication** (via ECDSA signatures) and the key exchange in S3/S4.

2.3.3 SHA-256 Hashing

Plain definition. A *hash function* takes any amount of data and produces a short, fixed-size fingerprint of it. SHA-256 always produces 256 bits (32 bytes). The same input always gives the same fingerprint; changing one bit of input changes the fingerprint completely and unpredictably.

Real-life analogy. A paper shredder that always outputs the same-size pile of confetti — but a magic one, where identical documents always give an identical pile, and you cannot rebuild the document from the confetti.

Why it exists. It is far faster to sign or compare a tiny 32-byte fingerprint than a huge document, and the fingerprint reliably represents the whole document.

How it works here. When the CA signs a certificate it does not sign the raw certificate bytes directly — it signs their SHA-256 hash. The code shows this as `algorithm=hashes.SHA256()` passed to the `sign` call, and `issue_cert.py` also uses SHA-256 to compute a human-readable *fingerprint* of the finished certificate.

Security property it provides. Hashing supports **integrity**: any change to the certificate changes its hash, so the signature no longer matches.

2.3.4 Digital Signatures and ECDSA

Plain definition. A digital signature is a short value, produced with a *private* key, that anyone holding the matching *public* key can check. ECDSA (Elliptic Curve Digital Signature Algorithm) is the elliptic-curve version.

Real-life analogy. A wax seal on a letter. Only the owner of the unique seal-ring (private key) can press a genuine seal; anyone can look at the seal and recognise it (public key) without owning the ring.

Why it exists. It answers two questions at once: “who really wrote this?” (authentication) and “has it been changed since?” (integrity) — and because only the owner could have signed it, they cannot later deny it (non-repudiation).

How it works here. *Signing*: the CA computes the SHA-256 hash of the certificate body and, using its private key and the P-256 curve, produces a signature. *Verifying*: a vehicle takes the certificate body, re-hashes it, and uses the CA’s *public* key to confirm the signature matches. In `verify_cert.py` this is the line `ca_public_key.verify(certificate.signature, certificate.tbs_certificate_bytes, ec.ECDSA(...))`.

Security property it provides. **Authentication, integrity and non-repudiation.**

2.3.5 X.509 Certificates

Plain definition. An X.509 certificate is a standard-format digital document that *binds* a public key to an identity, and is *signed* by an issuer so others can trust the binding.

Real-life analogy. A passport. It contains your photo and name (the identity), is printed in a worldwide standard format, and carries the issuing government’s tamper-proof features (the signature). A passport with no government features is just a booklet — worthless.

Why it exists. A bare public key is just a number — it does not say *whose* key it is. The certificate attaches a name (“vehicle-a”) to the key and has a trusted party vouch for that attachment.

How it works here. A certificate in this project contains:

- **Subject** — who the certificate is about (e.g. Common Name `vehicle-a`, Organisation `V2V-Security-Lab`).
- **Issuer** — who signed it (the CA, `V2V-Root-CA`).
- **Public key** — the subject’s P-256 public key.
- **Serial number** — a unique random number for this certificate.

- **Validity period** — “not valid before” and “not valid after” dates.
 - **Extensions** — extra rules: `BasicConstraints` and `KeyUsage`.
 - **Signature** — the issuer’s ECDSA signature over all of the above.
- Security property it provides.** The foundation of **authentication**.

2.3.6 Certificate Extensions: `BasicConstraints` and `KeyUsage`

Plain definition. Extensions are extra fields that state *rules* about how a certificate may be used. Each extension can be marked **critical**, meaning a verifier that does not understand it must reject the certificate.

`BasicConstraints` answers: “Is this a Certificate Authority?”

- For the CA: `ca=True` — yes, it is allowed to sign *other* certificates.
- For a vehicle: `ca=False` — no, a vehicle may not sign certificates for anyone else.

`KeyUsage` answers: “What is this key allowed to do?”

- For the CA: `key_cert_sign=True` — the only thing the CA key may do is sign certificates.
- For a vehicle: `digital_signature=True` — the vehicle key may sign data (used later in the handshake and BSM signing).

Real-life analogy. `BasicConstraints` is the difference between a *passport office* (may issue passports) and a *citizen* (may not). `KeyUsage` is like a driving licence that says “car only, not motorbike”.

Why it exists. Without these limits, a stolen vehicle certificate could be abused to issue fake certificates for a whole fleet. The constraints box each key into exactly one job.

Security property it provides. It strengthens **authentication** by preventing privilege escalation (a vehicle pretending to be a CA).

2.3.7 Encodings: Base64, DER and PEM

Plain definition.

- **DER** (Distinguished Encoding Rules) is the compact *binary* form of a certificate or key — raw bytes.
- **Base64** is a way of writing binary bytes using only 64 ordinary printable letters/digits, so binary survives being copied as text.
- **PEM** (Privacy-Enhanced Mail) wraps Base64 text between header and footer lines such as `---BEGIN CERTIFICATE---` and `---END CERTIFICATE---`.

Real-life analogy. DER is a delicate object; Base64 is bubble-wrapping it so it travels safely; PEM is the labelled cardboard box with “BEGIN” and “END” written on the flaps.

Why it exists. Binary bytes can be corrupted by e-mail, text editors or copy-paste. PEM text survives all of that, and the BEGIN/END labels make it obvious what is inside.

How it works here. Every file the CA writes ends in `.pem`. The code calls `certificate.public_bytes(serialization.Encoding.PEM)` and `private_key.private_bytes(encoding=serialization.Encoding.PEM, ...)`. When loading, it calls `x509.load_pem_x509_certificate(...)`.

Security property it provides. None directly — encoding is about *format*, not secrecy. (A PEM file is still readable; secrecy of a private key comes from keeping the file itself protected.)

2.3.8 PKI, the Chain of Trust, and the Self-Signed Root

Plain definition. A **Public Key Infrastructure (PKI)** is the whole system of authorities, certificates and rules that lets strangers trust each other’s public keys. A **chain of trust** is the

line of signatures from a certificate you are checking back to one you already trust. A **trust anchor** (or **root**) is that already-trusted starting point.

Real-life analogy. You trust a stranger’s passport because you trust the *country* that issued it — you do not need to personally know the stranger. The country is the trust anchor; the passport chains back to it.

The self-signed root problem. The CA is the top of the tree — nobody is above it to sign *its* certificate. So the CA signs its own certificate with its own private key: a **self-signed certificate**. Its “issuer” and “subject” are the same name. This is not a security trick; it simply means “trust in this root must be established out-of-band” — you must obtain `ca_cert.pem` through a trusted route and decide to trust it. Every vehicle is given a copy of `ca_cert.pem` as its trust anchor.

How it works here. The chain in this project is exactly two links long:

V2V-Root-CA (self-signed) → vehicle-a / vehicle-b certificate.

To trust a vehicle’s certificate, you check one signature: “did V2V-Root-CA sign it?”

Security property it provides. The whole basis of **authentication** and the defence against MITM.

2.3.9 Serial Number and Fingerprint

Serial number. A unique number the issuer puts inside each certificate so it can be referred to and, if needed, revoked. The code uses `x509.random_serial_number()` — a large random number, so two certificates practically never collide.

Fingerprint. The SHA-256 hash *of the whole finished certificate*, shown as colon-separated hex. It is computed *after* signing and is a quick way for a human to compare “is this the same certificate?”. Do not confuse it with the signature: the signature is made by the CA’s key; the fingerprint is just a hash anyone can recompute.

2.4 Code Walkthrough

The three files form a clear story: `ca.py` *creates* the authority, `issue_cert.py` *issues* a vehicle certificate, and `verify_cert.py` *checks* a certificate.

2.4.1 File: `ca/ca.py` — creating the Certificate Authority

`generate_ca_key()` — make the CA’s key pair

Purpose: create the CA’s private/public key pair on curve P-256.

```
1 def generate_ca_key() -> EllipticCurvePrivateKey:
2     """Generate the CA private key."""
3     try:
4         return generate_private_key(SECP256R1())
5     except Exception as exc:
6         raise RuntimeError(f"Failed to generate CA key pair: {exc}") from exc
```

Listing 2.1: `ca/ca.py` — generate the CA key pair.

Step by step:

1. `generate_private_key(SECP256R1())` asks the cryptography library to create a fresh random private key on the NIST P-256 curve.
2. The returned object is the *private* key, but it also carries the matching *public* key — you get it later with `.public_key()`.

3. If anything fails, the error is wrapped in a clear `RuntimeError` so the caller sees a helpful message.

Arguments / return / exceptions: no arguments; returns an `EllipticCurvePrivateKey`; may raise `RuntimeError`. **Theory link:** this is *elliptic-curve key generation* — the “choose a secret number of hops” step. *Analogy:* the passport office forging its own unique master seal-ring.

`create_self_signed_certificate()` — build the root certificate

Purpose: build the CA’s own X.509 certificate and sign it with its own key.

```
1 def create_self_signed_certificate(private_key):
2     subject = x509.Name([
3         x509.NameAttribute(NameOID.COMMON_NAME, "V2V-Root-CA"),
4         x509.NameAttribute(NameOID.ORGANIZATION_NAME, "V2V-Security-Lab"),
5     ])
6
7     valid_from = datetime.utcnow()
8     try:
9         valid_to = valid_from.replace(year=valid_from.year + 10)
10    except ValueError:
11        valid_to = valid_from + timedelta(days=3650)    # 10 years
12
13    builder = (
14        x509.CertificateBuilder()
15        .subject_name(subject)
16        .issuer_name(subject)                        # self-signed: issuer ==
17        .public_key(private_key.public_key())
18        .serial_number(x509.random_serial_number())
19        .not_valid_before(valid_from)
20        .not_valid_after(valid_to)
21        .add_extension(x509.BasicConstraints(ca=True, path_length=None),
22                        critical=True)
23        .add_extension(x509.KeyUsage(key_cert_sign=True, crl_sign=False,
24                                    digital_signature=False, ...), critical=True)
25    )
26    return builder.sign(private_key=private_key, algorithm=hashes.SHA256())
```

Listing 2.2: `ca/ca.py` — build and self-sign the root certificate (KeyUsage flags abbreviated with “...” for space; in the real file every flag is written out).

Step by step:

1. Build the **subject name**: Common Name V2V-Root-CA, Organisation V2V-Security-Lab. This names the CA.
2. Set the **validity window**: from now until ten years later. `replace(year=...+10)` can fail on a 29-February date, so the `except ValueError` falls back to adding 3650 days. Honest, defensive coding.
3. Use `CertificateBuilder()` to assemble the fields. Crucially, `.subject_name(subject)` and `.issuer_name(subject)` use the *same* name — this is what makes it **self-signed**.
4. `.public_key(private_key.public_key())` embeds the CA’s public key so others can later verify things the CA signed.
5. `.serial_number(x509.random_serial_number())` gives it a unique random serial.
6. `BasicConstraints(ca=True)` marks it as a CA; `KeyUsage(key_cert_sign=True)` says its key may only sign certificates. Both are `critical=True`.
7. `builder.sign(private_key=private_key, algorithm=hashes.SHA256())` produces the final signed certificate. It signs the SHA-256 hash of the body *with the CA’s own key*.

Arguments: `private_key` — the CA key used both to embed the public key and to sign. **Returns:** an `x509.Certificate`. **Exceptions:** wrapped as `RuntimeError` on failure. **Theory link:** X.509 certificate, BasicConstraints/KeyUsage extensions, ECDSA signing, the self-signed root. *Analogy:* the passport office printing its *own* master credential and stamping it with its own seal — because nobody ranks above it.

save_ca_artifacts() — write the PEM files

Purpose: save the certificate (public) and the private key (secret) to disk.

```

1 def save_ca_artifacts(certificate, private_key, output_directory):
2     output_directory.mkdir(parents=True, exist_ok=True)
3
4     cert_path = output_directory / "ca_cert.pem"
5     cert_path.write_bytes(certificate.public_bytes(serialization.Encoding.PEM))
6
7     key_path = output_directory / "ca_key.pem"
8     key_path.write_bytes(private_key.private_bytes(
9         encoding=serialization.Encoding.PEM,
10        format=serialization.PrivateFormat.TraditionalOpenSSL,
11        encryption_algorithm=serialization.NoEncryption(),
12    ))
13    return cert_path, key_path

```

Listing 2.3: `ca/ca.py` — write the CA certificate and key as PEM.

Step by step:

1. Create the output folder if it does not exist (`exist_ok=True` means “do not error if it already exists”).
2. Write `ca_cert.pem` — the certificate is *public* and may be shared freely.
3. Write `ca_key.pem` — the private key. `NoEncryption()` means the key file is *not* password-locked.

Honest note: `NoEncryption()` keeps the demo simple, but it means anyone who can read `ca_key.pem` owns the whole CA. A real CA stores its key encrypted, or inside dedicated tamper-proof hardware (a Hardware Security Module). This is a deliberate, declared simplification.

Returns: the two file paths. **Theory link:** PEM encoding; public-versus-private asymmetry.

The supporting functions

- `configure_logging()` — sets the log message format so the terminal shows timestamped, readable status lines. No security role.
- `log_certificate_summary(certificate)` — prints the new certificate’s subject, issuer, serial, validity dates and key type so a human can sanity-check it.
- `setup_ca()` — the conductor: it calls `generate_ca_key` → `create_self_signed_certificate` → `log_certificate_summary` → `save_ca_artifacts`, and returns 0 on success or 1 on failure (an *exit code* the operating system can read).

2.4.2 File: `ca/issue_cert.py` — issuing a vehicle certificate

parse_args() — read the command line

Purpose: let the user choose which vehicle and where to save its files. It defines two required options: `-vehicle-id` (e.g. `vehicle-a`) and `-output-dir` (e.g. `ca/certs`). `argparse` is Python’s standard command-line reader.

load_ca_materials() — load the CA’s certificate and key

Purpose: read `ca_cert.pem` and `ca_key.pem` back from disk.

```
1 def load_ca_materials(ca_dir):
2     ca_cert_path = ca_dir / "ca_cert.pem"
3     ca_key_path = ca_dir / "ca_key.pem"
4
5     if not ca_cert_path.exists():
6         raise FileNotFoundError(f"CA certificate not found ... Run ca/ca.py first")
7     if not ca_key_path.exists():
8         raise FileNotFoundError(f"CA private key not found ... Run ca/ca.py first")
9
10    ca_cert = x509.load_pem_x509_certificate(ca_cert_path.read_bytes())
11    ca_key_obj = serialization.load_pem_private_key(ca_key_path.read_bytes(),
12                                                    password=None)
13    if not isinstance(ca_key_obj, EllipticCurvePrivateKey):
14        raise TypeError("Loaded CA key is not an EC private key.")
15    return ca_cert, ca_key_obj
```

Listing 2.4: `ca/issue_cert.py` — load the CA so we can sign with it.

Step by step:

1. Build the two file paths inside the CA’s `certs` folder.
2. Check both files exist; if not, raise a `FileNotFoundError` that *tells the user what to do* (“Run `ca/ca.py` first”).
3. Parse the PEM certificate with `load_pem_x509_certificate`.
4. Parse the PEM private key with `load_pem_private_key`; `password=None` matches the un-encrypted key written earlier.
5. Confirm the key really is an elliptic-curve key before trusting it.

Theory link: PEM decoding; loading the trust anchor’s private key so it can act as the issuer.

generate_vehicle_key() — the vehicle’s own key pair

Identical idea to `generate_ca_key`: it calls `generate_private_key(SECP256R1())` to create a *fresh* P-256 key pair for the vehicle. Crucially, the vehicle’s private key is generated here and *never leaves* the vehicle’s files — the CA never sees it. The CA only signs the vehicle’s *public* key.

create_vehicle_certificate() — the CA signs the vehicle

Purpose: build the vehicle’s X.509 certificate and sign it with the CA’s private key.

```
1 def create_vehicle_certificate(vehicle_id, vehicle_public_key, ca_cert, ca_key):
2     subject = x509.Name([
3         x509.NameAttribute(NameOID.COMMON_NAME, vehicle_id),
4         x509.NameAttribute(NameOID.ORGANIZATION_NAME, "V2V-Security-Lab"),
5     ])
6
7     valid_from = datetime.utcnow()
8     try:
9         valid_to = valid_from.replace(year=valid_from.year + 1) # 1 year
10    except ValueError:
11        valid_to = valid_from + timedelta(days=365)
12
13    builder = (
14        x509.CertificateBuilder()
```

```

15     .subject_name(subject)
16     .issuer_name(ca_cert.subject)                # issuer = the CA
17     .public_key(vehicle_public_key)              # the VEHICLE's public key
18     .serial_number(x509.random_serial_number())
19     .not_valid_before(valid_from)
20     .not_valid_after(valid_to)
21     .add_extension(x509.BasicConstraints(ca=False, path_length=None),
22                   critical=True)
23     .add_extension(x509.KeyUsage(digital_signature=True, key_cert_sign=False,
24                                 ...), critical=True)
25 )
26 return builder.sign(private_key=ca_key, algorithm=hashes.SHA256())

```

Listing 2.5: `ca/issue_cert.py` — build and CA-sign a vehicle certificate.

Step by step: compare it carefully with the CA’s certificate above — three differences matter:

1. **Subject** is the *vehicle’s* name; **issuer** is `ca_cert.subject` — the CA. Subject $\neq$ issuer, so this is *not* self-signed; it is part of a chain.
2. `.public_key(vehicle_public_key)` embeds the *vehicle’s* public key (not the CA’s).
3. `BasicConstraints(ca=False)` — a vehicle is *not* a CA; `KeyUsage(digital_signature=True)` — it may sign data (used in the S4 handshake and S5 BSM signing), but `key_cert_sign` is `False`.
4. Validity is *one* year (the CA’s was ten) — vehicle credentials are shorter-lived.
5. `builder.sign(private_key=ca_key, ...)` — the certificate is signed with the **CA’s** private key. *This single line is the whole point of a CA.*

Arguments: `vehicle_id`, `vehicle_public_key`, `ca_cert` (used only for its subject), `ca_key` (the signer). **Returns:** the signed `x509.Certificate`. **Theory link:** the chain of trust is created right here — the CA’s signature is the link between root and vehicle. *Analogy:* the passport office stamping a citizen’s passport with the office’s own seal.

`save_vehicle_artifacts()` and `format_fingerprint()`

`save_vehicle_artifacts` writes `<id>_cert.pem` and `<id>_key.pem` exactly like the CA’s saver (PEM, un-encrypted key). `format_fingerprint` computes `certificate.fingerprint(SHA256())` and joins the bytes as colon-separated lowercase hex, e.g. `a1:b2:c3:...`, so a human can compare certificates at a glance.

`issue_vehicle_certificate()` — the conductor

It reads the arguments, loads the CA, generates the vehicle key, builds and signs the certificate, saves the two files, prints the fingerprint and serial, and returns exit code 0 or 1.

2.4.3 File: `ca/verify_cert.py` — checking a certificate

This script is the “border guard”. It runs three independent checks and the certificate must pass *all three*.

`check_signature()` — was it really signed by our CA?

Purpose: the most important check — prove the CA signed this certificate.

```

1 def check_signature(certificate, ca_certificate):
2     try:
3         ca_public_key = ca_certificate.public_key()
4         if not isinstance(ca_public_key, EllipticCurvePublicKey):

```

```

5         return False, "[FAIL] Signature invalid - CA public key is not ECDSA."
6
7     ca_public_key.verify(
8         certificate.signature,                # the signature bytes
9         certificate.tbs_certificate_bytes,     # the "to-be-signed"
10        body
11        ec.ECDSA(certificate.signature_hash_algorithm),
12    )
13    issuer_cn = ca_certificate.subject.get_attributes_for_oid(NameOID.COMMON_NAME)
14    issuer_name = issuer_cn[0].value if issuer_cn else "trusted CA"
15    return True, f"[OK] Signature valid - signed by {issuer_name}"
16 except InvalidSignature:
17     return False, "[FAIL] Signature invalid - NOT signed by trusted CA"
18 except Exception as exc:
19     return False, f"[FAIL] Signature check error - {exc}"

```

Listing 2.6: ca/verify_cert.py — verify the CA’s signature on a certificate.

Step by step:

1. Get the CA’s *public* key out of the CA certificate.
2. Make sure it is an elliptic-curve key (the project only uses ECDSA).
3. Call `.verify(...)` with three things:
 - `certificate.signature` — the signature bytes the CA produced.
 - `certificate.tbs_certificate_bytes` — “TBS” means *To Be Signed*: the exact body bytes the signature covers.
 - `ec.ECDSA(certificate.signature_hash_algorithm)` — the algorithm to use (ECDSA with the cert’s own hash, SHA-256).
4. If the maths matches, `verify` returns quietly — success. If anything is wrong (forged certificate, edited bytes) it raises `InvalidSignature`, which is caught and turned into a [FAIL] message.

Why this is the heart of the section: `verify` succeeds *only* if the body bytes were signed by the matching private key. An attacker who forges a certificate does not have the CA’s private key, so this call raises `InvalidSignature` — the forgery is rejected. **Theory link:** ECDSA verification + the chain of trust. *Analogy:* the border guard holding the passport under UV light to check the government’s hidden seal.

`check_dates()` — is it inside its validity window?

Purpose: reject certificates that are expired or not yet valid.

```

1 def check_dates(certificate):
2     now = datetime.now(timezone.utc)
3     not_before, not_after = get_validity_window(certificate)
4     if now < not_before:
5         return False, f"[FAIL] Date invalid - not valid until {not_before.date()}"
6
7     if now > not_after:
8         return False, f"[FAIL] Date invalid - expired on {not_after.date()}"
9     return True, f"[OK] Date valid - expires {not_after.date()}"

```

Listing 2.7: ca/verify_cert.py — validity-date check.

Step by step:

1. Get “now” as a timezone-aware UTC time.

2. `get_validity_window` reads the certificate's not-before and not-after dates (it has a small compatibility shim so it works with both old and new versions of the `cryptography` library).
3. If now is before the start date → fail ("not valid yet").
4. If now is after the end date → fail ("expired").
5. Otherwise → pass.

Theory link: certificate validity period. *Analogy:* a guard checking the "expires" line on a passport.

`check_subject()` — who does it belong to?

Purpose: extract and display the subject name. It calls `certificate.subject.rfc4514_string()` to get a readable string like `CN=vehicle-a,0=V2V-Security-Lab`. If there is no subject it fails; otherwise it reports who owns the certificate. This is informational — it tells you *which* vehicle, but does not by itself prove trust (the signature check does that).

`verify_certificate()` — the conductor

It loads both PEM files, runs the three checks, prints each result line, and prints `RESULT: CERTIFICATE IS VALID` only if *all three* passed (returning exit code 0); otherwise `RESULT: CERTIFICATE IS INVALID` and exit code 1.

2.5 Hands-On / Practical

2.5.1 Step 1 — Create the Certificate Authority

```
python ca\ca.py
```

Expected output (sample):

```
[2026-05-17 10:00:00] INFO: Generating CA key pair (ECDSA P-256)...
[2026-05-17 10:00:00] INFO: CA key pair generated.
[2026-05-17 10:00:00] INFO: Creating self-signed root certificate...
[2026-05-17 10:00:00] INFO: Certificate details:
    Subject   : CN=V2V-Root-CA,0=V2V-Security-Lab
    Issuer    : CN=V2V-Root-CA,0=V2V-Security-Lab (self-signed)
    Serial    : 0x3f1a9c...
    Valid From: 2026-05-17
    Valid To   : 2036-05-17
    Key Type   : ECDSA P-256
[2026-05-17 10:00:00] INFO: Saved: ca/certs/ca_cert.pem
[2026-05-17 10:00:00] INFO: Saved: ca/certs/ca_key.pem
[2026-05-17 10:00:00] INFO: CA initialised successfully. Share ca_cert.pem with
    all vehicles.
```

What the lines mean: the Subject and Issuer being identical confirms it is self-signed; "Valid To" is ten years ahead; two PEM files are written.

2.5.2 Step 2 — Issue a certificate for each vehicle

```
python ca\issue_cert.py --vehicle-id vehicle-a --output-dir ca\certs
python ca\issue_cert.py --vehicle-id vehicle-b --output-dir ca\certs
```

Expected output (sample, for vehicle-a):

```
[INFO] Loading CA certificate and key...
[INFO] Generating key pair for vehicle-a...
[INFO] Creating certificate for vehicle-a...
[INFO] Certificate signed by CA.
[INFO] Saved: ca/certs/vehicle-a_cert.pem
[INFO] Saved: ca/certs/vehicle-a_key.pem
[INFO] Fingerprint (SHA-256): a1:b2:c3:d4:e5:f6:...:9a
[INFO] Serial: 0x7c2e54...
```

What the lines mean: the vehicle got its own key pair; the CA signed its certificate; the fingerprint is the SHA-256 of the finished certificate.

2.5.3 Step 3 — Verify a certificate

```
python ca\verify_cert.py --cert ca\certs\vehicle-a_cert.pem --ca-cert ca\certs\
ca_cert.pem
```

Expected output (sample):

```
[CHECKING] ca/certs/vehicle-a_cert.pem
[OK] Signature valid - signed by V2V-Root-CA
[OK] Date valid - expires 2027-05-17
[OK] Subject: CN=vehicle-a, O=V2V-Security-Lab
RESULT: CERTIFICATE IS VALID
```

What the lines mean: all three checks passed — the CA’s signature verified, the dates are current, and the certificate names `vehicle-a`.

2.5.4 Try This Yourself (mini-experiment): forge detection

Open `ca/certs/vehicle-a_cert.pem` in a text editor. It is PEM text — between the BEGIN and END lines is Base64. Change *one* character in the middle (e.g. turn an A into a B), save, and re-run the verify command from Step 3.

Predict before you run. The body bytes no longer match the CA’s signature, so the ECDSA check must fail. Prediction:

```
[FAIL] Signature invalid - NOT signed by trusted CA
RESULT: CERTIFICATE IS INVALID
```

This is exactly what happens to an attacker’s forged or edited certificate. **Restore the file afterwards** by re-running Step 2 for `vehicle-a` (or keep a backup copy first).

2.6 Attack Relevance

S2 is the defence behind **Scenario A of the Man-in-the-Middle attack** and the first half of **impersonation**.

The attack. A man-in-the-middle attacker wants Vehicle B to accept the attacker as “Vehicle A”. The attacker cannot use the real Vehicle A certificate’s private key (they do not have it), so they generate their *own* key pair and either (i) make a self-signed certificate claiming to be `vehicle-a`, or (ii) edit a captured real certificate.

How S2 stops it. Whichever the attacker tries, `check_signature` in `verify_cert.py` (and the equivalent code inside the S4 handshake) calls `ca_public_key.verify(cert.signature, cert.tbs_certificate_bytes, ...)`:

- A **self-signed forgery** was signed by the attacker’s key, not the CA’s → `verify` raises `InvalidSignature` → rejected.

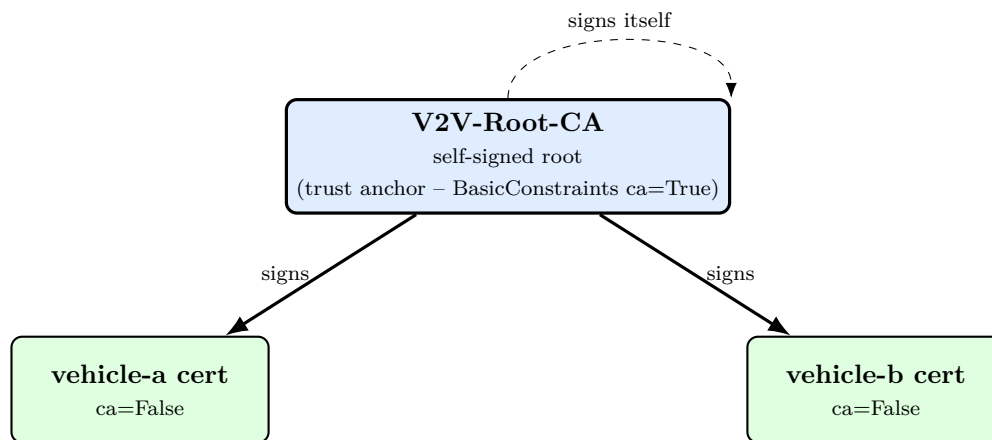
- An **edited real certificate** has different `tbs_certificate_bytes` than when the CA signed it → the signature no longer matches → rejected.

The attacker would need the CA’s private key (`ca_key.pem`) to produce a certificate that passes — and that key never leaves the CA. The `BasicConstraints` extension adds a second wall: even a validly signed *vehicle* certificate has `ca=False`, so it can never be used to sign further certificates.

Honest scope note. S2 stops a *forged identity*. It does not, by itself, stop an attacker who has stolen the real `ca_key.pem`, nor does it prove *liveness* (that the holder is online right now and owns the matching private key). That “prove you hold the private key” step is the challenge-response in the S4 handshake. S2 and S4 together defeat MITM.

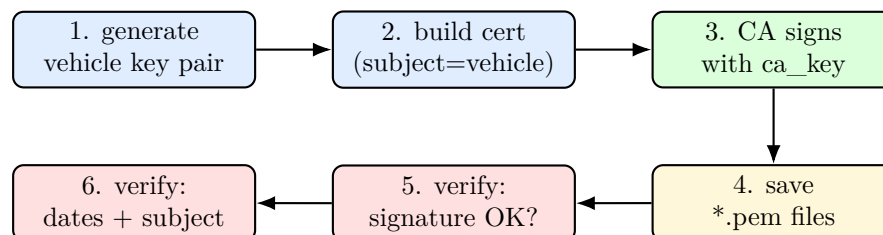
2.7 Illustrations

2.7.1 Diagram 1 — The Chain of Trust



To trust *vehicle-a*, a verifier checks just one signature: “did V2V-Root-CA sign this?” Trust flows down from the root.

2.7.2 Diagram 2 — Issue then Verify



Steps 1–4: `issue_cert.py` Steps 5–6: `verify_cert.py`

2.8 Presentation Pack

2.8.1 Slide Outline

Slide 1 — The Identity Problem.

- Two cars must trust each other before they talk.
- A bare public key is just a number — it does not say *whose*.
- We need a digital “passport” that cannot be faked.
- Solution: a Certificate Authority — a digital passport office.

Slide 2 — The Certificate Authority (`ca.py`).

- The CA generates a P-256 key pair (`generate_ca_key`).
- It builds a *self-signed* root certificate (issuer = subject).
- `BasicConstraints ca=True` — it is allowed to sign others.
- It saves `ca_cert.pem` (public) and `ca_key.pem` (secret).

Slide 3 — Issuing a Vehicle Certificate (`issue_cert.py`).

- The vehicle generates *its own* key pair; the CA never sees the private half.
- The CA builds a cert with subject = vehicle, issuer = CA.
- `builder.sign(private_key=ca_key)` — the CA’s signature is the link in the chain of trust.
- Vehicle cert is `ca=False`, valid one year.

Slide 4 — Verifying a Certificate (`verify_cert.py`).

- Three checks: signature, dates, subject.
- Signature check: `ca_public_key.verify(...)` — the heart of it.
- A forged or edited certificate raises `InvalidSignature`.
- All three must pass → `CERTIFICATE IS VALID`.

Slide 5 — Why This Beats a Forged-Certificate MITM.

- Attacker has no CA private key, so cannot make a cert that verifies.
- Self-signed forgery → wrong signer → rejected.
- Edited real cert → body bytes changed → rejected.
- S2 (identity) + S4 (prove you own the key) together defeat MITM.

2.8.2 Speaker Talking Points (about 30–60 seconds each)

Slide 1. “Before our two cars can have a secure conversation, each one has to know it is really talking to a genuine vehicle and not an impostor. The problem is that a public key on its own is just a long number — it carries no name. So we need something like a passport: a document that ties a key to an identity and that nobody can forge. To create and check those passports we build a Certificate Authority — think of it as a digital passport office that everyone agrees to trust.”

Slide 2. “The file `ca.py` builds that authority. First it generates a key pair on the P-256 elliptic curve — a private key it keeps secret and a public key it will share. Then it builds its own certificate. Because nobody ranks above the CA, it signs that certificate with its own key — that is what ‘self-signed’ means, and you can see it in the code where subject and issuer are the same name. It marks the certificate ‘ca equals true’, meaning this key is allowed to sign other certificates, and saves two files: the public `ca_cert.pem`, which every vehicle gets a copy of, and the secret `ca_key.pem`.”

Slide 3. “Issuing a vehicle certificate is the job of `issue_cert.py`. An important detail: the vehicle generates its *own* key pair — the CA never sees the vehicle’s private key. The CA only takes the vehicle’s public key, wraps it in a certificate that names the vehicle as subject and the CA as issuer, and then signs it with the CA’s private key. That one signing line is the entire purpose of a CA — it is the link in the chain of trust. The vehicle certificate is marked ‘ca equals false’, so a vehicle can never issue certificates itself, and it lasts one year instead of ten.”

Slide 4. “`verify_cert.py` is the border guard. It runs three checks. The date check makes sure the certificate has not expired. The subject check reads out who it belongs to. But the real test is the signature check: it takes the CA’s public key and asks ‘did the matching CA private

key sign this exact certificate body?’. If yes, it passes silently. If the certificate was forged or even slightly edited, the library raises an `InvalidSignature` error and we reject it. Only if all three checks pass do we print `CERTIFICATE IS VALID`.”

Slide 5. “Here is why this defeats a man-in-the-middle who tries a fake certificate. To make a certificate that passes our signature check, the attacker would need the CA’s private key — and that key never leaves the CA. If they self-sign a fake, it was signed by the wrong key and is rejected. If they edit a real certificate, the body bytes change and the old signature no longer matches, so it is rejected too. Section S2 proves *identity*; Section S4 then makes the vehicle prove it actually *owns* the private key. Together they shut the door on man-in-the-middle.”

2.8.3 Live-Demo Script

1. **Build the CA.** Run `python ca\ca.py`. Point at the line where Subject equals Issuer and say “this is self-signed.”
2. **Issue two certificates.** Run the two `issue_cert.py` commands. Point at “Certificate signed by CA” and at the SHA-256 fingerprint.
3. **Verify a good certificate.** Run `verify_cert.py` on `vehicle-a`. Point at the three [OK] lines and `CERTIFICATE IS VALID`.
4. **Break it live.** Open `vehicle-a_cert.pem`, change one Base64 character, save, re-run `verify`. Point at [FAIL] `Signature invalid` — “this is what happens to every forgery.” Then re-issue the certificate to restore it.

2.8.4 Examiner Q&A

1. **Q: What is a Certificate Authority?**
A: A trusted party that issues and signs certificates — a digital passport office. In this project it is created by `ca.py` and is named `V2V-Root-CA`.
2. **Q: What does “self-signed” mean and why is the root self-signed?**
A: The certificate’s issuer and subject are the same, and it is signed with its own private key. The root is self-signed because nothing ranks above it; trust in the root must be established out-of-band by distributing `ca_cert.pem`.
3. **Q: Does the CA ever see a vehicle’s private key?**
A: No. The vehicle generates its own key pair in `generate_vehicle_key`; only the *public* key is sent to the CA to be placed in the certificate and signed.
4. **Q: How does `verify_cert.py` detect a forged certificate?**
A: `check_signature` calls `ca_public_key.verify` over `tbs_certificate_bytes`. A forgery was not signed by the CA’s private key, so verification raises `InvalidSignature`.
5. **Q: What are `tbs_certificate_bytes`?**
A: “To Be Signed” — the exact certificate body bytes that the signature covers (everything except the signature itself). Verifying re-hashes these bytes and checks them against the signature.
6. **Q: What do `BasicConstraints` and `KeyUsage` do?**
A: `BasicConstraints ca=True/False` says whether the certificate may sign other certificates. `KeyUsage` limits the key’s job — `key_cert_sign` for the CA, `digital_signature` for a vehicle. Marked critical, so a verifier must honour them.
7. **Q: What is PEM and how does it differ from DER?**
A: DER is the raw binary encoding; PEM is that binary Base64-encoded as text and wrapped in `BEGIN/END` lines so it survives copy-paste and email.

8. **Q: Name one realistic weakness of this CA design.**

A: `ca_key.pem` is stored unencrypted (`NoEncryption()`), so anyone who reads that file controls the whole PKI. A real CA encrypts the key or keeps it in a Hardware Security Module. Also there is no revocation (no CRL/OCSP) — a compromised vehicle certificate cannot be cancelled before it expires.

2.9 Glossary

PKI

Public Key Infrastructure: the system of CAs, certificates and rules that lets strangers trust public keys.

CA

Certificate Authority: the trusted issuer and signer of certificates.

Certificate (X.509)

A standard digital document binding an identity to a public key, signed by an issuer.

Symmetric cryptography

One shared secret key for both locking and unlocking.

Asymmetric / public-key cryptography

A key pair: a private key kept secret and a public key shared freely.

Private key

The secret half of a key pair; used to sign.

Public key

The shareable half of a key pair; used to verify.

ECC

Elliptic-Curve Cryptography: public-key crypto based on curve maths.

P-256 / SECP256R1

The specific NIST elliptic curve used in this project.

SHA-256

A hash function producing a 256-bit fixed-size fingerprint.

Hash function

A one-way function turning any data into a short fingerprint.

Digital signature

A value made with a private key and checkable with the public key, proving authorship and integrity.

ECDSA

Elliptic Curve Digital Signature Algorithm.

Self-signed

A certificate whose issuer and subject are the same; signed with its own key.

Chain of trust

The line of signatures from a certificate back to a trusted root.

Trust anchor / root

The already-trusted certificate at the top of a chain.

Subject

Who a certificate is about.

Issuer

Who signed a certificate.

Serial number

A unique number identifying a certificate.

Fingerprint

A SHA-256 hash of a whole finished certificate, for human comparison.

BasicConstraints

Extension stating whether a certificate is a CA.

KeyUsage

Extension stating what a key is allowed to do.

Critical extension

An extension a verifier must understand or reject the cert.

Validity period

The not-before / not-after dates of a certificate.

DER

The compact binary encoding of a certificate or key.

Base64

A text encoding of binary data using 64 printable characters.

PEM

A text format: Base64 wrapped in BEGIN/END header lines.

tbs_certificate_bytes

The “to-be-signed” body bytes a signature covers.

HSM

Hardware Security Module: tamper-proof hardware for storing keys.

CRL / OCSP

Mechanisms for revoking (cancelling) a certificate early — not implemented in this project.

2.10 References and Further Study

- **Public Key Infrastructure (Wikipedia)** — https://en.wikipedia.org/wiki/Public_key_infrastructure
Read for: the big picture of CAs, certificates and trust.
- **X.509 (Wikipedia)** — <https://en.wikipedia.org/wiki/X.509>
Read for: the certificate format, fields and extensions.
- **RFC 5280 — Internet X.509 Certificate Profile** — <https://datatracker.ietf.org/doc/html/rfc5280>
Read for: the authoritative definition of X.509 certificates and BasicConstraints / KeyUsage.
- **Public-key cryptography (Wikipedia)** — https://en.wikipedia.org/wiki/Public-key_cryptography
Read for: how key pairs work and why public keys are safe to share.
- **Elliptic-curve cryptography (Wikipedia)** — https://en.wikipedia.org/wiki/Elliptic-curve_cryptography
Read for: a gentle introduction to ECC.

- **ECDSA (Wikipedia)** — https://en.wikipedia.org/wiki/Elliptic_Curve_Digital_Signature_Algorithm
Read for: how elliptic-curve signatures sign and verify.
- **NIST FIPS 186-5 — Digital Signature Standard** — <https://csrc.nist.gov/pubs/fips/186-5/final>
Read for: the official standard behind ECDSA and the P-256 curve.
- **SHA-2 / SHA-256 (Wikipedia)** — <https://en.wikipedia.org/wiki/SHA-2>
Read for: how the hash function used in signing works.
- **PEM / DER explained (Python cryptography docs)** — <https://cryptography.io/en/latest/x509/>
Read for: how this project’s exact library handles certificates and PEM.
- **Python cryptography — X.509 reference** — <https://cryptography.io/en/latest/x509/reference/>
Read for: `CertificateBuilder`, `load_pem_x509_certificate`, and every API used in S2.
- **Computerphile — “Public Key Cryptography” (video)** — https://www.youtube.com/watch?v=GSIDS_1vRv4
Read/watch for: an intuitive, visual explanation of key pairs.

End of Section S2. Next: S3 — the Cryptographic Core (AES-GCM, ECDH, HKDF, ECDSA).

Chapter 3

S3 — The Cryptographic Core

3.1 Title and “What You’ll Learn”

Section Title

S3 — The Cryptographic Core: the toolbox of encryption, key exchange, key derivation and signing that the rest of the project is built from.

Plain-English Summary (read this first)

Section S3 is a single file, `crypto/crypto.py`, that contains no networking and no protocol logic — it is purely a *toolbox* of cryptographic functions. Four tools live here: **AES-256-GCM** locks a message in a tamper-evident box; **ECDH** lets two cars agree on a shared secret without ever sending it; **HKDF** turns that raw secret into a clean, usable key; and **ECDSA** stamps a message with an unforgeable signature. Every later section — the handshake (S4), the secure messaging (S5) — simply calls these functions. Understand this file and the cryptography of the whole project is in your hands.

Learning Outcomes

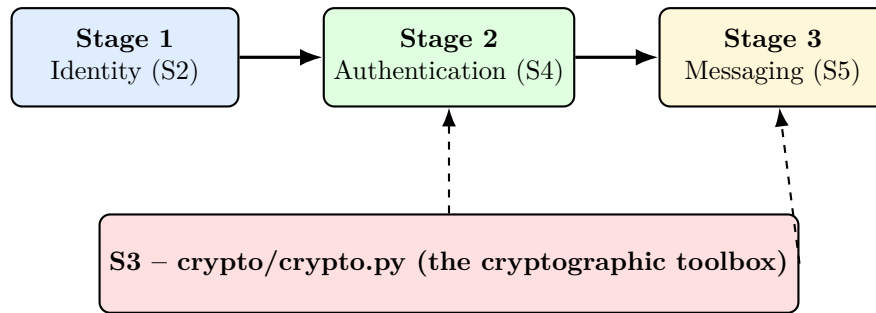
After studying this section you will be able to:

- Explain **symmetric encryption** and what **AES** is.
- Explain **authenticated encryption** and why **AES-256-GCM** gives confidentiality *and* integrity at once.
- Explain the role of the **nonce** and the **authentication tag**.
- Explain **ECDH** key exchange and the Diffie–Hellman idea.
- Explain **HKDF** key derivation and why a raw shared secret is not used directly as a key.
- Explain **ECDSA** signing and verifying.
- Walk through every function in `crypto.py`, naming each argument, return value and exception.
- Run the built-in five-test demonstration and explain every output line.
- Explain how AES-GCM defeats the tamper attack.

3.2 Where This Fits

S3 sits *underneath* Stage 2 of the pipeline. It is the engine room. The handshake in S4 (`auth_protocol.py`) imports seven functions directly from `crypto.py`; the secure-messaging code

in S5 uses the AES-GCM and ECDSA functions for every BSM. S3 itself produces nothing visible — it is the shared library that makes Stages 2 and 3 possible.



3.3 The Theory You Must Study

3.3.1 Symmetric Encryption and AES

Plain definition. *Symmetric encryption* scrambles a message (*plaintext*) into unreadable *ciphertext* using a secret key, and the *same* key unscrambles it. **AES** (Advanced Encryption Standard) is the worldwide-standard symmetric cipher. “AES-256” means the key is 256 bits (32 bytes) long.

Real-life analogy. A lockbox with one key. Whoever holds the key can both lock things in and unlock them. The box without the key is just an opaque brick — you cannot see what is inside.

Why it exists. Symmetric encryption is *fast* — fast enough to encrypt every safety message a car sends. (Public-key crypto, by contrast, is slow; that is why it is used only for the handshake, not for every BSM.)

How it works here. The project uses AES with a 256-bit key (`AES_KEY_LENGTH = 32`). The key itself comes from the handshake (S4). The function `aes_gcm_encrypt` enforces the size: “key must be exactly 32 bytes”.

Security property it provides. **Confidentiality.**

3.3.2 Authenticated Encryption and AES-256-GCM

Plain definition. *Authenticated encryption* does two jobs in one operation: it hides the message (confidentiality) *and* produces a small checksum-like value that detects any change (integrity). **GCM** (Galois/Counter Mode) is the “mode” that adds the integrity job to AES.

Real-life analogy. Tamper-evident packaging on medicine. The box hides the pills (confidentiality) *and* carries a seal that visibly breaks if anyone opened it (integrity). You get both protections from one package.

Why it exists. Plain encryption hides content but does *not* stop an attacker from flipping bits in the ciphertext. The receiver would happily decrypt the corrupted message. GCM closes this hole by refusing to decrypt if the message was changed.

How it works here. `aes_gcm_encrypt` returns three things:

- the **ciphertext** — the scrambled message;
- the **nonce** — a 12-byte one-time value;
- the **authentication tag** — a 16-byte (128-bit) “seal” computed over the whole ciphertext.

On decryption, GCM recomputes the tag and compares. If even one bit changed, the tags differ and the library raises `InvalidTag` — the code turns this into a `DecryptionError`.

Security property it provides. **Confidentiality** and **integrity** together. AES-256-GCM is the same authenticated cipher used by TLS 1.3.

3.3.3 The Nonce

Plain definition. A *nonce* (“number used once”) is a value used exactly once with a given key. AES-GCM needs a fresh 12-byte nonce for every single encryption.

Real-life analogy. A single-use serial number on every parcel. Two parcels must never carry the same number.

Why it exists. If you encrypt two different messages with the *same* key *and* the same nonce, AES-GCM’s security collapses — an attacker can learn relationships between the messages and even forge the tag. The nonce makes each encryption unique.

How it works here. `aes_gcm_encrypt` calls `os.urandom(12)` — 12 cryptographically random bytes — for every message. With 96 random bits the chance of an accidental repeat is negligible. The nonce is *not* secret; it is sent alongside the ciphertext because the receiver needs it to decrypt.

Security property it provides. It *protects* confidentiality and integrity by keeping every encryption unique.

3.3.4 ECDH — Elliptic-Curve Diffie–Hellman Key Exchange

Plain definition. ECDH is a method by which two parties, each with their own key pair, can compute the *same* shared secret number — and an eavesdropper who sees both public keys still cannot compute it.

Real-life analogy (the paint analogy). Alice and Bob each start with a secret paint colour. They mix their secret into a public colour and swap the mixtures in the open. Each then mixes their own secret into the mixture they received. Both end up with the identical final colour — yet an eavesdropper who saw only the swapped mixtures cannot un-mix them to find it.

Why it exists. It solves the key-distribution problem: two cars that have never met can agree on a secret AES key over a channel the attacker is watching, *without ever transmitting the key*.

How it works here (the mathematics, gently). On the P-256 curve there is a fixed public point G . A private key is a secret number a ; the matching public key is the point $a \cdot G$ (computed by “hopping” G along the curve a times). The shared-secret magic is:

$$a \cdot (b \cdot G) = ab \cdot G = b \cdot (a \cdot G).$$

Vehicle A computes $a \cdot (B\text{'s public})$; Vehicle B computes $b \cdot (A\text{'s public})$; both get $ab \cdot G$. An attacker sees $a \cdot G$ and $b \cdot G$ but cannot recover a or b (the “elliptic-curve discrete logarithm problem” is infeasible). In code this is one line: `my_private_key.exchange(ec.ECDH(), peer_public_key)`.

Security property it provides. It is the basis of **confidentiality** and, because the project uses *fresh* ECDH keys each session, of **forward secrecy**.

3.3.5 Ephemeral Keys and Forward Secrecy

Plain definition. An *ephemeral* key is generated fresh for one session and thrown away afterwards. **Forward secrecy** is the guarantee that recording today’s encrypted traffic and stealing a key *tomorrow* does not let the attacker decrypt today’s traffic.

Real-life analogy. A self-destructing one-time code. Even if someone later finds your code notebook, today’s code is already gone, so today’s messages stay locked forever.

How it works here. `generate_ecdh_keypair` creates a brand-new P-256 key pair every time it is called, and the handshake calls it once per session. The comment in the code says it plainly: “even if an old key leaks later, past sessions can’t be decrypted because this key no longer exists.”

Security property it provides. **Forward secrecy.**

3.3.6 HKDF — Turning a Raw Secret into a Real Key

Plain definition. HKDF (HMAC-based Key Derivation Function) is a function that takes a rough, possibly non-uniform secret and produces a clean, fixed-length, uniformly random key.

Real-life analogy. The raw ECDH output is like muddy river water — there is good water in it, but it is not fit to drink as-is. HKDF is the filtration plant: in goes the rough secret, out comes clean, drinkable, standard-sized water (a 32-byte AES key).

Why it exists. The raw number ECDH produces is secret, but its bits are *not* perfectly uniform, and you often want a key of an exact size. Feeding the raw secret straight into AES is poor practice. HKDF fixes both issues. It also lets you mix in extra public data so the final key is unique to this session.

How it works here. `derive_session_key` calls HKDF with:

- `algorithm = SHA256` — the hash HKDF is built on;
- `length = 32` — produce exactly a 32-byte AES-256 key;
- `salt = nonce_a + nonce_b` — the two handshake nonces joined, making the key unique to this session;
- `info = b"v2v-session-key-v1"` — a label tying the key to this exact purpose.

The same raw secret plus the same two nonces always yields the same 32-byte key — which is why both vehicles end up with an identical session key.

Security property it provides. It strengthens **confidentiality** by producing a sound key, and ties the key to this session.

3.3.7 SHA-256 Hashing

Plain definition. A hash function turns any input into a short, fixed-size fingerprint (SHA-256 → 256 bits). Same input, same fingerprint; any change, completely different fingerprint; you cannot run it backwards.

Why it exists here. Two of S3's tools rely on it: HKDF is built on SHA-256, and ECDSA signs the SHA-256 hash of the message rather than the whole message.

Security property it provides. Supports **integrity**.

3.3.8 ECDSA — Digital Signatures

Plain definition. ECDSA produces a signature with a *private* key that anyone with the matching *public* key can verify. It proves *who* created the message and that it has *not changed*.

Real-life analogy. A handwritten signature that is impossible to forge and that also changes if the document is edited — so the signature both identifies the author and protects the text.

Why it exists. Encryption hides a message but does not prove who sent it. A signature proves authorship, so the sender cannot later deny it.

How it works here. `ecdsa_sign` calls `private_key.sign(message, ec.ECDSA(SHA256()))`; `ecdsa_verify` calls `public_key.verify(signature, message, ec.ECDSA(SHA256()))`. The signature is returned in **DER** encoding — a compact standard binary format.

Security property it provides. **Authentication, integrity, non-repudiation.**

3.3.9 PEM Serialization of Keys

Plain definition. *Serialization* means turning a key object in memory into bytes that can be saved or sent; *loading* is the reverse. PEM is the text format (Base64 between BEGIN/END lines) used.

Why it exists here. During the handshake a vehicle must *send* its ephemeral ECDH public key to the other vehicle. You cannot send a Python object over a network — you send bytes.

`serialize_public_key` produces those bytes; `load_public_key` rebuilds the key object on the other side.

Security property it provides. None directly — it is about *format*. (A public key is meant to be shared, so there is nothing secret to protect here.)

3.4 Code Walkthrough

`crypto/crypto.py` is organised as: three custom exceptions, a tiny type-checking helper, then the four tool groups (AES-GCM, ECDH+HKDF, ECDSA, PEM), then a built-in demonstration.

3.4.1 The custom exceptions and the type guard

```
1 class CryptoError(Exception):
2     """Base exception for cryptographic errors in this simulation."""
3
4 class DecryptionError(CryptoError):
5     """Raised when AES-GCM authentication fails during decryption."""
6
7 class SignatureError(CryptoError):
8     """Raised when signature-related operations fail."""
9
10 def _ensure_bytes(value: bytes, name: str) -> None:
11     if not isinstance(value, bytes):
12         raise TypeError(f"{name} must be bytes")
```

Listing 3.1: `crypto.py` — custom exceptions and the bytes guard.

Explanation:

1. `CryptoError` is the *parent* of all crypto errors, so other code can catch every crypto problem with one `except`.
2. `DecryptionError` specifically means “the message failed its integrity check” — i.e. possible tampering.
3. `SignatureError` means a signing operation failed.
4. `_ensure_bytes` is a small helper: cryptographic functions must be given raw `bytes`, not text strings. The leading underscore is a Python convention meaning “private — internal use only”.

3.4.2 `aes_gcm_encrypt()` — lock a message

Purpose: encrypt a message with AES-256-GCM, returning ciphertext, nonce and tag.

```
1 def aes_gcm_encrypt(key: bytes, plaintext: bytes) -> tuple[bytes, bytes, bytes]:
2     _ensure_bytes(key, "key")
3     _ensure_bytes(plaintext, "plaintext")
4
5     if len(key) != 32:
6         raise ValueError("key must be exactly 32 bytes for AES-256-GCM")
7
8     nonce = os.urandom(12)
9     aesgcm = AESGCM(key)
10    encrypted_blob = aesgcm.encrypt(nonce, plaintext, None)
11    ciphertext = encrypted_blob[:-16]
12    auth_tag = encrypted_blob[-16:]
13    return ciphertext, nonce, auth_tag
```

Listing 3.2: `crypto.py` — AES-256-GCM encryption.

Step by step:

1. Check both arguments are **bytes**.
2. Check the key is exactly 32 bytes — that is what makes it AES-256. A wrong size raises **ValueError**.
3. `os.urandom(12)` draws a fresh 12-byte random nonce — new for every call.
4. `AESGCM(key)` creates the cipher object bound to that key.
5. `aesgcm.encrypt(nonce, plaintext, None)` encrypts. The third argument is the *associated data* (extra data to authenticate but not encrypt); **None** means none is used here. The library returns a single blob: **ciphertext** followed by the 16-byte tag.
6. `encrypted_blob[:-16]` is everything except the last 16 bytes — the ciphertext. `encrypted_blob[-16:]` is the last 16 bytes — the authentication tag. The code splits them so callers can store them separately.

Arguments: `key` (32 bytes), `plaintext` (bytes). **Returns:** a 3-tuple (`ciphertext`, `nonce`, `auth_tag`). **Exceptions:** **TypeError** (not bytes), **ValueError** (key not 32 bytes). **Theory link:** authenticated encryption, the nonce, the tag. *Analogy:* sealing the message in tamper-evident packaging and printing a fresh serial number (nonce) on it.

3.4.3 `aes_gcm_decrypt()` — unlock and verify

Purpose: decrypt AES-GCM ciphertext *and* verify it was not tampered with.

```
1 def aes_gcm_decrypt(key, ciphertext, nonce, tag) -> bytes:
2     _ensure_bytes(key, "key"); _ensure_bytes(ciphertext, "ciphertext")
3     _ensure_bytes(nonce, "nonce"); _ensure_bytes(tag, "tag")
4
5     if len(key) != 32: raise ValueError("key must be exactly 32 bytes ...")
6     if len(nonce) != 12: raise ValueError("nonce must be exactly 12 bytes ...")
7     if len(tag) != 16: raise ValueError("tag must be exactly 16 bytes ...")
8
9     encrypted_blob = ciphertext + tag
10    aesgcm = AESGCM(key)
11    try:
12        return aesgcm.decrypt(nonce, encrypted_blob, None)
13    except InvalidTag as exc:
14        raise DecryptionError(
15            "Message authentication failed - possible tampering detected"
16        ) from exc
```

Listing 3.3: `crypto.py` — AES-256-GCM decryption with tamper detection.

Step by step:

1. Check all four arguments are **bytes**.
2. Check the exact lengths: key 32, nonce 12, tag 16. Any wrong size raises **ValueError** — a clear “programmer error” rather than a security event.
3. Re-join `ciphertext + tag` into the single blob the library expects (the opposite of the split done during encryption).
4. `aesgcm.decrypt(...)` does two things in one step: it decrypts and it re-computes the tag and compares it.
5. If the tag matches, it returns the original plaintext. If the ciphertext, nonce or tag was altered, the library raises **InvalidTag**; the code catches it and re-raises a friendlier **DecryptionError**.

The crucial idea: a tampered message does *not* decrypt to garbage — it *refuses to decrypt at all*. There is no way to “ignore” a bad tag. **Theory link:** authenticated encryption providing integrity. *Analogy:* the pharmacist seeing the broken seal and refusing to hand over the medicine.

3.4.4 generate_ecdh_keypair() — a fresh key pair per session

```
1 def generate_ecdh_keypair():
2     # We generate a NEW key pair each session (ephemeral).
3     # This provides "Forward Secrecy".
4     private_key = ec.generate_private_key(ec.SECP256R1(), default_backend())
5     public_key = private_key.public_key()
6     return private_key, public_key
```

Listing 3.4: crypto.py — generate an ephemeral ECDH key pair.

Step by step: create a new P-256 private key; derive its public key; return both. There are no arguments. **Returns:** a tuple (private_key, public_key). **Theory link:** ephemeral keys and forward secrecy — because it is called fresh for each session, an old key simply does not exist any more.

3.4.5 ecdh_shared_secret() — compute the shared number

```
1 def ecdh_shared_secret(my_private_key, peer_public_key) -> bytes:
2     if not isinstance(my_private_key, ec.EllipticCurvePrivateKey):
3         raise TypeError("my_private_key must be an EllipticCurvePrivateKey")
4     if not isinstance(peer_public_key, ec.EllipticCurvePublicKey):
5         raise TypeError("peer_public_key must be an EllipticCurvePublicKey")
6     try:
7         return my_private_key.exchange(ec.ECDH(), peer_public_key)
8     except Exception as exc:
9         raise CryptoError("failed to compute ECDH shared secret") from exc
```

Listing 3.5: crypto.py — ECDH shared-secret computation.

Step by step:

1. Confirm the first argument is a private key and the second is a public key.
2. my_private_key.exchange(ec.ECDH(), peer_public_key) performs the elliptic-curve multiplication $a \cdot (b \cdot G)$ and returns the raw shared-secret bytes.
3. Any failure is wrapped as a **CryptoError**.

Key fact: if Vehicle A calls this with (A-private, B-public) and Vehicle B calls it with (B-private, A-public), *both get the identical bytes*. **Returns:** the raw shared secret (bytes). **Theory link:** ECDH key exchange. *Analogy:* both painters finishing with the same final colour.

3.4.6 derive_session_key() — raw secret to clean AES key

```
1 def derive_session_key(shared_secret, nonce_a, nonce_b) -> bytes:
2     _ensure_bytes(shared_secret, "shared_secret")
3     _ensure_bytes(nonce_a, "nonce_a")
4     _ensure_bytes(nonce_b, "nonce_b")
5
6     hkdf = HKDF(
7         algorithm=hashes.SHA256(),
8         length=32,
9         salt=nonce_a + nonce_b,
10        info=b"v2v-session-key-v1",
11        backend=default_backend(),
12    )
```

```
13 return hkdf.derive(shared_secret)
```

Listing 3.6: crypto.py — HKDF key derivation.

Step by step:

1. Check all three arguments are bytes.
2. Build an HKDF object: SHA-256 as the base hash; `length=32` so the output is exactly an AES-256 key; `salt` is the two session nonces joined together; `info` is the fixed purpose label.
3. `hkdf.derive(shared_secret)` runs the raw ECDH secret through HKDF and returns the clean 32-byte key.

Why `salt = nonce_a + nonce_b`? The raw ECDH secret could repeat if the same ephemeral keys were ever reused; mixing in two fresh random nonces guarantees the *derived* key is unique to this session. **Honest note:** the string `b"v2v-session-key-v1"` is written directly here even though `config.py` already defines `SESSION_KEY_INFO` with the same value — a small duplication worth mentioning if asked. **Returns:** a 32-byte key. **Theory link:** HKDF. *Analogy:* the filtration plant turning muddy water into clean water.

3.4.7 `ecdsa_sign()` and `ecdsa_verify()` — digital signatures

```
1 def ecdsa_sign(private_key, message: bytes) -> bytes:
2     if not isinstance(private_key, ec.EllipticCurvePrivateKey):
3         raise TypeError("private_key must be an EllipticCurvePrivateKey")
4     _ensure_bytes(message, "message")
5     try:
6         return private_key.sign(message, ec.ECDSA(hashes.SHA256()))
7     except Exception as exc:
8         raise SignatureError("failed to create ECDSA signature") from exc
9
10 def ecdsa_verify(public_key, message: bytes, signature: bytes) -> bool:
11     if not isinstance(public_key, ec.EllipticCurvePublicKey):
12         raise TypeError("public_key must be an EllipticCurvePublicKey")
13     _ensure_bytes(message, "message")
14     _ensure_bytes(signature, "signature")
15     try:
16         public_key.verify(signature, message, ec.ECDSA(hashes.SHA256()))
17         return True
18     except InvalidSignature:
19         return False
```

Listing 3.7: crypto.py — ECDSA signing and verification.

`ecdsa_sign` — step by step:

1. Confirm the key is a private key and the message is bytes.
2. `private_key.sign(message, ec.ECDSA(SHA256()))` hashes the message with SHA-256 and signs that hash. It returns the signature in DER encoding.
3. Any failure becomes a `SignatureError`.

`ecdsa_verify` — step by step:

1. Confirm the key is a public key and the other two arguments are bytes.
2. `public_key.verify(signature, message, ec.ECDSA(SHA256()))` re-hashes the message and checks the signature against it.
3. If valid, `verify` returns silently and the function returns `True`. If invalid it raises `InvalidSignature`, which is caught and turned into `False`.

Design note: `ecdsa_verify` returns a plain `True/False` rather than raising — this makes it easy for the handshake code to write `if not ecdsa_verify(...)`. **Theory link:** ECDSA, authentication, integrity, non-repudiation. *Analogy:* pressing the wax seal (sign) and inspecting it (verify).

3.4.8 `serialize_public_key()`, `load_public_key()`, `load_private_key()`

These three are the PEM bridge between in-memory key objects and bytes.

```
1 def serialize_public_key(pub_key) -> bytes:
2     if not isinstance(pub_key, ec.EllipticCurvePublicKey):
3         raise TypeError("pub_key must be an EllipticCurvePublicKey")
4     return pub_key.public_bytes(
5         encoding=serialization.Encoding.PEM,
6         format=serialization.PublicFormat.SubjectPublicKeyInfo,
7     )
8
9 def load_public_key(pem_data: bytes) -> ec.EllipticCurvePublicKey:
10     _ensure_bytes(pem_data, "pem_data")
11     try:
12         public_key = serialization.load_pem_public_key(pem_data, backend=
13             default_backend())
14     except Exception as exc:
15         raise ValueError("invalid PEM public key data") from exc
16     if not isinstance(public_key, ec.EllipticCurvePublicKey):
17         raise ValueError("loaded key is not an elliptic-curve public key")
18     return public_key
```

Listing 3.8: `crypto.py` — PEM serialization helpers.

Explanation:

1. `serialize_public_key` turns a public-key object into PEM text bytes (so it can be put inside a handshake message and sent).
2. `load_public_key` parses PEM bytes back into a key object, and rejects anything that is not a valid elliptic-curve public key.
3. `load_private_key` (not shown, same shape) parses a PEM private key with `password=None`, matching the un-encrypted key files from S2.

Theory link: PEM serialization. *Analogy:* flattening a 3-D object into a printed sheet so it can be posted, then folding it back up on arrival.

3.4.9 The built-in demonstration (`if __name__ == "__main__"`)

At the bottom of the file is a five-test self-demonstration that runs only when the file is executed directly. It is excellent for learning because it exercises every tool: Test 1 proves ECDH gives both sides the same secret; Test 2 derives a session key; Test 3 encrypts then decrypts a message; Test 4 signs and verifies (and shows verification *fails* with the wrong key); Test 5 flips one bit of ciphertext and confirms the tampered message is rejected. We run it in the next section.

3.5 Hands-On / Practical

3.5.1 Run the cryptographic core by itself

`crypto.py` needs no certificates and no network — just run it:

```
python crypto\crypto.py
```

Expected output (sample). Note: where this guide shows `[OK]`, the real terminal prints a small check-mark character, because the source uses the escape `\u2713`.

```

=== CRYPTO MODULE DEMONSTRATION ===

[TEST 1] ECDH Key Exchange
Vehicle A secret == Vehicle B secret: True

[TEST 2] Session Key Derivation
Session key (hex): 9f3c1a...e7b2    (64 hex characters)
Session key length: 32 bytes

[TEST 3] AES-GCM Encrypt/Decrypt
Decrypted: Hello from Vehicle A! [OK]

[TEST 4] ECDSA Sign/Verify
Signature valid with matching key: True
Signature valid with wrong key: False

[TEST 5] Tamper Detection
Tampered message correctly rejected [OK]

```

3.5.2 What each output line means

- **Test 1 — True:** the two vehicles, using only each other’s *public* keys, computed the *same* secret. ECDH works.
- **Test 2 — 32 bytes:** HKDF turned the raw secret into exactly a 256-bit AES key. The hex string is 64 characters because each byte is two hex digits.
- **Test 3 — “Decrypted: Hello from Vehicle A!”:** encrypt then decrypt round-trips perfectly — confidentiality with no data loss.
- **Test 4 — True then False:** the signature verifies with the correct public key, but *fails* with the wrong key. That “False” is exactly how impersonation is detected.
- **Test 5 — “rejected”:** one bit of ciphertext was flipped; decryption refused. That is the tamper defence working.

3.5.3 Try This Yourself (mini-experiment): break the tag

Test 5 already flips a *ciphertext* bit. Try flipping the *tag* instead. Add a few lines to the demo (or open a Python shell):

```

ct, nc, tag = aes_gcm_encrypt(session_key, b"hello")
bad_tag = bytearray(tag)
bad_tag[0] ^= 0x01                                # flip one bit of the tag
aes_gcm_decrypt(session_key, ct, nc, bytes(bad_tag))

```

Predict before you run. The tag is part of what GCM checks, so a changed tag is just as fatal as changed ciphertext. Prediction: a `DecryptionError` (“possible tampering detected”) is raised. Lesson: the nonce, the ciphertext *and* the tag are all protected — you cannot edit any of them and still decrypt.

Second experiment: keep the ciphertext and tag correct but change the *nonce* by one bit. Predict: still a `DecryptionError`, because the tag was computed for the original nonce.

3.6 Attack Relevance

S3 is the *toolbox*, so it supplies the defence used against *three* of the four attacks. The defences are exercised directly by `attacks/tamper_attack.py`.

Attack	How an S3 tool stops it	The function
Tamper	Any change to ciphertext, nonce or tag makes the GCM tag mismatch; decryption refuses.	<code>aes_gcm_decrypt</code> (raises <code>DecryptionError</code>).
Eavesdrop	The message is AES-256 ciphertext; without the session key it is unreadable noise.	<code>aes_gcm_encrypt</code> .
Impersonation	A forged message will not carry a signature that verifies under the real vehicle's public key.	<code>ecdsa_verify</code> (returns <code>False</code>).

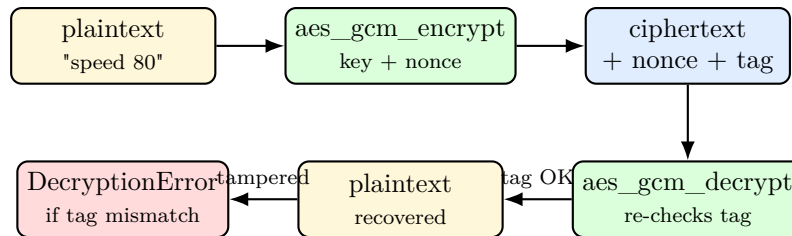
Table 3.1: Which S3 tool defends against which attack.

The tamper attack in detail. The attacker captures an encrypted BSM and flips bits hoping to change “speed 80” into something dangerous. But the AES-GCM tag is computed over the *whole* ciphertext. The receiver’s call to `aes_gcm_decrypt` re-computes the tag, sees a mismatch, and raises `DecryptionError` instead of returning data. The attacker cannot fix the tag because they do not have the session key. **Forward secrecy** is also an S3 contribution: `generate_ecdh_keypair` makes a throw-away key each session, so a key stolen later cannot decrypt recorded traffic.

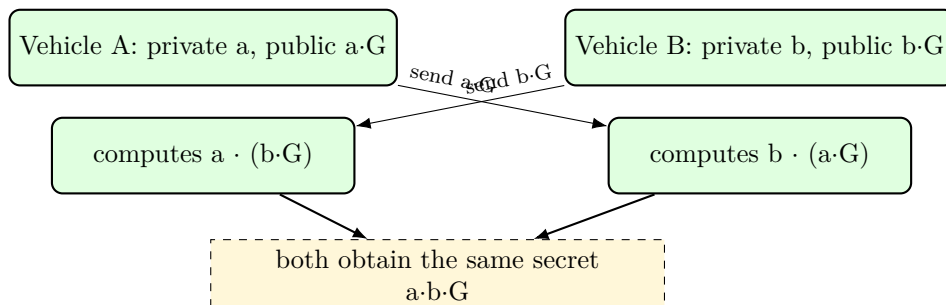
Honest scope note. S3 provides the *primitives*; it does not by itself decide *when* to use them or check timestamps and nonces — that sequencing is the protocol logic in S4 and S5. A primitive used incorrectly (for example, reusing a nonce) would still be insecure; S3 just makes correct use possible.

3.7 Illustrations

3.7.1 Diagram 1 — AES-256-GCM Encrypt then Decrypt

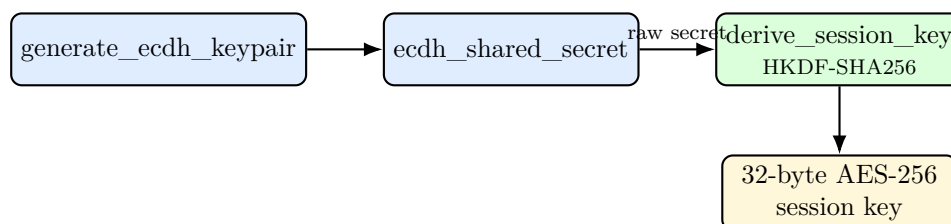


3.7.2 Diagram 2 — ECDH gives both sides the same secret



An eavesdropper sees only $a \cdot G$ and $b \cdot G$ – not enough to find $a \cdot b \cdot G$.

3.7.3 Diagram 3 — The S3 pipeline that builds a session key



3.8 Presentation Pack

3.8.1 Slide Outline

Slide 1 — The Crypto Toolbox.

- `crypto.py` is a library, not a program — four tools.
- AES-256-GCM: lock + tamper seal. ECDH: agree a secret. HKDF: clean the secret. ECDSA: sign.
- Every later section calls these functions.
- Industry-standard — the same set used by TLS 1.3.

Slide 2 — AES-256-GCM: Authenticated Encryption.

- Encrypt returns three things: ciphertext, nonce, 16-byte tag.
- The tag is a seal over the whole ciphertext.
- Decrypt re-checks the tag; one changed bit → `DecryptionError`.
- Gives confidentiality AND integrity in one step.

Slide 3 — ECDH + HKDF: Building a Shared Key.

- ECDH: two cars compute the same secret without sending it.
- Maths: $a \cdot (b \cdot G) = b \cdot (a \cdot G)$.
- Keys are ephemeral → forward secrecy.
- HKDF filters the raw secret into a clean 32-byte AES key.

Slide 4 — ECDSA: Digital Signatures.

- Sign with the private key, verify with the public key.
- Proves who wrote it (authentication) and that it is unchanged (integrity).
- `ecdsa_verify` returns `True/False`.
- Wrong key → `False` → impersonation detected.

Slide 5 — Live Proof: the Five-Test Demo.

- Test 1: ECDH secrets match. Test 2: 32-byte key derived.
- Test 3: encrypt/decrypt round-trips.
- Test 4: signature good with right key, fails with wrong key.
- Test 5: a flipped bit is rejected — the tamper defence.

3.8.2 Speaker Talking Points (about 30–60 seconds each)

Slide 1. “This section is one file, `crypto.py`, and it is unusual — it contains no networking and no protocol. It is purely a toolbox of four cryptographic tools. AES-256-GCM is our lockbox with a built-in tamper seal. ECDH lets two cars secretly agree on a key. HKDF cleans that key up. And ECDSA is our unforgeable signature. Everything else in the project — the handshake, the secure messaging — just calls these functions. And these are not toy algorithms: this is the exact same set of building blocks used by TLS 1.3, the security behind every HTTPS website.”

Slide 2. “AES-256-GCM is authenticated encryption. When you encrypt, you get back three things: the ciphertext, a nonce, and a 16-byte authentication tag. Think of the tag as a wax seal stamped over the whole ciphertext. When you decrypt, the library does not just unscramble — it recomputes the tag and compares. If even one bit of the ciphertext, the nonce or the tag was changed, the tags disagree and decryption is refused; our code raises a `DecryptionError`. So with one operation we get confidentiality — the content is hidden — and integrity — any change is caught.”

Slide 3. “How do two cars that have never met get a shared secret key over a channel an attacker is watching? That is ECDH. Each car has a private number and a public point. They swap public points in the open. Then each mixes its own private number with the other’s public point. Thanks to the curve maths — $a \text{ times } b\text{-times-}G \text{ equals } b \text{ times } a\text{-times-}G$ — both arrive at the identical secret, while an eavesdropper who saw only the public points cannot compute it. And because we generate a fresh ECDH key for every session and throw it away, a key stolen tomorrow cannot unlock today’s traffic — that is forward secrecy. Finally HKDF takes that raw secret, which is not perfectly uniform, and filters it into a clean 32-byte AES key.”

Slide 4. “ECDSA is our digital signature. You sign with your private key; anyone can verify with your public key. A signature proves two things at once: who created the message, and that it has not been changed since. In the code, `ecdsa_verify` simply returns `True` or `False`. And here is the security point — if an impostor signs a message with the wrong key, verification returns `False`. That single `False` is how we catch impersonation.”

Slide 5. “The best part is that `crypto.py` proves itself. Run it directly and it executes five tests. Test one shows the two ECDH secrets are equal. Test two shows HKDF produced a 32-byte key. Test three encrypts and decrypts a message and recovers it exactly. Test four signs a message and shows verification succeeds with the right key and fails with the wrong one. And test five flips a single bit of ciphertext and shows the message is rejected. Five tests, every tool, all visible on screen.”

3.8.3 Live-Demo Script

1. **Run the toolbox.** Execute `python crypto\crypto.py`. Point at `[TEST 1] ... True` — “both cars derived the same secret without sending it.”
2. **Point at Test 2.** “32 bytes — that is our AES-256 key.”
3. **Point at Test 4.** “True with the right key, False with the wrong key — that False is impersonation being caught.”
4. **Point at Test 5.** “One bit flipped, message rejected — the tamper defence.”
5. **Optional.** In a Python shell, flip a bit of the *tag* (see the mini-experiment) and show the same `DecryptionError`.

3.8.4 Examiner Q&A

1. **Q: What does the “GCM” in AES-256-GCM add over plain AES?**
A: GCM (Galois/Counter Mode) adds *authentication*: a 16-byte tag computed over the

ciphertext. It turns AES into *authenticated* encryption, giving integrity as well as confidentiality.

2. **Q: Why must the nonce never repeat for the same key?**

A: Reusing a (key, nonce) pair in GCM breaks security — it can leak relationships between messages and allow tag forgery. The code uses a fresh `os.urandom(12)` nonce every time to prevent this.

3. **Q: Is the nonce secret?**

A: No. It is sent alongside the ciphertext because the receiver needs it to decrypt. Security depends on *uniqueness*, not secrecy.

4. **Q: How does ECDH let two cars share a key without sending it?**

A: Each car combines its own private key with the other's public key. The curve maths makes both results equal ($a \cdot b \cdot G$), and an eavesdropper with only the public keys cannot compute it.

5. **Q: Why use HKDF instead of using the ECDH output directly?**

A: The raw ECDH secret is not perfectly uniform and may not be the right size. HKDF extracts and expands it into a clean, fixed-length key, and the nonce salt ties the key to this one session.

6. **Q: What happens in `aes_gcm_decrypt` if a message is tampered with?**

A: The recomputed tag does not match, the library raises `InvalidTag`, and the code re-raises it as `DecryptionError` — no plaintext is ever returned.

7. **Q: Why does `ecdsa_verify` return a boolean instead of raising?**

A: A failed signature is an expected, normal event during an attack, not a programming bug. Returning `False` lets callers write a simple `if not ecdsa_verify(...)` check.

8. **Q: This file is called a “toolbox” — what does it *not* do?**

A: It provides primitives only. It does not check timestamps, manage nonces against replay, or sequence the handshake — that protocol logic lives in S4 and S5. Correct *use* of the primitives is the protocol's responsibility.

3.9 Glossary

Plaintext

The original readable message before encryption.

Ciphertext

The scrambled, unreadable output of encryption.

Symmetric encryption

Encryption using one shared secret key for both directions.

AES

Advanced Encryption Standard: the worldwide-standard symmetric cipher.

AES-256

AES with a 256-bit (32-byte) key.

GCM

Galois/Counter Mode: the AES mode that adds an authentication tag.

Authenticated encryption

Encryption that also detects any tampering.

Nonce

A number used exactly once with a given key.

Authentication tag

The 16-byte value GCM computes to detect tampering.

Associated data (AAD)

Extra data authenticated but not encrypted (here: none).

ECDH

Elliptic-Curve Diffie–Hellman: a shared-secret key-exchange method.

Shared secret

The identical secret value two parties compute via ECDH.

Ephemeral key

A key generated for one session and then discarded.

Forward secrecy

Past traffic stays secret even if a key leaks later.

HKDF

HMAC-based Key Derivation Function: turns a raw secret into a clean key.

Salt

Extra input to HKDF (here the two nonces) making the key session-unique.

Info string

A label fed to HKDF binding the key to a specific purpose.

SHA-256

A hash function producing a 256-bit fingerprint.

ECDSA

Elliptic Curve Digital Signature Algorithm.

Digital signature

A private-key-made value verifiable with the public key.

DER

A compact binary encoding (used here for signatures).

PEM

A text encoding (Base64 + BEGIN/END lines) used for keys.

Serialization

Converting an in-memory object into bytes for storage/transmission.

InvalidTag

The library error raised when a GCM tag does not match.

DecryptionError

This project’s exception meaning “possible tampering”.

Session key

The 32-byte AES key used to encrypt one conversation.

3.10 References and Further Study

- **Python cryptography — AES-GCM (AESGCM) docs** — <https://cryptography.io/en/latest/hazmat/primitives/aead/>
Read for: the exact AESGCM class used in `crypto.py`.

- **Python cryptography — elliptic-curve docs** — <https://cryptography.io/en/latest/hazmat/primitives/asymmetric/ec/>
Read for: ECDH exchange() and ECDSA sign/verify.
- **Python cryptography — HKDF docs** — <https://cryptography.io/en/latest/hazmat/primitives/key-derivation-functions/>
Read for: the HKDF class, its salt and info parameters.
- **NIST SP 800-38D — GCM mode of operation** — <https://csrc.nist.gov/pubs/sp/800/38/d/final>
Read for: the official definition of AES-GCM and the nonce rule.
- **NIST FIPS 197 — the AES standard** — <https://csrc.nist.gov/pubs/fips/197/final>
Read for: what AES itself is.
- **RFC 5869 — HKDF** — <https://datatracker.ietf.org/doc/html/rfc5869>
Read for: why extract-then-expand key derivation is done this way.
- **Diffie–Hellman key exchange (Wikipedia)** — https://en.wikipedia.org/wiki/Diffie%E2%80%93Hellman_key_exchange
Read for: the paint analogy and the original idea behind ECDH.
- **Authenticated encryption (Wikipedia)** — https://en.wikipedia.org/wiki/Authenticated_encryption
Read for: why confidentiality alone is not enough.
- **Galois/Counter Mode (Wikipedia)** — https://en.wikipedia.org/wiki/Galois/Counter_Mode
Read for: how the GCM tag is built.
- **Forward secrecy (Wikipedia)** — https://en.wikipedia.org/wiki/Forward_secrecy
Read for: why ephemeral keys matter.
- **Computerphile — “Elliptic Curve Cryptography” (video)** — <https://www.youtube.com/watch?v=NF1pwjL9-DE>
Read/watch for: a visual, beginner-friendly take on the curve maths.

End of Section S3. Next: S4 — the 4-step Authentication Handshake.

Chapter 4

S4 — The 4-Step Authentication Handshake

4.1 Title and “What You’ll Learn”

Section Title

S4 — The 4-Step Mutual Authentication Handshake: how two vehicles prove who they are to *each other* and secretly agree on a session key.

Plain-English Summary (read this first)

Before two cars exchange a single safety message, they perform a *handshake* — a short, fixed conversation in four steps. By the end of it each car is sure of two things: (1) the other car is genuine (its certificate is real *and* it actually owns the matching private key), and (2) both cars now hold an identical secret key that was *never sent over the network*. The handshake is built from a **HELLO**, a **CHALLENGE**, a **RESPONSE** and an **ACK**. It is implemented by one file, `protocol/auth_protocol.py`, which stitches together the certificates of Section S2 and the cryptographic tools of Section S3.

Learning Outcomes

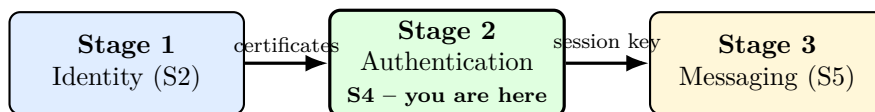
After studying this section you will be able to:

- Explain **mutual authentication** and why one-sided authentication is not enough.
- Explain **challenge–response** and the role of the 32-byte handshake **nonce**.
- Explain how the **ReplayCache** and **TimestampValidator** block replayed handshake messages.
- Describe the four messages — HELLO, CHALLENGE, RESPONSE, ACK — and the data each carries.
- Walk through every method of the `AuthProtocol` class step by step.
- Explain how the same session key is derived on both sides via ECDH+HKDF.
- Run the handshake demo, read its log, and force it to fail in three different ways (stale message, replayed nonce, bad signature).

4.2 Where This Fits

S4 *is* Stage 2 (Authentication) of the pipeline. It is the bridge: it consumes the certificates created in Stage 1 (S2) and the cryptographic toolbox of S3, and it produces the **session key**

that Stage 3 (S5, secure messaging) needs. Without a completed handshake there is no key, and without a key no BSM can be encrypted.



S4 imports 7 functions from crypto.py (S3) and reuses the cert checks of S2.

4.3 The Theory You Must Study

4.3.1 Mutual Authentication

Plain definition. *Authentication* is proving who you are. *Mutual* authentication means *both* parties prove their identity to each other — not just one side.

Real-life analogy. When you log in to a bank website, normally only *you* prove who you are (your password). Mutual authentication is stronger: it is like meeting a courier where the courier shows you ID *and* you show the courier ID — neither hands anything over until both are satisfied.

Why it exists. If only Vehicle B checked Vehicle A, an attacker could impersonate B. If only A checked B, an attacker could impersonate A. A man-in-the-middle relies on each side failing to check the other. Mutual authentication closes both doors at once.

How it works here. In the handshake, B verifies A’s certificate *and* A’s signature, and A verifies B’s certificate *and* B’s signature. Both directions are checked before any safety message flows.

Security property it provides. **Authentication** (and, by defeating MITM, it protects **confidentiality** and **integrity** too).

4.3.2 Challenge–Response and the Handshake Nonce

Plain definition. A *challenge–response* is a test: one party sends a fresh random value (the *challenge*); the other must perform an operation on it that only the genuine party could perform (the *response*). The fresh random value is a **nonce**.

Real-life analogy. A bouncer says “say today’s password backwards”. Only someone who truly knows the password can do it, and because the request is new each time, an eavesdropper who recorded yesterday’s answer cannot reuse it.

Why it exists. Showing a certificate only proves the certificate is real — it does *not* prove the sender *owns* the matching private key. A certificate is public; an attacker could copy it. The challenge–response forces the sender to *use* its private key on a value it could not have predicted, proving live possession of the key.

How it works here. Each vehicle generates a 32-byte random nonce (`os.urandom(32).hex()`). Each then *signs* the pair of nonces with its certificate private key (`ecdsa_sign`). The other side verifies that signature with the public key from the certificate (`ecdsa_verify`). A valid signature over a *freshly chosen* nonce proves: “you are online now and you hold the private key”.

Note on two kinds of nonce. This 32-byte handshake nonce is *not* the same as the 12-byte AES-GCM nonce of S3. The handshake nonce is a freshness challenge; the AES-GCM nonce makes each encryption unique. Same word, two jobs.

Security property it provides. **Authentication** and **replay prevention**.

4.3.3 Replay Attacks and the Replay Cache

Plain definition. A *replay attack* re-sends a previously captured, genuine message. The `ReplayCache` is an in-memory list of nonces already seen; a repeat is rejected.

Real-life analogy. A nightclub that ink-stamps each ticket as it is used. A photocopied ticket carries a stamp number the door already recorded — entry refused.

Why it exists. Even a perfectly genuine, correctly signed HELLO becomes dangerous if an attacker can record it and replay it later to start a bogus session. The cache guarantees each nonce is accepted *at most once*.

How it works here. `ReplayCache.check_and_add(nonce)` raises `ReplayError` if the nonce is already stored, otherwise stores it with an expiry of `now + 300 s`. Old entries are cleaned out automatically. 300 seconds (5 minutes) is the constant `REPLAY_WINDOW_SECONDS`.

Security property it provides. **Replay prevention.**

4.3.4 Timestamp / Freshness Validation

Plain definition. Every handshake message carries the time it was created. The `TimestampValidator` rejects any message whose time is too far from “now”.

Real-life analogy. A concert ticket printed with a date. The usher glances at it: yesterday’s ticket is refused on sight, without even checking the seat number.

Why it exists. The replay cache only remembers nonces for 5 minutes and costs memory. The timestamp check is a cheap first filter: a message hours old is thrown out instantly. The two together are belt-and-braces.

How it works here. `TimestampValidator.validate(timestamp)` computes `age = abs(now - timestamp)` and raises `StaleMessageError` if `age > 5.0` seconds. Note the `abs`: a timestamp from the *future* is rejected too, which guards against clock-skew tricks.

Security property it provides. **Replay prevention.**

4.3.5 Certificate Verification (recap from S2)

The handshake must check that the certificate a peer presents was really signed by the trusted CA. S4 does this with the helper `verify_certificate_against_ca`, which (1) parses the peer and CA certificates, (2) verifies the CA’s ECDSA-SHA256 signature on the peer certificate, (3) checks the validity dates, and (4) returns the peer’s public key. This is the same logic explained fully in Section S2 — S4 simply *calls* it at the right moment. *Security property: authentication.*

4.3.6 ECDH Ephemeral Key Exchange (recap from S3)

During the handshake each vehicle generates a *fresh* ECDH key pair (`generate_ecdh_keypair`), sends the public half inside `RESPONSE` or `ACK`, and combines its private half with the peer’s public half (`ecdh_shared_secret`) to get a shared secret. Both sides then run `derive_session_key` (HKDF) to turn that secret plus the two nonces into the *same* 32-byte AES key. Because the ECDH keys are ephemeral, the session has **forward secrecy**. The maths is covered in Section S3.

4.3.7 Dataclasses and the JSON Wire Format

Plain definition. A Python `@dataclass` is a small class that just holds named fields. *JSON* (JavaScript Object Notation) is a simple text format for structured data — the form the messages take while crossing the network.

Real-life analogy. A dataclass is a labelled paper form with named boxes; `to_json` folds the form into an envelope of text; `from_json` opens the envelope and reads the boxes back.

Why it exists. A network carries bytes, not Python objects. Each message class has `to_json()` (object $\rightarrow$ text) and `from_json()` (text $\rightarrow$ object), so a message can be sent and faithfully rebuilt on the far side.

How it works here. `HelloMessage`, `ChallengeMessage`, `ResponseMessage` and `AckMessage` are dataclasses. Each carries a `msg_type` tag (“HELLO”, “CHALLENGE”, ...) so the receiver knows

which form it has been handed. Binary values (nonces, signatures) are carried as *hex strings* because raw bytes cannot live inside JSON text.

Security property it provides. None directly — it is the *format*, not a defence.

4.3.8 Why Exactly Four Steps?

Each step has a job, and you cannot remove one:

1. **HELLO** — A introduces itself and sends its certificate + nonce. (B can now check A's identity.)
2. **CHALLENGE** — B sends its certificate + nonce *and* a signature over both nonces. (A can now check B's identity *and* that B holds its private key.)
3. **RESPONSE** — A signs both nonces too, and sends its ephemeral ECDH public key. (B can now check A holds its key, and start the key exchange.)
4. **ACK** — B sends its ephemeral ECDH public key. (A can now finish the key exchange.)

Steps 1–2 exchange certificates; steps 2–3 are the two-way challenge–response; steps 3–4 exchange the ECDH public keys. Four messages is the minimum for *mutual* authentication plus a key agreement.

4.4 Code Walkthrough

`auth_protocol.py` has four parts: custom exceptions, four message dataclasses, two small protection classes (`ReplayCache`, `TimestampValidator`), the certificate helper, and the main `AuthProtocol` class.

4.4.1 The custom exceptions

```
1 class AuthError(Exception):
2     """Base exception for authentication errors."""
3
4 class ReplayError(AuthError):
5     """Raised when a replayed nonce is detected."""
6
7 class StaleMessageError(AuthError):
8     """Raised when a message timestamp is too old."""
9
10 class CertificateError(AuthError):
11     """Raised when certificate verification fails."""
```

Listing 4.1: `auth_protocol.py` — the exception hierarchy.

Explanation: `AuthError` is the parent; the three children name the three specific ways a handshake can be rejected — a replayed nonce, a stale message, a bad certificate. Naming each one separately makes the failure reason obvious in logs and in the attack demos.

4.4.2 The four message dataclasses

```
1 @dataclass
2 class HelloMessage:
3     vehicle_id: str
4     certificate_pem: str
5     nonce: str
6     timestamp: float
7     msg_type: str = "HELLO"
8
9     def to_json(self) -> str:
```



```

10         return json.dumps(asdict(self))
11
12     @classmethod
13     def from_json(cls, data: str) -> HelloMessage:
14         return cls(**json.loads(data))

```

Listing 4.2: `auth_protocol.py` — the HELLO message (the other three messages follow the same pattern).

Step by step:

1. The `@dataclass` decorator auto-generates the boring boilerplate (constructor, etc.) so the class is just a list of named fields.
2. `HelloMessage` fields: the sender's `vehicle_id`, its `certificate_pem` (its passport as text), its `nonce` (32 random bytes as a 64-character hex string), the `timestamp`, and a fixed `msg_type` tag.
3. `to_json` turns the object into a JSON text string ready to send.
4. `from_json` is a *classmethod* that rebuilds the object from a received JSON string.

The other three dataclasses differ only in their fields:

- `ChallengeMessage` adds `signed_nonces` (an ECDSA signature over both nonces, as hex).
- `ResponseMessage` carries `signed_nonces` and `ecdh_public_key_pem` (the ephemeral ECDH public key) — note it has *no* certificate field, because A already sent its certificate in HELLO.
- `AckMessage` carries only `ecdh_public_key_pem` and a `session_established` flag — and, unlike the others, it has *no timestamp field*.

4.4.3 ReplayCache — catching repeated nonces

```

1 class ReplayCache:
2     def __init__(self, window_seconds: int = 300) -> None:
3         self._window = window_seconds
4         self._seen: dict[str, float] = {} # {nonce_hex: expiry_timestamp}
5
6     def check_and_add(self, nonce: str) -> None:
7         self._cleanup()
8         if nonce in self._seen:
9             raise ReplayError(f"Nonce already seen: {nonce[:16]}...")
10        self._seen[nonce] = time.time() + self._window
11
12    def _cleanup(self) -> None:
13        now = time.time()
14        expired = [n for n, exp in self._seen.items() if exp < now]
15        for n in expired:
16            del self._seen[n]

```

Listing 4.3: `auth_protocol.py` — the replay cache.

Step by step:

1. `__init__` sets the remembering window (default 300s) and an empty dictionary `_seen` mapping each nonce to the time it should be forgotten.
2. `check_and_add` first calls `_cleanup`.
3. If the nonce is already in `_seen`, it raises `ReplayError` — this is a replay attack being caught.
4. Otherwise it records the nonce with expiry `now + 300 s`.
5. `_cleanup` deletes nonces whose expiry has passed, so the cache does not grow forever.

Arguments: nonce (hex string). **Returns:** nothing. **Exceptions:** `ReplayError`. *Analogy:* the ink-stamp register at the nightclub door.

4.4.4 TimestampValidator — catching stale messages

```
1 class TimestampValidator:
2     def __init__(self, max_age_seconds: float = 5.0) -> None:
3         self._max_age = max_age_seconds
4
5     def validate(self, timestamp: float) -> None:
6         age = abs(time.time() - timestamp)
7         if age > self._max_age:
8             raise StaleMessageError(
9                 f"Message is {age:.1f}s old (max {self._max_age}s)"
10            )
```

Listing 4.4: `auth_protocol.py` — the timestamp validator.

Step by step:

1. `__init__` stores the maximum allowed age (default 5.0s).
2. `validate` computes the *absolute* difference between now and the message's timestamp.
3. If that age exceeds 5 seconds, it raises `StaleMessageError`.

Why `abs`? It rejects both *too-old* messages (a real replay) and *too-future* messages (a forged or clock-skewed timestamp). *Analogy:* the usher refusing a ticket whose date is not today.

4.4.5 `verify_certificate_against_ca` — the certificate gate

```
1 def verify_certificate_against_ca(cert_pem, ca_cert_pem):
2     try:
3         cert_bytes = cert_pem.encode("utf-8") if isinstance(cert_pem, str) else
4             cert_pem
5         cert = x509.load_pem_x509_certificate(cert_bytes)
6         ca_cert = x509.load_pem_x509_certificate(ca_cert_pem)
7
8         # Verify the CA's signature on the vehicle certificate.
9         ca_public_key = ca_cert.public_key()
10        ca_public_key.verify(
11            cert.signature,
12            cert.tbs_certificate_bytes,
13            ec.ECDSA(hashes.SHA256()),
14        )
15
16        # Check validity dates (now must be inside the window).
17        now = datetime.now(timezone.utc)
18        nb = getattr(cert, "not_valid_before_utc", None) or \
19            cert.not_valid_before.replace(tzinfo=timezone.utc)
20        na = getattr(cert, "not_valid_after_utc", None) or \
21            cert.not_valid_after.replace(tzinfo=timezone.utc)
22        if now < nb: raise CertificateError("Certificate is not yet valid")
23        if now > na: raise CertificateError("Certificate has expired")
24
25        pub_key = cert.public_key()
26        if not isinstance(pub_key, ec.EllipticCurvePublicKey):
27            raise CertificateError("Certificate does not contain an EC public key")
28
29        return pub_key
30    except CertificateError:
31        raise
32    except Exception as exc:
```

```

31     raise CertificateError(f"Certificate verification failed: {exc}") from
        exc

```

Listing 4.5: auth_protocol.py — verify a peer certificate against the trusted CA.

Step by step:

1. Parse the peer’s certificate and the CA’s certificate from PEM text.
2. `ca_public_key.verify(...)` checks the CA’s ECDSA-SHA256 signature on the peer certificate’s `tbs_certificate_bytes` (the “to-be-signed” body). A forged certificate fails here.
3. Check that “now” is inside the certificate’s validity window; otherwise raise `CertificateError`.
4. Confirm the embedded public key is an elliptic-curve key, then **return** it — the caller needs it to verify signatures.
5. Any failure becomes a `CertificateError`.

Theory link: certificate verification / chain of trust (S2). This is the first of the two identity checks; the second is the signature challenge.

4.4.6 The AuthProtocol class

__init__ — set up one vehicle’s handshake state

The constructor stores the vehicle’s id, its private key, its certificate (PEM bytes), and the CA certificate. It creates one `ReplayCache` and one `TimestampValidator`, and prepares empty “slots” for the handshake state: `my_nonce`, `peer_nonce`, `peer_public_key`, `session_key`, and the ephemeral ECDH key pair. Both vehicles create one `AuthProtocol` object each.

Step 1 — build_hello() (initiator)

```

1 def build_hello(self) -> HelloMessage:
2     self.my_nonce = os.urandom(32).hex()
3     hello = HelloMessage(
4         vehicle_id=self.vehicle_id,
5         certificate_pem=self.certificate_pem.decode("utf-8"),
6         nonce=self.my_nonce,
7         timestamp=time.time(),
8     )
9     return hello

```

Listing 4.6: auth_protocol.py — Step 1: build the HELLO.

Steps: (1) generate a fresh 32-byte random nonce, stored as hex in `my_nonce`; (2) build a `HelloMessage` carrying the vehicle id, the certificate, the nonce and the current time; (3) return it to be sent. **Theory link:** starts the challenge–response by issuing this vehicle’s challenge nonce.

Step 2 — process_hello() (responder)

```

1 def process_hello(self, msg: HelloMessage) -> ChallengeMessage:
2     # 1. Reject stale messages
3     self.timestamp_validator.validate(msg.timestamp)
4     # 2. Reject replayed nonces
5     self.replay_cache.check_and_add(msg.nonce)
6     # 3. Verify peer certificate was signed by our trusted CA
7     self.peer_public_key = verify_certificate_against_ca(
8         msg.certificate_pem, self.ca_cert_pem)
9     # 4. Store peer nonce; generate our own
10    self.peer_nonce = msg.nonce

```

```

11     self.my_nonce = os.urandom(32).hex()
12     # 5. Sign (peer_nonce + our_nonce) -- proves private key possession
13     nonce_data = bytes.fromhex(self.peer_nonce) + bytes.fromhex(self.my_nonce)
14     signature = ecdsa_sign(self.private_key, nonce_data)
15     challenge = ChallengeMessage(
16         vehicle_id=self.vehicle_id,
17         certificate_pem=self.certificate_pem.decode("utf-8"),
18         nonce=self.my_nonce,
19         signed_nonces=signature.hex(),
20         timestamp=time.time(),
21     )
22     return challenge

```

Listing 4.7: auth_protocol.py — Step 2: verify HELLO, build CHALLENGE.

Step by step:

1. Validate the HELLO's timestamp — reject if older than 5 seconds.
2. Replay-check the HELLO's nonce — reject if seen before.
3. Verify A's certificate against the CA; on success, store A's public key.
4. Save A's nonce as `peer_nonce`; generate this vehicle's own fresh nonce.
5. Join the two nonces into bytes and *sign* them with this vehicle's private key. This signature is the response to A's challenge *and* a new challenge to A.
6. Build and return the CHALLENGE carrying B's certificate, B's nonce, the signature, and a timestamp.

Theory link: certificate verification + the responding half of challenge–response. *Analogy:* the bouncer checks A's ID, then says “now *you* repeat *my* password too.”

Step 3 — process_challenge() (initiator)

```

1  def process_challenge(self, msg: ChallengeMessage) -> ResponseMessage:
2      self.timestamp_validator.validate(msg.timestamp)
3      self.replay_cache.check_and_add(msg.nonce)
4      # Verify peer certificate
5      self.peer_public_key = verify_certificate_against_ca(
6          msg.certificate_pem, self.ca_cert_pem)
7      # Verify peer's signature on (our_nonce + their_nonce)
8      self.peer_nonce = msg.nonce
9      expected_data = bytes.fromhex(self.my_nonce) + bytes.fromhex(self.peer_nonce)
10     if not ecdsa_verify(self.peer_public_key, expected_data,
11                         bytes.fromhex(msg.signed_nonces)):
12         raise AuthError("Challenge signature invalid -- possible impersonation")
13     # Generate ephemeral ECDH key pair (forward secrecy)
14     self._ecdh_private, self._ecdh_public = generate_ecdh_keypair()
15     # Sign nonces with OUR private key
16     our_sig = ecdsa_sign(self.private_key, expected_data)
17     response = ResponseMessage(
18         signed_nonces=our_sig.hex(),
19         ecdh_public_key_pem=serialize_public_key(self._ecdh_public).decode("utf-8"),
20         timestamp=time.time(),
21     )
22     return response

```

Listing 4.8: auth_protocol.py — Step 3: verify CHALLENGE, build RESPONSE.

Step by step:

1. Validate the CHALLENGE timestamp and replay-check B's nonce.

2. Verify B's certificate against the CA; store B's public key.
3. Re-build the exact nonce pair B claims to have signed (`my_nonce + peer_nonce`) and check B's signature with `ecdsa_verify`. If it fails, raise `AuthError` — “possible impersonation”. *This is where a fake B is caught.*
4. Generate this vehicle's *ephemeral* ECDH key pair (forward secrecy).
5. Sign the same nonce pair with this vehicle's certificate private key — the response to B's challenge.
6. Build and return the RESPONSE carrying the signature and the ECDH public key (as PEM text).

Theory link: verifying the challenge response = the second identity check; `generate_ecdh_keypair` starts the key agreement.

Step 4a — `process_response()` (responder)

```

1 def process_response(self, msg: ResponseMessage, original_hello: HelloMessage) ->
  AckMessage:
2     self.timestamp_validator.validate(msg.timestamp)
3     # Verify initiator's signature on (their_nonce + our_nonce)
4     nonce_data = bytes.fromhex(self.peer_nonce) + bytes.fromhex(self.my_nonce)
5     if not ecdsa_verify(self.peer_public_key, nonce_data,
6                         bytes.fromhex(msg.signed_nonces)):
7         raise AuthError("Response signature invalid")
8     # Generate our ECDH key pair and compute shared secret
9     self._ecdh_private, self._ecdh_public = generate_ecdh_keypair()
10    peer_ecdh_pub = load_public_key(msg.ecdh_public_key_pem.encode("utf-8"))
11    shared_secret = ecdh_shared_secret(self._ecdh_private, peer_ecdh_pub)
12    # Derive session key -- initiator nonce first for consistency
13    nonce_a = bytes.fromhex(self.peer_nonce) # initiator's nonce
14    nonce_b = bytes.fromhex(self.my_nonce)   # our nonce (responder)
15    self.session_key = derive_session_key(shared_secret, nonce_a, nonce_b)
16    ack = AckMessage(
17        ecdh_public_key_pem=serialize_public_key(self._ecdh_public).decode("utf-8"),
18    )
19    return ack

```

Listing 4.9: `auth_protocol.py` — Step 4a: verify RESPONSE, derive key, build ACK.

Step by step:

1. Validate the RESPONSE timestamp.
2. Verify A's signature over the nonce pair (`peer_nonce + my_nonce`). Fail → `AuthError`. Now B is sure A holds A's private key.
3. Generate B's ephemeral ECDH key pair; load A's ECDH public key from the RESPONSE; compute the ECDH shared secret.
4. Derive the session key with HKDF, putting the *initiator's* nonce first (`nonce_a`) and the *responder's* second (`nonce_b`) — a fixed order so both sides agree.
5. Build and return the ACK carrying B's ECDH public key.

Honest note: the parameter `original_hello` is part of the signature but is not actually used inside the method body — the demo passes the HELLO object anyway. Worth mentioning if an examiner asks.

Step 4b — process_ack() (initiator)

```
1 def process_ack(self, msg: AckMessage) -> bytes:
2     peer_ecdh_pub = load_public_key(msg.ecdh_public_key_pem.encode("utf-8"))
3     shared_secret = ecdh_shared_secret(self._ecdh_private, peer_ecdh_pub)
4     # Same nonce order as the responder used
5     nonce_a = bytes.fromhex(self.my_nonce)      # initiator's nonce (us)
6     nonce_b = bytes.fromhex(self.peer_nonce)    # responder's nonce
7     self.session_key = derive_session_key(shared_secret, nonce_a, nonce_b)
8     return self.session_key
```

Listing 4.10: auth_protocol.py — Step 4b: process ACK, derive the matching key.

Step by step:

1. Load B's ECDH public key from the ACK.
2. Compute the ECDH shared secret from A's ephemeral private key and B's ephemeral public key. By ECDH maths this equals the secret B computed.
3. Derive the session key with HKDF — and crucially with the *same nonce order* B used (initiator's nonce first). Same secret + same nonces + same HKDF $\Rightarrow$ *identical 32-byte key*.
4. Return the session key. The handshake is complete.

Theory link: ECDH + HKDF (S3). *Analogy:* both painters reach the same final colour — and both label the tin the same way, so they truly match.

The demo block

The `if __name__ == "__main__"` block at the bottom loads the CA and both vehicle certificates, creates two `AuthProtocol` objects, and runs the five calls in order: `build_hello` $\rightarrow$ `process_hello` $\rightarrow$ `process_challenge` $\rightarrow$ `process_response` $\rightarrow$ `process_ack`. It then prints both session keys and confirms they match. *In this demo the two objects call each other's methods directly in one process; in the real system (S6) the messages travel over TCP between two programs.*

4.5 Hands-On / Practical

4.5.1 Run the handshake by itself

The handshake demo needs the certificates first (Section S2). Run, in order:

```
python ca\ca.py
python ca\issue_cert.py --vehicle-id vehicle-a --output-dir ca\certs
python ca\issue_cert.py --vehicle-id vehicle-b --output-dir ca\certs
python protocol\auth_protocol.py
```

Expected output (sample). Where this guide shows [OK] the real terminal prints a small check-mark character.

```
=== V2V AUTHENTICATION HANDSHAKE DEMO ===

[10:00:00] INFO: [vehicle-a] Sent HELLO (nonce=3f1a9c2e...)
[10:00:00] INFO: [vehicle-b] Received HELLO from vehicle-a
[10:00:00] INFO: [vehicle-b] Certificate VALID for vehicle-a
[10:00:00] INFO: [vehicle-b] Sent CHALLENGE (nonce=8b4d77e1...)
[10:00:00] INFO: [vehicle-a] Received CHALLENGE from vehicle-b
[10:00:00] INFO: [vehicle-a] Certificate VALID for vehicle-b
[10:00:00] INFO: [vehicle-a] Challenge signature verified [OK]
[10:00:00] INFO: [vehicle-a] Sent RESPONSE with ECDH public key
```

```
[10:00:00] INFO: [vehicle-b] Received RESPONSE
[10:00:00] INFO: [vehicle-b] Response signature verified [OK]
[10:00:00] INFO: [vehicle-b] Session key derived (forward secrecy: ENABLED)
[10:00:00] INFO: [vehicle-b] Sent ACK -- handshake complete (responder)
[10:00:00] INFO: [vehicle-a] Received ACK -- session established
[10:00:00] INFO: [vehicle-a] MUTUAL AUTHENTICATION COMPLETE -- session key
    derived

Vehicle A session key: 1c9e7a4f0b2d6688...
Vehicle B session key: 1c9e7a4f0b2d6688...
Keys match: True
Handshake time: 7.3 ms
[OK] Mutual authentication successful - both vehicles share the same key!
```

4.5.2 What the output lines mean

- Sent HELLO — Step 1: A issued its nonce.
- Certificate VALID (twice) — each side confirmed the other's passport was signed by the CA.
- Challenge/Response signature verified — each side proved it *owns* its private key (the challenge-response succeeded).
- Session key derived (forward secrecy: ENABLED) — the ephemeral ECDH exchange produced the key.
- Keys match: True — the whole point: both sides hold the *identical* key, though it was never transmitted.

4.5.3 Try This Yourself (three ways to break the handshake)

Open `auth_protocol.py` and edit the demo block at the bottom.

Experiment 1 — replayed nonce. After the line `challenge = auth_b.process_hello(hello)` add a second identical call:

```
challenge = auth_b.process_hello(hello)
auth_b.process_hello(hello)           # <-- feed the SAME hello again
```

Predict: the first call stored the HELLO's nonce; the second call finds it already in the `ReplayCache` and raises `ReplayError: Nonce already seen`. This is exactly the replay defence.

Experiment 2 — stale message. Before `process_hello`, age the HELLO by editing its timestamp:

```
hello = auth_a.build_hello()
hello.timestamp = hello.timestamp - 100    # pretend it is 100 s old
challenge = auth_b.process_hello(hello)
```

Predict: `TimestampValidator` sees an age of about 100 seconds (> 5) and raises `StaleMessageError`.

Experiment 3 — bad signature (impersonation). Corrupt the challenge signature before A processes it:

```
challenge.signed_nonces = "00" + challenge.signed_nonces[2:]
response = auth_a.process_challenge(challenge)
```

Predict: `ecdsa_verify` returns `False`, so `process_challenge` raises `AuthError: Challenge signature invalid - possible impersonation`. Undo all edits afterwards.

4.6 Attack Relevance

S4 is the busiest defensive section: it stops *three* of the four attacks and is exercised by both `attacks/replay_attack.py` and `attacks/mitm_attack.py`.

Attack	How S4 stops it	The mechanism
Replay	A captured HELLO/CHALLENGE is re-sent; its nonce is already in the cache, or its timestamp is too old.	<code>ReplayCache.check_and_add</code> , <code>TimestampValidator.validate</code> .
MITM (fake cert)	The attacker's certificate is not signed by the CA.	<code>verify_certificate_against_ca</code> raises <code>CertificateError</code> .
MITM / Impersonation (forged signature)	The attacker cannot sign the fresh nonce pair without the real private key.	<code>ecdsa_verify</code> returns <code>False</code> → <code>AuthError</code> .

Table 4.1: S4 defeats replay, fake-certificate MITM, and impersonation.

Why the handshake defeats a man-in-the-middle. An MITM attacker sits between A and B. To impersonate A to B, the attacker must send a CHALLENGE or RESPONSE that B accepts. Two walls block this:

1. **The certificate wall.** The attacker's certificate must be CA-signed. A self-signed forgery fails `verify_certificate_against_ca`.
2. **The signature wall.** Even if the attacker *replays* A's real certificate, it must then sign the *fresh* nonce pair B just chose. Signing needs A's private key, which the attacker does not have. So `ecdsa_verify` fails and the handshake aborts.

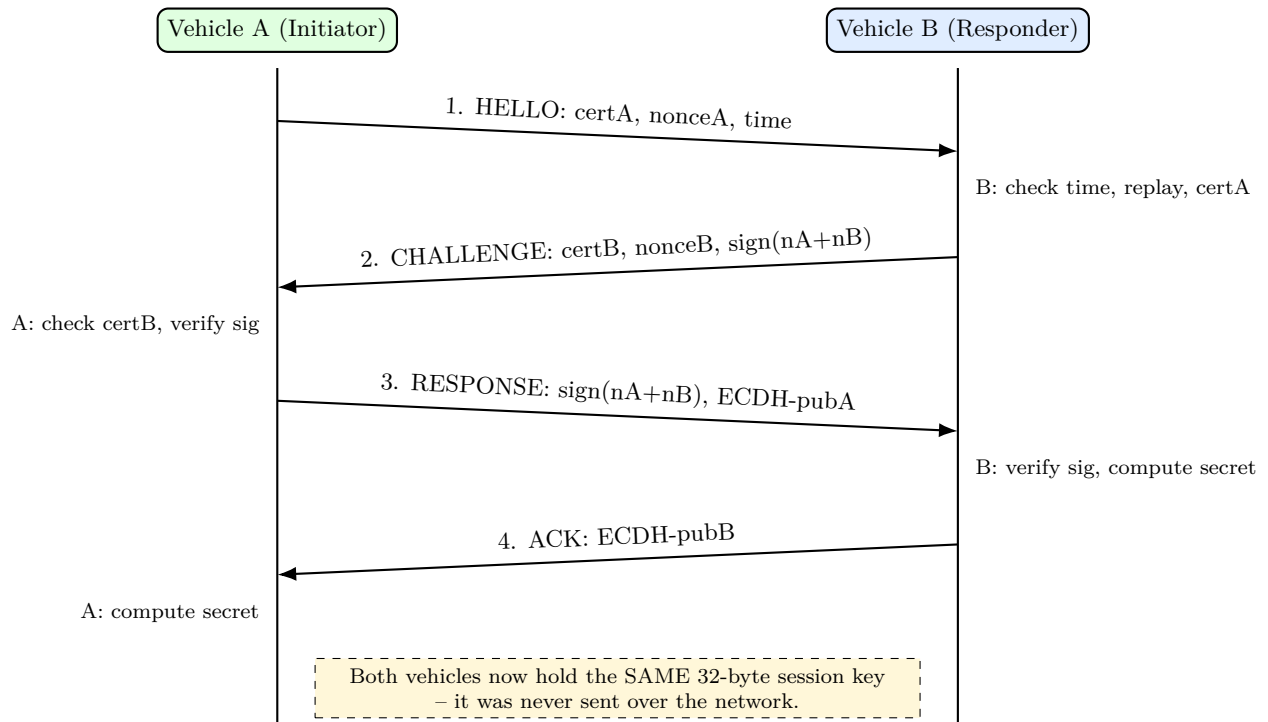
This is why S2 (identity) alone is not enough and S4 is needed: S2 proves a certificate is genuine; S4's challenge–response proves the sender *owns* the matching private key *right now*.

Forward secrecy contribution. Because the ECDH keys are ephemeral (`generate_ecdh_keypair` per session), even an attacker who later steals a vehicle's *certificate* private key cannot reconstruct a past session key — the ephemeral keys that built it no longer exist.

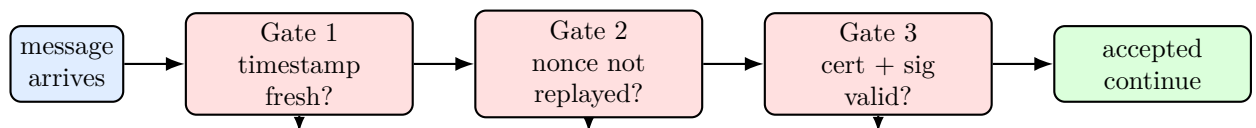
Honest scope note. The handshake assumes clocks are roughly synchronised (within 5 s) for the timestamp check; on badly skewed clocks an honest handshake could be wrongly rejected. Also, the demo wires the two objects together in one process — real networking, framing and threading are added in Section S6.

4.7 Illustrations

4.7.1 Diagram 1 — The 4-Step Handshake (message flow)



4.7.2 Diagram 2 — The three gates every handshake message passes



Any gate that fails raises an exception: StaleMessageError, ReplayError, or CertificateError / AuthError – the handshake stops.

4.8 Presentation Pack

4.8.1 Slide Outline

Slide 1 — Why a Handshake?

- Before any safety message, the cars must trust each other.
- Showing a certificate is not enough — certificates are public.
- The handshake adds a live “prove you own the key” test.
- It also secretly agrees on a shared encryption key.

Slide 2 — The Four Steps.

- HELLO: A sends its certificate + a random nonce.
- CHALLENGE: B sends its certificate + nonce + a signature.
- RESPONSE: A sends its signature + an ECDH public key.
- ACK: B sends its ECDH public key — done.

Slide 3 — Challenge–Response: Proving Key Ownership.

- Each side signs the pair of fresh nonces with its private key.
- The other side verifies with the certificate’s public key.

- A valid signature over a brand-new nonce = “you own the key, now”.
- Wrong/missing key → `AuthError` — impersonation caught.

Slide 4 — Three Gates Stop Replay.

- Timestamp gate: message older than 5 s → `StaleMessageError`.
- Nonce gate: nonce seen before → `ReplayError`.
- Certificate gate: not CA-signed → `CertificateError`.
- Belt-and-braces: cheap time check plus exact-duplicate check.

Slide 5 — A Shared Key, Never Sent.

- Each side makes an ephemeral ECDH key pair.
- Public halves are swapped in RESPONSE and ACK.
- Both compute the same secret; HKDF turns it into a 32-byte key.
- Ephemeral keys → forward secrecy. “Keys match: True”.

4.8.2 Speaker Talking Points (about 30–60 seconds each)

Slide 1. “Before our two cars send a single safety message, they run a handshake — a short, fixed four-step conversation. Why bother? Because simply showing a certificate is not enough. A certificate is public; anyone can copy it. So the handshake does two extra jobs. First, it makes each car prove, live, that it actually *owns* the private key behind its certificate — not just that it has a copy of the certificate. Second, while doing that, the two cars secretly agree on a shared encryption key. By the end, each car is certain who it is talking to and they share a secret nobody else knows.”

Slide 2. “The handshake has four messages. In HELLO, car A introduces itself, sending its certificate and a fresh random number called a nonce. In CHALLENGE, car B replies with its own certificate, its own nonce, and — importantly — a digital signature over both nonces. In RESPONSE, car A signs both nonces too and includes a special temporary key for the key exchange. Finally in ACK, car B sends its matching temporary key. Four messages: two to swap certificates, and the signatures going both ways for the proof.”

Slide 3. “The heart of the handshake is challenge–response. Each car picks a brand-new random nonce that the other side could not have predicted. Each car then signs the pair of nonces with its private key. The other side verifies that signature using the public key inside the certificate. Here is the clever part: because the nonce is fresh, a recorded old signature is useless, and because signing needs the private key, only the genuine car can produce a signature that verifies. If the signature is wrong, our code raises an `AuthError` and says ‘possible impersonation’. That single check is what stops a fake car.”

Slide 4. “Every handshake message passes three gates. The first gate checks the timestamp — if the message is more than five seconds old, it is stale and rejected immediately; that is cheap and fast. The second gate checks the nonce against a replay cache — if we have seen that exact nonce before, it is a replay and we reject it. The third gate checks the certificate is signed by our trusted CA. Any gate failing throws a specific exception and the handshake stops. Two of those gates — timestamp and nonce — together make replaying an old handshake impossible.”

Slide 5. “Finally, the key. During the handshake each car generates a temporary, throw-away key pair for something called ECDH. They swap only the *public* halves, inside the RESPONSE and ACK messages. Then each car combines its own private half with the other’s public half, and thanks to the maths both end up with the *same* secret — even though that secret was never sent. HKDF turns it into a clean 32-byte key. And because those ECDH keys are thrown away after the session, even a key stolen later cannot unlock this conversation — that is forward secrecy. The demo ends by printing ‘Keys match: True’”

4.8.3 Live-Demo Script

1. **Set up.** Run `ca.py` and the two `issue_cert.py` commands so the certificates exist.
2. **Run the handshake.** Execute `python protocol/auth_protocol.py`. Read the log aloud, pointing at each of the four steps in turn.
3. **Point at the proof lines.** “Certificate VALID” (the cert gate) and “signature verified” (the challenge–response).
4. **Point at the result.** “Keys match: True” — “the same key on both sides, never sent over the wire.”
5. **Break it live.** Apply Experiment 1 (replay) or Experiment 3 (bad signature) from the Hands-On section and show the `ReplayError` or `AuthError`. Restore the file afterwards.

4.8.4 Examiner Q&A

1. **Q: What is mutual authentication and why not just one-sided?**
A: Both parties prove identity to each other. One-sided authentication leaves the unchecked side open to impersonation and enables a man-in-the-middle; mutual authentication checks both directions.
2. **Q: A certificate is already trusted — why also do challenge–response?**
A: A certificate is public and can be copied. The challenge–response forces the sender to sign a *fresh, unpredictable* nonce with the private key, proving it *owns* the key right now, not just holds a copy of the certificate.
3. **Q: Why are there two nonces, one from each side?**
A: So that *each* side issues a challenge the *other* must answer. Both nonces are signed, giving a two-way proof, and both feed HKDF so the session key is bound to this exact exchange.
4. **Q: How does the handshake stop a replayed HELLO?**
A: Two ways: `TimestampValidator` rejects it if older than 5s, and `ReplayCache` rejects it if its nonce was already seen (remembered for 300s).
5. **Q: How is the session key the same on both sides if it is never sent?**
A: ECDH: each side combines its ephemeral private key with the peer’s ephemeral public key to get the identical shared secret; both then run HKDF with the same nonce order, producing the identical 32-byte key.
6. **Q: Where does forward secrecy come from in the handshake?**
A: The ECDH key pairs are *ephemeral* — generated by `generate_ecdh_keypair` for this session and discarded. A later theft of the long-term certificate key cannot rebuild them.
7. **Q: What exception is raised for a forged signature, and where?**
A: `ecdsa_verify` returns `False`, and `process_challenge` (or `process_response`) raises `AuthError` — “Challenge signature invalid, possible impersonation”.
8. **Q: Name a limitation of this handshake.**
A: It needs clocks synchronised within 5s, so heavy clock skew could reject an honest handshake; the ACK has no timestamp field, so it is not freshness-checked; and the parameter `original_hello` in `process_response` is passed but unused. Networking is added separately in S6.

4.9 Glossary

Handshake

A short fixed exchange of messages that sets up a secure session.

Authentication

Proving who you are.

Mutual authentication

Both parties prove identity to each other.

Initiator

The vehicle that starts the handshake (sends HELLO).

Responder

The vehicle that answers the handshake.

Challenge-response

A test: send a fresh value, demand an operation only the genuine party can perform.

Nonce

A fresh single-use random value (here 32 bytes, as hex).

HELLO

Step 1 message: certificate + nonce + timestamp.

CHALLENGE

Step 2 message: certificate + nonce + signature + timestamp.

RESPONSE

Step 3 message: signature + ECDH public key + timestamp.

ACK

Step 4 message: ECDH public key, completing the handshake.

ReplayCache

A store of seen nonces that rejects duplicates.

TimestampValidator

A check that rejects messages older (or newer) than 5 s.

Replay attack

Re-sending a previously captured valid message.

Stale message

A message whose timestamp is outside the allowed window.

Dataclass

A Python class that simply holds named fields.

JSON

A text format for structured data, used as the wire format.

Serialization

Converting an object to bytes/text for sending.

ECDH

Elliptic-Curve Diffie-Hellman key exchange.

Ephemeral key

A key generated for one session then discarded.

HKDF

The key-derivation function turning the shared secret into the key.

Session key

The shared 32-byte AES key produced by the handshake.

Forward secrecy

Past sessions stay secret even if a key leaks later.

AuthError

This project's exception for a failed authentication step.

ReplayError

Exception raised by the ReplayCache for a repeated nonce.

StaleMessageError

Exception raised for an out-of-window timestamp.

CertificateError

Exception raised when certificate verification fails.

tbs_certificate_bytes

The “to-be-signed” body bytes of a certificate.

4.10 References and Further Study

- **Mutual authentication (Wikipedia)** — https://en.wikipedia.org/wiki/Mutual_authentication
Read for: why both sides must authenticate.
- **Challenge–response authentication (Wikipedia)** — https://en.wikipedia.org/wiki/Challenge%E2%80%93response_authentication
Read for: the core idea behind the nonce + signature proof.
- **Cryptographic nonce (Wikipedia)** — https://en.wikipedia.org/wiki/Cryptographic_nonce
Read for: why a fresh single-use value defeats replay.
- **Replay attack (Wikipedia)** — https://en.wikipedia.org/wiki/Replay_attack
Read for: the attack the ReplayCache and TimestampValidator block.
- **TLS 1.3 handshake (RFC 8446 — read Section 2)** — <https://datatracker.ietf.org/doc/html/rfc8446>
Read for: a real-world handshake using the same ideas (certificates, signatures, ephemeral key exchange).
- **Diffie–Hellman key exchange (Wikipedia)** — https://en.wikipedia.org/wiki/Diffie%E2%80%93Hellman_key_exchange
Read for: how both sides reach the same secret without sending it.
- **Forward secrecy (Wikipedia)** — https://en.wikipedia.org/wiki/Forward_secrecy
Read for: why ephemeral handshake keys matter.
- **Python dataclasses documentation** — <https://docs.python.org/3/library/dataclasses.html>
Read for: how the four message classes work.
- **Python json module documentation** — <https://docs.python.org/3/library/json.html>
Read for: `json.dumps` / `json.loads` used by `to_json` / `from_json`.

- **Python cryptography — elliptic-curve docs** — <https://cryptography.io/en/latest/hazmat/primitives/asymmetric/ec/>
Read for: the ECDH and ECDSA calls the handshake relies on.
- **NIST SP 800-63B — authentication guidance** — <https://pages.nist.gov/800-63-3/sp800-63b.html>
Read for: professional terminology around authentication and replay.

End of Section S4. The handshake's output — the session key — is consumed by S5 (Secure Messaging).

Chapter 5

S5 — Secure Messaging

5.1 Title and “What You’ll Learn”

Section Title

S5 — Secure Messaging: how a plain safety message is signed, encrypted, packed into wire bytes, and then unpacked, decrypted and verified on arrival.

Plain-English Summary (read this first)

Once the handshake of Section S4 has finished, the two cars share a secret key and can finally talk. Section S5 is the file that turns a plain **Basic Safety Message (BSM)** — “I am going 80 km/h, not braking” — into a protected packet safe to send across the untrusted network, and turns it back again on the other side. The recipe is called **Sign-then-Encrypt**: first stamp the message with a digital signature, then seal everything inside an encrypted, tamper-evident envelope. The file also defines the exact byte layout of the packet (the *wire format*) and a small `MessageLogger` that records what happened for the dashboard.

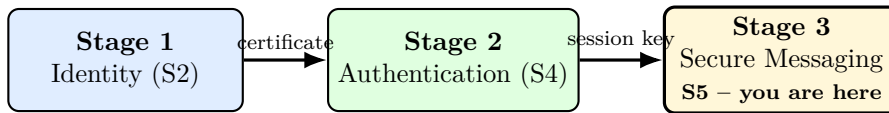
Learning Outcomes

After studying this section you will be able to:

- Describe the `BasicSafetyMessage` dataclass and every field it carries.
- Explain **Sign-then-Encrypt** and why it is preferred over Encrypt-then-Sign.
- Explain the `SecureBSM` wire format byte-by-byte.
- Explain **length-prefixing** and big-endian packing with the Python `struct` module.
- Explain how **sequence numbers** and **timestamps** stop replay.
- Walk through `secure_send` and `secure_receive` step by step.
- Run the messaging demo, read its output, and force a tamper rejection.

5.2 Where This Fits

S5 is **Stage 3: Secure Messaging** — the final stage of the pipeline. It *consumes* the session key produced by the S4 handshake, and it *uses* the cryptographic tools of S3 (`aes_gcm_encrypt`, `aes_gcm_decrypt`, `ecdsa_sign`, `ecdsa_verify`). Its output — a stream of wire bytes — is handed to the networking code of S6 to actually travel across a TCP connection.



Output: SecureBSM wire bytes, handed to the networking layer (S6).

5.3 The Theory You Must Study

5.3.1 The Basic Safety Message (BSM)

Plain definition. A BSM is the small data record a vehicle broadcasts: its speed, heading, position, braking state and so on. In code it is the `BasicSafetyMessage` dataclass.

Real-life analogy. A quick spoken status update: “This is car A, doing 80, heading north-east, not braking, message number 5.”

Why it exists. Other cars need a small, standard, frequently-updated record of what each nearby vehicle is doing, so they can react to danger.

How it works here. The dataclass carries nine fields: `vehicle_id`, `speed_kmh`, `heading_deg` (0 = North, 90 = East), `latitude`, `longitude`, `timestamp`, `sequence_num`, `brake_applied`, and `acceleration`. Real V2V broadcasts a BSM about ten times a second; this project sends one per second — a deliberate slowdown so a human can watch the dashboard.

Security property it relates to. The BSM is the *thing being protected*; the rest of S5 is the protection.

5.3.2 Sign-then-Encrypt

Plain definition. “Sign-then-Encrypt” means: first create a digital signature over the *plaintext* message, then encrypt the message *and* its signature together.

Real-life analogy. You sign a letter by hand, then put the signed letter inside a sealed, tamper-evident envelope. Anyone who opens the envelope finds a letter that already carries your signature.

Why it exists — the alternative is worse. The opposite order, Encrypt-then-Sign, signs the *ciphertext*. An attacker could then peel that signature off and glue it onto a *different* ciphertext, because the signature was never tied to the actual message content. By signing the plaintext first, the signature is bound to what the message really says. The file’s own docstring states this reasoning explicitly.

How it works here. `secure_send` runs: `serialize BSM` → `ecdsa_sign` the BSM bytes → glue `[length] [BSM] [signature]` together → `aes_gcm_encrypt` the whole thing.

Security property it provides. **Non-repudiation** and **authentication** (from the signature) wrapped inside **confidentiality** and **integrity** (from the encryption).

5.3.3 ECDSA Digital Signatures (recap from S3)

A digital signature is a value made with a vehicle’s *private* key that anyone with its *public* key can verify. It proves *who* wrote the message and that it has not changed — and because only the owner could sign, they cannot later deny it. S5 simply calls `ecdsa_sign` when sending and `ecdsa_verify` when receiving. *Security property: non-repudiation, authentication, integrity.*

5.3.4 AES-256-GCM Authenticated Encryption (recap from S3)

AES-256-GCM both hides the message (confidentiality) and attaches a 16-byte authentication tag that detects any change (integrity). S5 calls `aes_gcm_encrypt` when sending and `aes_gcm_decrypt` when receiving; a tampered packet makes decryption fail. *Security property: confidentiality, integrity.*

5.3.5 Serialization: JSON and Binary Packing

Plain definition. *Serialization* turns a structured object into a flat sequence of bytes. S5 uses *two* kinds:

- **JSON** (JavaScript Object Notation) — a human-readable text format. The BSM fields are turned into JSON text, then into bytes (`to_bytes` uses `json.dumps`).
- **Binary struct packing** — a compact fixed-layout byte format, used for the outer `SecureBSM` envelope.

Real-life analogy. JSON is filling in a paper form with words you can read. Binary packing is a barcode: dense, exact, not human-readable, but quick and unambiguous for a machine.

Why two formats? The BSM *content* benefits from JSON's flexibility. The outer envelope needs an exact, compact layout so the receiver can find each field at a known position — JSON would be wasteful and harder to parse byte-precisely.

Security property it provides. None directly — it is about format.

5.3.6 Length-Prefixing and Big-Endian (the `struct` module)

Plain definition. *Length-prefixing* means writing a field's length *just before* the field itself, so the reader knows exactly how many bytes to take. *Big-endian* is the convention of writing the most significant byte first (the “natural” left-to-right order for numbers).

Real-life analogy. A parcel label that says “contents: 1742 grams” before you open it — you know what to expect. Big-endian is writing the number “one thousand” as 1000, not 0001.

Why it exists. Variable-length fields (the sender's name, the ciphertext) have no fixed size. Without a length written in front, the receiver cannot tell where one field ends and the next begins.

How it works here. The `struct` module packs numbers into bytes: `struct.pack(">H", n)` writes `n` as a 2-byte big-endian unsigned integer ("`>`" = big-endian, "`H`" = 2-byte), and `struct.pack(">I", n)` writes a 4-byte one ("`I`" = 4-byte). `struct.unpack_from` reads them back at a given offset.

Security property it provides. None directly — but *reliable* framing avoids parsing errors that could be abused.

5.3.7 Sequence Numbers and Replay Defence

Plain definition. A *sequence number* is a counter that increases by one with every message a vehicle sends. The receiver remembers the last number it saw and rejects anything not strictly greater.

Real-life analogy. Numbered tickets at a deli counter. If you are holding ticket 8 and someone hands you ticket 5 again, you know it is an old, re-used ticket.

Why it exists. A replayed message carries an *old* sequence number. A suppressed-and-delayed message arrives out of order. Requiring the number to always increase catches both — a replayed or stale BSM has a number less than or equal to the last accepted one.

How it works here. `secure_receive` takes a `last_seq` argument; if the new BSM's `sequence_num` is `<= last_seq` it raises `SequenceError`. Combined with the 5-second timestamp window (`TimestampValidator`, from S4), an old message has two ways to be caught.

Honest note. `secure_receive` also *accepts* a `replay_cache` argument, but — unlike the handshake — it does *not* actually consult the nonce cache in its body. For BSMs the replay defence is the *sequence number* plus the *timestamp window*, not the nonce cache. This is worth stating plainly if an examiner asks.

Security property it provides. **Replay prevention.**

5.4 Code Walkthrough

`v2v_protocol.py` contains: three exceptions, two dataclasses (`BasicSafetyMessage`, `SecureBSM`), the `MessageLogger` class, and the two key functions `secure_send` and `secure_receive`.

5.4.1 The three exceptions

```
1 class TamperError(Exception):
2     """Message was modified in transit (AES-GCM tag mismatch or bad sig)."""
3
4 class ReplayError(Exception):
5     """Duplicate message detected (same nonce or old timestamp)."""
6
7 class SequenceError(Exception):
8     """Sequence number out of order -- possible message suppression."""
```

Listing 5.1: `v2v_protocol.py` — the messaging exceptions.

Explanation: `TamperError` means the packet failed an integrity check (the AES-GCM tag or the ECDSA signature). `SequenceError` means the sequence number went backwards. `ReplayError` is defined for completeness. Naming each makes the dashboard’s alert log precise.

5.4.2 `BasicSafetyMessage` — the plain message

```
1 @dataclass
2 class BasicSafetyMessage:
3     vehicle_id: str
4     speed_kmh: float
5     heading_deg: float      # 0 = North, 90 = East
6     latitude: float
7     longitude: float
8     timestamp: float
9     sequence_num: int
10    brake_applied: bool
11    acceleration: float = 0.0
12
13    def to_bytes(self) -> bytes:
14        return json.dumps(asdict(self)).encode("utf-8")
15
16    @classmethod
17    def from_bytes(cls, data: bytes) -> BasicSafetyMessage:
18        return cls(**json.loads(data.decode("utf-8")))
```

Listing 5.2: `v2v_protocol.py` — the `BasicSafetyMessage` dataclass.

Step by step:

1. The nine fields hold everything one safety broadcast says.
2. `to_bytes`: `asdict(self)` turns the object into a plain dictionary; `json.dumps` turns that into JSON text; `.encode("utf-8")` turns the text into bytes ready to be signed and encrypted.
3. `from_bytes`: the reverse — decode bytes to text, parse the JSON to a dictionary, and build a fresh `BasicSafetyMessage`.

Theory link: JSON serialization. *Analogy:* filling in the paper form (`to_bytes`) and reading it back (`from_bytes`).

5.4.3 `SecureBSM` — the on-the-wire envelope

```

1 @dataclass
2 class SecureBSM:
3     ciphertext: bytes
4     nonce: bytes          # 12-byte AES-GCM nonce
5     auth_tag: bytes       # 16-byte AES-GCM tag
6     sender_id: str
7     sequence_num: int
8
9     def to_bytes(self) -> bytes:
10         # Wire format:
11         # [sid_len:2][sid][seq:4][nonce:12][tag:16][ct_len:4][ciphertext]
12         sid = self.sender_id.encode("utf-8")
13         parts = [
14             struct.pack(">H", len(sid)),
15             sid,
16             struct.pack(">I", self.sequence_num),
17             self.nonce,      # always 12 bytes
18             self.auth_tag,   # always 16 bytes
19             struct.pack(">I", len(self.ciphertext)),
20             self.ciphertext,
21         ]
22         return b"".join(parts)
23
24     @classmethod
25     def from_bytes(cls, data: bytes) -> SecureBSM:
26         offset = 0
27         sid_len = struct.unpack_from(">H", data, offset)[0]; offset += 2
28         sender_id = data[offset:offset + sid_len].decode("utf-8"); offset +=
            sid_len
29         seq = struct.unpack_from(">I", data, offset)[0]; offset += 4
30         nonce = data[offset:offset + 12]; offset += 12
31         tag = data[offset:offset + 16]; offset += 16
32         ct_len = struct.unpack_from(">I", data, offset)[0]; offset += 4
33         ciphertext = data[offset:offset + ct_len]
34         return cls(ciphertext, nonce, tag, sender_id, seq)

```

Listing 5.3: v2v_protocol.py — SecureBSM and its binary wire format.

Step by step (to_bytes):

1. Encode the sender's name to bytes.
2. Write its length as a 2-byte big-endian number (">H"), then the name itself — length-prefixing in action.
3. Write the sequence number as a 4-byte big-endian number (">I").
4. Append the nonce (always 12 bytes) and the auth tag (always 16 bytes) — no length prefix needed because their sizes are fixed.
5. Write the ciphertext length (4 bytes), then the ciphertext.
6. `b"".join(parts)` concatenates everything into one byte string.

from_bytes walks an `offset` cursor through the bytes, reading each field at its known position — the exact reverse of **to_bytes**. **Theory link:** binary serialization, length-prefixing, big-endian. *Analogy:* packing a parcel with a clear packing list, then unpacking it item by item.

5.4.4 MessageLogger — the record keeper

Purpose: keep a rolling record of sent/received messages and security alerts, plus counters, for the dashboard (S7) to display.

```

1 class MessageLogger:

```

```

2  def __init__(self, max_messages: int = 50) -> None:
3      self._messages: list[dict] = []
4      self._alerts: list[dict] = []
5      self._max = max_messages
6      self.total_sent = 0
7      self.total_received = 0
8      self.tamper_blocked = 0
9      self.replay_blocked = 0
10
11  def log_sent(self, bsm): ...          # +1 total_sent, append a SENT entry
12  def log_received(self, bsm, security_props): ... # +1 total_received
13  def log_alert(self, alert_type, details):
14      if alert_type == "TAMPER": self.tamper_blocked += 1
15      elif alert_type == "REPLAY": self.replay_blocked += 1
16      self._alerts.append({...})

```

Listing 5.4: v2v_protocol.py — MessageLogger (key parts).

Step by step:

1. It holds two rolling lists — `_messages` and `_alerts` — and four counters.
2. `log_sent` / `log_received` append a small dictionary describing the message and trim the list to `max_messages` (default 50) so memory does not grow forever.
3. `log_alert` bumps `tamper_blocked` or `replay_blocked` depending on the alert type, and records the alert with a timestamp.
4. `get_recent`, `get_alerts` and `clear_alerts` are read/clear helpers the dashboard calls.

Theory link: this is *observability*, not a defence — it lets a human *see* that the defences fired.

Analogy: the CCTV logbook at a building's front desk.

5.4.5 secure_send() — Sign-then-Encrypt

```

1  def secure_send(bsm, session_key, signing_key, logger) -> bytes:
2      # Step 1: serialize BSM to bytes
3      bsm_bytes = bsm.to_bytes()
4      # Step 2: sign the PLAINTEXT (Sign-then-Encrypt)
5      signature = ecdsa_sign(signing_key, bsm_bytes)
6      # Step 3: combine with a 4-byte length prefix
7      payload = struct.pack(">I", len(bsm_bytes)) + bsm_bytes + signature
8      # Step 4: AES-256-GCM encrypt -- confidentiality + integrity
9      ciphertext, nonce, auth_tag = aes_gcm_encrypt(session_key, payload)
10     # Step 5: wrap in SecureBSM and return wire bytes
11     secure = SecureBSM(ciphertext, nonce, auth_tag, bsm.vehicle_id, bsm.
12         sequence_num)
13     logger.log_sent(bsm)
14     return secure.to_bytes()

```

Listing 5.5: v2v_protocol.py — secure_send: the sending pipeline.

Step by step:

1. **Serialize** — turn the BSM into JSON bytes.
2. **Sign the plaintext** — `ecdsa_sign` produces a signature over those exact bytes. Doing this *before* encryption is the whole point of Sign-then-Encrypt.
3. **Combine** — glue [4-byte length] [BSM bytes] [signature] so the receiver can later split them apart again.
4. **Encrypt** — `aes_gcm_encrypt` encrypts the combined payload, returning ciphertext, a fresh nonce, and a 16-byte tag.

5. **Wrap** — build a `SecureBSM`, log the send, and return `secure.to_bytes()` — the final wire bytes.

Arguments: `bsm` (the message), `session_key` (32-byte AES key from the handshake), `signing_key` (this vehicle's ECDSA private key), `logger`. **Returns:** wire-format bytes. **Theory link:** Sign-then-Encrypt; ECDSA; AES-GCM; length-prefixing.

5.4.6 `secure_receive()` — Decrypt-then-Verify

```
1 def secure_receive(data, session_key, peer_verify_key, replay_cache,
2                     timestamp_validator, logger, last_seq=None):
3     # Step 1: parse SecureBSM from raw bytes
4     secure = SecureBSM.from_bytes(data)
5     # Step 2: AES-GCM decrypt -- 1 changed bit -> failure
6     try:
7         plaintext = aes_gcm_decrypt(session_key, secure.ciphertext,
8                                     secure.nonce, secure.auth_tag)
9
10    except DecryptionError as exc:
11        logger.log_alert("TAMPER", f"AES-GCM decryption failed: {exc}")
12        raise TamperError("Message tampered -- AES-GCM auth tag mismatch") from
13            exc
14    # Step 3: split plaintext into BSM bytes + signature
15    bsm_len = struct.unpack_from(">I", plaintext, 0)[0]
16    bsm_bytes = plaintext[4:4 + bsm_len]
17    signature = plaintext[4 + bsm_len:]
18    # Step 4: verify the ECDSA signature
19    if not ecdsa_verify(peer_verify_key, bsm_bytes, signature):
20        logger.log_alert("TAMPER", "ECDSA signature verification failed")
21        raise TamperError("Invalid ECDSA signature -- message not authentic")
22    # Step 5: parse BSM, check sequence number, validate timestamp
23    bsm = BasicSafetyMessage.from_bytes(bsm_bytes)
24    if last_seq is not None and bsm.sequence_num <= last_seq:
25        logger.log_alert("REPLAY", f"Seq {bsm.sequence_num} <= last {last_seq}")
26        raise SequenceError(f"Sequence {bsm.sequence_num} not greater than {
27            last_seq}")
28    timestamp_validator.validate(bsm.timestamp)
29    logger.log_received(bsm, {"authenticated": True, "encrypted": True,
30                            "integrity_verified": True, "replay_protected":
31                                True})
32    return bsm
```

Listing 5.6: `v2v_protocol.py` — `secure_receive`: the receiving pipeline.

Step by step:

1. **Parse** the raw bytes back into a `SecureBSM` (`from_bytes`).
2. **Decrypt** with `aes_gcm_decrypt`. If a single bit was changed, the tag check fails: the code catches `DecryptionError`, logs a `TAMPER` alert, and raises `TamperError`.
3. **Split** the decrypted payload using the 4-byte length prefix into the BSM bytes and the signature.
4. **Verify the signature** with `ecdsa_verify` against the peer's public key. If it fails, log a `TAMPER` alert and raise `TamperError`. This is the second integrity check — and the one that gives **non-repudiation**.
5. **Parse and validate**. Rebuild the BSM. If the sequence number is not greater than `last_seq`, raise `SequenceError` (replay). Then `timestamp_validator.validate` rejects a stale timestamp. If all pass, log a successful receive and return the BSM.

Arguments / return / exceptions: returns the verified `BasicSafetyMessage`; raises `TamperError` (bad tag or bad signature), `SequenceError` (replay), or `StaleMessageError` (stale timestamp).

Honest note: the `replay_cache` parameter is accepted but not used in the body — BSM replay defence here is the sequence number and timestamp. **Theory link:** AES-GCM integrity, ECDSA authentication, sequence/timestamp replay defence.

5.4.7 The demo block

The `if __name__ == "__main__"` block builds a session key, creates a BSM, calls `secure_send` then `secure_receive` (showing a clean round-trip), then flips a byte and shows the tampered message is rejected with a `TamperError`. We run it next.

5.5 Hands-On / Practical

5.5.1 Run the messaging protocol by itself

`v2v_protocol.py` needs no certificates and no network — run it directly:

```
python protocol\v2v_protocol.py
```

Expected output (sample):

```
=== V2V MESSAGE PROTOCOL DEMO ===

Session key: 4a9c1f0b7e2d6688c3a5...

[SEND] BSM: speed=72.5 km/h, seq=1
[WIRE] 338 bytes on the wire (encrypted)
[RECV] BSM: speed=72.5 km/h, seq=1
[OK] All security checks passed!

--- TAMPER TEST ---
[OK] Tampered message REJECTED: Message tampered -- AES-GCM auth tag mismatch

Stats: sent=1, recv=1, tamper_blocked=1
```

5.5.2 What the output lines mean

- **Session key** — a 32-byte key (here simulated, normally from the S4 handshake).
- **[SEND]** — the plain BSM before protection.
- **[WIRE] 338 bytes** — the size of the signed-and-encrypted packet; the exact number depends on the JSON content.
- **[RECV] ... [OK]** — decryption, signature check, sequence and timestamp checks all passed; the message came back identical.
- **Tampered message REJECTED** — one flipped byte made the AES-GCM tag mismatch, so `secure_receive` raised `TamperError`.
- **tamper_blocked=1** — the `MessageLogger` counted the blocked attack.

5.5.3 Try This Yourself (mini-experiment)

In the demo's tamper test, the line `tampered[len(tampered) // 2] ^= 0xFF` flips one byte in the *middle* of the packet. Change it to flip a byte near the *start* instead:

```
tampered[0] ^= 0xFF          # flip the very first byte
```

Predict before you run. Byte 0 is part of the `sid_len` field (the 2-byte sender-name length). Corrupting it makes `SecureBSM.from_bytes` read a wrong length and the unpacking fails *before* decryption even starts — so instead of a clean `TamperError` you may see a different exception (for example a `struct` or decode error), caught by the demo’s general `except Exception` branch as “Message rejected”. Lesson: tampering is caught *somewhere* on every path — but exactly which check fires depends on *which* bytes you damage.

5.6 Attack Relevance

S5 is the layer that defends *every individual safety message*. It is exercised directly by `attacks/tamper_attack.py` and `attacks/replay_attack.py`.

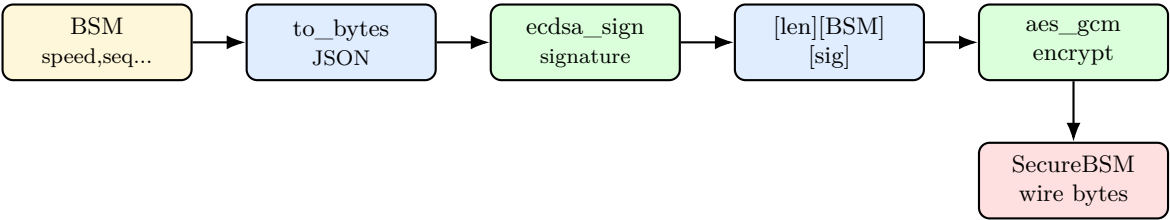
Attack	How S5 stops it	The mechanism
Tamper	A changed byte breaks the AES-GCM tag; a forged body breaks the ECDSA signature.	<code>aes_gcm_decrypt</code> / <code>ecdsa_verify</code> → <code>TamperError</code> .
Replay	A re-sent BSM carries an old sequence number or a stale timestamp.	<code>sequence check</code> → <code>SequenceError</code> ; <code>TimestampValidator</code> → <code>StaleMessageError</code> .
Eavesdrop	The BSM travels as AES-256 ciphertext; unreadable without the session key.	<code>aes_gcm_encrypt</code> in <code>secure_send</code> .
Forgery / denial	The plaintext signature ties the message to its true author, who cannot deny it.	ECDSA signature (non-repudiation).

Table 5.1: S5 defends confidentiality, integrity, replay prevention and non-repudiation per message.

Why Sign-then-Encrypt matters for the attacker. Because the signature covers the *plaintext*, an attacker cannot take a valid signature from one message and re-attach it to a different one — the signature would not match the new content. And because the signature is *inside* the encrypted envelope, the attacker cannot even read or strip it without the session key. **Honest scope note:** S5 protects each message, but it relies on a correct session key from S4; if the handshake were skipped or compromised, S5 alone could not help.

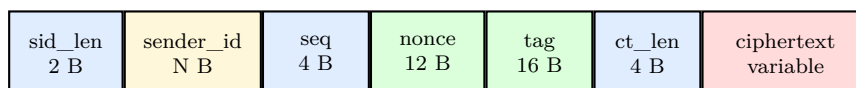
5.7 Illustrations

5.7.1 Diagram 1 — the Sign-then-Encrypt pipeline



Receiving runs this in reverse: *decrypt, split, verify signature, check seq + ti*

5.7.2 Diagram 2 — the SecureBSM wire format



Each field sits at a known position; variable-length fields (sender_id, ciphertext) are preceded by a length. B = bytes.

5.8 Presentation Pack

5.8.1 Slide Outline

Slide 1 — The Job of S5.

- The handshake gave us a shared key — now we can talk.
- A BSM is a tiny “speed, heading, braking” status message.
- S5 turns a plain BSM into a safe packet, and back again.
- One file: `v2v_protocol.py`.

Slide 2 — Sign-then-Encrypt.

- First sign the plaintext, then encrypt message + signature together.
- Sign first → the signature is tied to the real content.
- Encrypt-then-Sign would let an attacker swap signatures.

- Result: a signed letter inside a sealed envelope.

Slide 3 — The SecureBSM Wire Format.

- An exact byte layout: lengths, ids, nonce, tag, ciphertext.
- Length-prefixing tells the receiver where each field ends.
- Big-endian packing via Python’s `struct` module.
- Fixed-size fields (nonce 12 B, tag 16 B) need no length.

Slide 4 — Receiving: Four Checks.

- Decrypt — AES-GCM tag catches any tampering.
- Verify signature — ECDSA proves the true author.
- Sequence number — must always increase (anti-replay).
- Timestamp — must be within 5 seconds (anti-replay).

Slide 5 — What S5 Guarantees Per Message.

- Confidentiality + integrity (AES-256-GCM).
- Authentication + non-repudiation (ECDSA signature).
- Replay prevention (sequence + timestamp).
- A single flipped bit → `TamperError`.

5.8.2 Speaker Talking Points (about 30–60 seconds each)

Slide 1. “After the handshake in the previous section, our two cars share a secret key. Now they can actually talk. What they say are Basic Safety Messages — tiny records of speed, heading, position and braking. Section five is the file that protects each of those messages. It takes a plain

BSM, wraps it in security, produces the exact bytes that go on the wire, and on the other side unwraps it and checks it. It is all in one file, `v2v_protocol.py`.”

Slide 2. “The protection recipe is called Sign-then-Encrypt. We first make a digital signature over the plain message, and only then encrypt the message and its signature together. Order matters. If we encrypted first and signed the ciphertext, an attacker could peel that signature off and stick it on a different ciphertext, because the signature was never connected to the real words. By signing the plaintext first, the signature is glued to what the message actually says. Picture signing a letter by hand, then sealing it in a tamper-evident envelope.”

Slide 3. “A network only carries a flat stream of bytes, so we need an exact layout — the wire format. The SecureBSM packet is: a two-byte length, then the sender’s name, then a four-byte sequence number, then the twelve-byte nonce, the sixteen-byte tag, a four-byte ciphertext length, and finally the ciphertext. The trick is length-prefixing: before any variable-size field we write its length, so the receiver knows exactly where it ends. We build this with Python’s struct module in big-endian order.”

Slide 4. “Receiving is the reverse, and it is four checks in a row. First we decrypt — and AES-GCM’s tag means any tampering makes decryption fail outright. Second we verify the ECDSA signature, which proves the message really came from the claimed car. Third we check the sequence number is larger than the last one we accepted — an old or replayed message has an old number. Fourth we check the timestamp is fresh, within five seconds. Only a message that passes all four is delivered.”

Slide 5. “So per message, S5 delivers four security properties at once. AES-256-GCM gives confidentiality and integrity. The ECDSA signature gives authentication and non-repudiation — the sender cannot deny it. And the sequence number plus timestamp give replay prevention. The demo proves it: flip a single bit anywhere in the packet and the receiver throws a `TamperError` instead of trusting the message.”

5.8.3 Live-Demo Script

1. **Run the demo.** Execute `python protocol\v2v_protocol.py`.
2. **Point at the round-trip.** [SEND] then [RECV] with the same speed and sequence number — “protected and recovered perfectly.”
3. **Point at the wire size.** [WIRE] 338 bytes — “that is the signed, encrypted packet.”
4. **Point at the tamper test.** Tampered message REJECTED — “one byte changed, the whole message refused.”
5. **Optional break-it.** Change the flipped index to 0 (see the mini-experiment) and show a different rejection path.

5.8.4 Examiner Q&A

1. **Q: What is a BSM and what fields does it carry here?**
A: A Basic Safety Message. The `BasicSafetyMessage` dataclass carries vehicle id, speed, heading, latitude, longitude, timestamp, sequence number, brake flag and acceleration.
2. **Q: What is Sign-then-Encrypt and why not the reverse?**
A: Sign the plaintext first, then encrypt message + signature. The reverse (signing the ciphertext) would let an attacker move a signature onto a different ciphertext, since it would not be bound to content.
3. **Q: Describe the SecureBSM wire format.**
A: `[sid_len:2][sender_id][seq:4][nonce:12][tag:16][ct_len:4][ciphertext]` — length-prefixed, big-endian, built with `struct`.

4. **Q: Why length-prefix the variable fields?**

A: The sender name and ciphertext have no fixed size; writing the length first tells the receiver exactly how many bytes to read.

5. **Q: How does `secure_receive` detect a tampered message?**

A: Two ways: `aes_gcm_decrypt` fails if the ciphertext/nonce/tag changed (`DecryptionError` → `TamperError`), and `ecdsa_verify` fails if the body does not match the signature.

6. **Q: How does S5 stop replay?**

A: The sequence number must be strictly greater than the last accepted one (`SequenceError` otherwise), and the timestamp must be within 5 seconds.

7. **Q: The `replay_cache` argument — is it used?**

A: It is accepted by `secure_receive` but not consulted in the body. BSM replay defence is the sequence number plus the timestamp; the nonce cache is used by the handshake, not here.

8. **Q: What does `MessageLogger` contribute to security?**

A: Nothing directly — it is observability. It records sent/received messages and counts blocked tamper/replay attempts so the dashboard can display them.

5.9 Glossary

BSM

Basic Safety Message: the small status broadcast of a vehicle.

BasicSafetyMessage

The dataclass holding the BSM fields.

SecureBSM

The signed-and-encrypted on-the-wire form of a BSM.

Sign-then-Encrypt

Sign the plaintext, then encrypt message + signature.

Plaintext

The original readable message before encryption.

Ciphertext

The scrambled output of encryption.

Wire format

The exact byte layout of a packet sent over a network.

Serialization

Turning an object into a flat sequence of bytes.

JSON

A human-readable text format for structured data.

struct module

Python module that packs/unpacks numbers into bytes.

Length-prefixing

Writing a field's length just before the field.

Big-endian

Byte order with the most significant byte first.

Nonce

A one-time 12-byte value for AES-GCM encryption.

Authentication tag

The 16-byte AES-GCM value that detects tampering.

ECDSA

Elliptic Curve Digital Signature Algorithm.

Digital signature

A private-key value provable with the public key.

Sequence number

A per-sender counter that must always increase.

Timestamp

The recorded creation time, checked for freshness.

Session key

The shared 32-byte AES key from the handshake.

TamperError

Exception meaning a packet failed an integrity check.

SequenceError

Exception meaning the sequence number went backwards.

MessageLogger

Class that records messages and alerts for the dashboard.

Non-repudiation

A sender cannot later deny having sent a message.

Confidentiality

Only the intended recipient can read the message.

Integrity

Any change to the message is detected.

5.10 References and Further Study

- **Python struct module documentation** — <https://docs.python.org/3/library/struct.html>
Read for: the format codes ">H" and ">I" used in the wire format.
- **Python json module documentation** — <https://docs.python.org/3/library/json.html>
Read for: `json.dumps` / `json.loads` used by the BSM.
- **Python dataclasses documentation** — <https://docs.python.org/3/library/dataclasses.html>
Read for: how `BasicSafetyMessage` and `SecureBSM` work.
- **SAE J2735 (Basic Safety Message standard)** — https://www.sae.org/standards/content/j2735_202309/
Read for: the real-world BSM this project is inspired by.
- **Authenticated encryption (Wikipedia)** — https://en.wikipedia.org/wiki/Authenticated_encryption
Read for: why AES-GCM gives confidentiality and integrity together.
- **Sign-then-encrypt discussion (Cryptography Stack Exchange)** — <https://crypto.stackexchange.com/questions/5458/should-we-sign-then-encrypt-or-encrypt-then-sign>
Read for: the trade-offs between signing and encrypting order.

- **Endianness (Wikipedia)** — <https://en.wikipedia.org/wiki/Endianness>
Read for: what big-endian byte order means.
- **Digital signature (Wikipedia)** — https://en.wikipedia.org/wiki/Digital_signature
Read for: authentication, integrity and non-repudiation.
- **Python cryptography — AEAD docs** — <https://cryptography.io/en/latest/hazmat/primitives/aead/>
Read for: the AES-GCM functions S5 calls.
- **Replay attack (Wikipedia)** — https://en.wikipedia.org/wiki/Replay_attack
Read for: why sequence numbers and timestamps defeat replay.

End of Section S5. Next: S6 — the Vehicle Node and TCP networking that carries these SecureBSM bytes.

Chapter 6

S6 — The Vehicle Node and TCP Networking

6.1 Title and “What You’ll Learn”

Section Title

S6 — The Vehicle Node: the program you actually run, which connects two cars over a real network and drives the whole security pipeline end to end.

Plain-English Summary (read this first)

Everything so far — certificates, crypto, the handshake, the message protocol — has been a *library*. Section S6 is the **program**. One file, `vehicle_node.py`, is launched once per car. One car acts as the **server** (it waits for a connection); the other acts as the **client** (it dials in). They make a real network connection using **TCP**, run the 4-step handshake over it, and then continuously send and receive encrypted safety messages — one per second — using background **threads**. S6 is the conductor that calls S2, S3, S4 and S5 in the right order.

Learning Outcomes

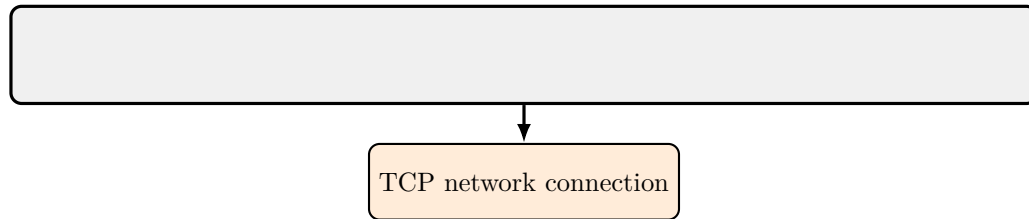
After studying this section you will be able to:

- Explain what **TCP** and a **socket** are.
- Explain the **message-framing** problem and how a 4-byte length header solves it.
- Explain the difference between the **server** and **client** roles.
- Explain why **threads** are needed and what a *daemon* thread is.
- Explain **connection retries** with exponential backoff.
- Walk through every function and method of `vehicle_node.py`.
- Launch two vehicle nodes, read their logs, and explain how the receive loop reacts to an attack.

6.2 Where This Fits

S6 is not one stage of the pipeline — it is the *frame around all three stages*. It loads the identity files from S2, builds the `AuthProtocol` of S4 (which rests on the crypto of S3), runs the handshake, and then repeatedly calls the `secure_send` / `secure_receive` of S5. It also starts the dashboard of S7. In short: S6 is the integration layer.

S6 – vehicle_node.py drives all of these in order



6.3 The Theory You Must Study

6.3.1 TCP and Sockets

Plain definition. TCP (Transmission Control Protocol) is a way for two programs to send data to each other reliably and in order across a network. A **socket** is the software “end point” of a connection — the object a program reads from and writes to, like the handset of a telephone.

Real-life analogy. A telephone call. TCP is the phone system that makes sure your words arrive, in order, with nothing dropped. The socket is the handset in your hand.

Why it exists. The two vehicles run as separate programs (possibly on separate laptops). They need a dependable channel: TCP guarantees that bytes arrive complete and in the order they were sent — something a raw network does not promise.

How it works here. The code uses Python’s `socket` module. `socket.socket(AF_INET, SOCK_STREAM)` creates a TCP socket. The server *binds* to a port and *listens*; the client *connects*.

Security property it provides. None. **This is important:** TCP is *not* encrypted. The radio/network is still untrusted. All the security comes from the *payload* (the SecureBSM of S5), not from TCP.

6.3.2 The Message-Framing Problem

Plain definition. TCP delivers a *byte stream* — an unbroken flow of bytes with *no markers* showing where one message ends and the next begins. *Framing* is the technique of adding those markers.

Real-life analogy. Imagine a friend reading you a long list of phone numbers with no pauses: “07700900461077009005...”. Without a gap or a stated length, you cannot tell where one number ends. Framing is agreeing in advance: “each number is exactly 11 digits”.

Why it exists. A single `recv()` call might return half a message, or two messages stuck together. Without framing the receiver cannot reliably split the stream into messages.

How it works here. The project uses **length-prefix framing**. Before every message, `send_framed` writes a 4-byte big-endian number saying how long the message is. `recv_framed` first reads exactly those 4 bytes, learns the length, then reads exactly that many more bytes.

Security property it provides. None directly — but reliable framing prevents parsing confusion that an attacker might otherwise exploit.

6.3.3 Server and Client Roles

Plain definition. The **server** is the side that waits to be contacted (it *listens* on a port). The **client** is the side that starts the conversation (it *connects* to the server’s address).

Real-life analogy. A shop and a customer. The shop opens its doors and waits (server). The customer walks in (client). After that, both can talk freely.

Why it exists. *Someone* has to be reachable first. One side must be ready and waiting before the other can connect.

How it works here. A vehicle started *without* `-peer-host` is a pure server: it just waits. A vehicle started *with* `-peer-host` also starts its own server thread *and* then connects to the

peer as a client. In the handshake, the client becomes the *initiator* (sends HELLO first) and the server becomes the *responder*.

Security property it provides. None — it is about connection setup.

6.3.4 Threads and Concurrency

Plain definition. A **thread** is a separate line of execution inside the same program, so the program can do more than one thing at a time.

Real-life analogy. One person can only talk *or* listen at once. To have a real conversation you need two people: one always ready to speak, one always ready to listen. Threads give the program that second “person”.

Why it exists. Sending a BSM every second and receiving BSMs at any time must happen *simultaneously*. If the program were stuck waiting to receive, it could never send, and vice versa. So the node runs a *send loop* in one thread and a *receive loop* in another.

How it works here. `threading.Thread(target=..., daemon=True)` creates a thread. A **daemon** thread is one that does not keep the program alive on its own — when the main program exits, daemon threads stop too. The node uses daemon threads for the server-accept loop, the send loop, the receive loop and the dashboard.

Security property it provides. None — it is about doing two jobs at once.

6.3.5 Connection Retries and Exponential Backoff

Plain definition. If the client cannot connect (the server is not ready yet), it tries again. *Exponential backoff* means waiting a bit longer before each retry.

Real-life analogy. Phoning a friend who is busy: you call, wait a minute, call again, wait two minutes, call again — you do not redial non-stop.

Why it exists. When two laptops start at slightly different times, the client may dial before the server is listening. Retrying with growing pauses gives the server time to come up without hammering the network.

How it works here. `connect_to_peer` tries up to five times; on failure it sleeps 2 * attempt seconds (2s, 4s, 6s, 8s) before the next try.

Security property it provides. None — it is about robustness.

6.4 Code Walkthrough

6.4.1 The TCP framing helpers

```
1 def send_framed(sock: socket.socket, data: bytes) -> None:
2     """Send *data* prefixed with a 4-byte length header."""
3     sock.sendall(struct.pack(">I", len(data)) + data)
4
5 def _recv_exactly(sock: socket.socket, n: int) -> bytes:
6     """Block until exactly *n* bytes have been received."""
7     buf = b""
8     while len(buf) < n:
9         chunk = sock.recv(n - len(buf))
10        if not chunk:
11            raise ConnectionError("Connection closed by peer")
12        buf += chunk
13    return buf
14
15 def recv_framed(sock: socket.socket) -> bytes:
16     """Receive a length-prefixed message."""
17     header = _recv_exactly(sock, 4)
18     length = struct.unpack(">I", header)[0]
19     return _recv_exactly(sock, length)
```

Listing 6.1: vehicle_node.py — length-prefix framing helpers.

Step by step:

1. `send_framed`: `struct.pack(">I", len(data))` makes a 4-byte big-endian length; it is glued in front of `data`; `sock.sendall` sends the whole thing.
2. `_recv_exactly`: a single `recv` may return fewer bytes than asked, so this loops, collecting chunks, until it has exactly `n` bytes. If `recv` returns nothing, the peer hung up — it raises `ConnectionError`.
3. `recv_framed`: read exactly 4 bytes (the header), decode the length, then read exactly that many more bytes — one complete message.

Theory link: solves the message-framing problem. *Analogy:* each parcel has its weight on the label, so you know when you have it all.

6.4.2 VehicleNode.__init__ — set the node up

Purpose: load the identity files, build the protocol objects, create the server socket.

```
1 class VehicleNode:
2     def __init__(self, vehicle_id, port, cert_path, key_path, ca_cert_path,
3                   bsm_interval=1.0, dashboard_port=5000):
4         # Load cryptographic material
5         self.cert_pem = Path(cert_path).read_bytes()
6         self.private_key = load_private_key(Path(key_path).read_bytes())
7         self.ca_cert_pem = Path(ca_cert_path).read_bytes()
8         # Build the AuthProtocol instance (S4)
9         self.auth = AuthProtocol(vehicle_id, self.private_key,
10                                  self.cert_pem, self.ca_cert_pem)
11
12         # Message subsystem (S5)
13         self.logger = MessageLogger()
14         self.replay_cache = ReplayCache()
15         self.timestamp_validator = TimestampValidator(max_age_seconds=5.0)
16         # Session state -- filled in after the handshake
17         self.session_key = None
18         self.peer_public_key = None
19         # Networking
20         self.server_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
21         self.server_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
22         self.running = True
23         self._seq = 0
24         self._last_peer_seq = None
```

Listing 6.2: vehicle_node.py — the VehicleNode constructor (key parts).

Step by step:

1. Read the vehicle's certificate, private key and the CA certificate from disk (the S2 artifacts).
2. Build an `AuthProtocol` object — the S4 handshake engine.
3. Create the S5 messaging helpers: a `MessageLogger`, a `ReplayCache` and a `TimestampValidator`.
4. Prepare empty session slots — `session_key` and `peer_public_key` stay `None` until the handshake completes.
5. Create a TCP server socket. `SO_REUSEADDR` lets the program re-bind the same port quickly after a restart instead of waiting for the operating system to release it.
6. `_seq` counts outgoing BSMs; `_last_peer_seq` remembers the last sequence number received (for the S5 replay check).

Theory link: this constructor wires S2 + S3 + S4 + S5 together.

6.4.3 start_server() and _accept_loop() — the server role

```
1 def start_server(self) -> None:
2     self.server_sock.bind(("0.0.0.0", self.port))
3     self.server_sock.listen(1)
4     t = threading.Thread(target=self._accept_loop, daemon=True)
5     t.start()
6
7 def _accept_loop(self) -> None:
8     conn, addr = self.server_sock.accept()          # blocks until a peer connects
9     self._run_handshake_responder(conn)
```

Listing 6.3: vehicle_node.py — listening for a peer.

Step by step:

1. `bind(("0.0.0.0", port))` attaches the socket to the chosen port on every network interface (0.0.0.0 = “all addresses”).
2. `listen(1)` marks it ready to accept one queued connection.
3. The actual waiting happens in a *daemon thread* so the main program is not frozen.
4. `accept()` blocks until a peer connects, then hands the new connection to the *responder* handshake.

Theory link: the server role; threads. *Analogy:* the shop opening its doors and a staff member waiting at the entrance.

6.4.4 connect_to_peer() — the client role with retries

```
1 def connect_to_peer(self, host, port) -> bool:
2     retries = 5
3     for attempt in range(1, retries + 1):
4         try:
5             conn = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
6             conn.settimeout(10)
7             conn.connect((host, port))
8             conn.settimeout(None)
9             self._run_handshake_initiator(conn)
10            return True
11        except (ConnectionRefusedError, OSError) as exc:
12            if attempt < retries:
13                time.sleep(2 * attempt)          # exponential backoff
14    return False
```

Listing 6.4: vehicle_node.py — connecting to a peer, with backoff.

Step by step:

1. Try up to five times.
2. Create a fresh socket; `settimeout(10)` means “give up a single connect attempt after 10 seconds”.
3. `connect((host, port))` dials the peer. On success, `settimeout(None)` switches back to normal blocking mode and the *initiator* handshake runs.
4. On failure, sleep `2 * attempt` seconds (growing each time) and retry. After five failures, give up and return `False`.

Theory link: the client role; exponential backoff.

6.4.5 The two handshake drivers

```
1 def _run_handshake_initiator(self, conn):
2     hello = self.auth.build_hello() # Step 1
3     send_framed(conn, hello.to_json().encode())
4     challenge = ChallengeMessage.from_json(recv_framed(conn).decode()) # Step 2
5     response = self.auth.process_challenge(challenge) # Step 3
6     send_framed(conn, response.to_json().encode())
7     ack = AckMessage.from_json(recv_framed(conn).decode()) # Step 4
8     self.session_key = self.auth.process_ack(ack)
9     self.peer_public_key = self.auth.peer_public_key
10    self._start_comm_threads(conn)
11
12 def _run_handshake_responder(self, conn):
13     hello = HelloMessage.from_json(recv_framed(conn).decode()) # Step 1
14     challenge = self.auth.process_hello(hello) # Step 2
15     send_framed(conn, challenge.to_json().encode())
16     response = ResponseMessage.from_json(recv_framed(conn).decode()) # Step 3
17     ack = self.auth.process_response(response, hello) # Step 4
18     send_framed(conn, ack.to_json().encode())
19     self.session_key = self.auth.session_key
20    self._start_comm_threads(conn)
```

Listing 6.5: vehicle_node.py — driving the 4-step handshake over the socket.

Step by step: these two methods take the abstract S4 handshake and run it over the real socket.

1. The *initiator* builds and sends HELLO, receives CHALLENGE, sends RESPONSE, receives ACK — calling the S4 AuthProtocol methods and using to_json/from_json to turn each message into bytes and back.
2. The *responder* mirrors it: receive HELLO, send CHALLENGE, receive RESPONSE, send ACK.
3. Both end by storing session_key and peer_public_key, then calling _start_comm_threads.
4. If any step raises AuthError, the handshake is abandoned and the connection closed — no session, no messaging.

Theory link: this is S4 made real over a network. *Analogy:* the secret handshake actually performed at the door, not just rehearsed.

6.4.6 The send and receive loops

```
1 def _start_comm_threads(self, conn):
2     threading.Thread(target=self._bsm_send_loop, args=(conn,), daemon=True).start()
3     threading.Thread(target=self._receive_loop, args=(conn,), daemon=True).start()
4
5 def _bsm_send_loop(self, conn):
6     while self.running:
7         self._seq += 1
8         bsm = BasicSafetyMessage(vehicle_id=self.vehicle_id,
9                                 speed_kmh=round(random.uniform(60, 100), 1), heading_deg=45.0,
10                                latitude=6.9271 + random.uniform(-0.001, 0.001),
11                                longitude=79.8612 + random.uniform(-0.001, 0.001),
12                                timestamp=time.time(), sequence_num=self._seq,
13                                brake_applied=random.random() < 0.1,
14                                acceleration=round(random.uniform(-2, 3), 1))
15         wire = secure_send(bsm, self.session_key, self.private_key, self.logger)
16         send_framed(conn, wire)
17         time.sleep(self.bsm_interval)
18
```

```

19 def _receive_loop(self, conn):
20     while self.running:
21         try:
22             raw = recv_framed(conn)
23             bsm = secure_receive(raw, self.session_key, self.peer_public_key,
24                                 self.replay_cache, self.timestamp_validator, self.logger,
25                                 last_seq=self._last_peer_seq)
26             self._last_peer_seq = bsm.sequence_num
27         except TamperError as exc:
28             log.warning("[%s] TAMPER ALERT: %s", self.vehicle_id, exc)
29         except (ReplayError, SequenceError) as exc:
30             log.warning("[%s] REPLAY ALERT: %s", self.vehicle_id, exc)
31         except ConnectionError:
32             break

```

Listing 6.6: vehicle_node.py — the two BSM loops (one per thread).

Step by step:

1. `_start_comm_threads` launches *two* daemon threads so sending and receiving run at the same time.
2. **Send loop:** every `bsm_interval` seconds (default 1), increment the sequence counter, build a BSM with slightly-randomised values, call `secure_send` (S5) to sign+encrypt it, then `send_framed` it.
3. **Receive loop:** read one framed message, call `secure_receive` (S5) to decrypt and verify it, and remember its sequence number.
4. **The security reaction:** if `secure_receive` raises `TamperError`, the loop logs a *TAMPER ALERT* and *keeps running* — it does not crash. `ReplayError` / `SequenceError` give a *REPLAY ALERT*. A real `ConnectionError` ends the loop.

Theory link: threads (two loops at once); this is where S5's exceptions become visible security events. *Analogy:* a guard who, on spotting a fake, notes it in the logbook and carries on watching — rather than fainting.

6.4.7 get_state() and shutdown()

`get_state` gathers a snapshot — certificate status, authentication status, peer id, recent messages, alerts and counters — as a dictionary for the S7 dashboard to display. `shutdown` sets `running = False` (so the loops stop) and closes the server socket.

6.4.8 main() — the program entry point

`main` reads the command-line arguments (`-vehicle-id`, `-port`, `-cert`, `-key`, `-ca-cert`, optional `-peer-host`/`-peer-port`, `-dashboard-port`), builds the `VehicleNode`, starts the Flask dashboard in a daemon thread, starts the TCP server, and — if a `-peer-host` was given — connects as a client. It then keeps the main thread alive until `Ctrl+C`.

6.5 Hands-On / Practical

6.5.1 Run two vehicle nodes

You need the certificates first (Section S2). Then open *two* PowerShell terminals.

Terminal 1 — Vehicle A (server):

```

python node\vehicle_node.py --vehicle-id vehicle-a --port 9001 '
--cert ca\certs\vehicle-a_cert.pem --key ca\certs\vehicle-a_key.pem '
--ca-cert ca\certs\ca_cert.pem --dashboard-port 5001

```

Terminal 2 — Vehicle B (client):

```
python node\vehicle_node.py --vehicle-id vehicle-b --port 9002 '
--cert ca\certs\vehicle-b_cert.pem --key ca\certs\vehicle-b_key.pem '
--ca-cert ca\certs\ca_cert.pem --peer-host 127.0.0.1 --peer-port 9001 '
--dashboard-port 5002
```

Expected output (sample, Vehicle B):

```
[10:00:01] INFO: [vehicle-b] Listening on port 9002 ...
[10:00:01] INFO: [vehicle-b] Dashboard at http://localhost:5002
[10:00:02] INFO: [vehicle-b] Connecting to 127.0.0.1:9001 (attempt 1/5)...
[10:00:02] INFO: [vehicle-b] === AUTHENTICATED with vehicle-a ===
[10:00:03] INFO: [vehicle-b] BSM #1 sent: speed=82.4 km/h
[10:00:03] INFO: [vehicle-b] BSM #1 from vehicle-a: speed=77.1 km/h VERIFIED
[10:00:04] INFO: [vehicle-b] BSM #2 sent: speed=69.8 km/h
[10:00:04] INFO: [vehicle-b] BSM #2 from vehicle-a: speed=91.3 km/h VERIFIED
```

6.5.2 What the output lines mean

- Listening on port 9002 — the server thread is up.
- Connecting ... (attempt 1/5) — the client role dialling A.
- === AUTHENTICATED with vehicle-a === — the 4-step handshake succeeded; a session key now exists.
- BSM #1 sent — the send loop fired once (one second later).
- BSM #1 from vehicle-a ... VERIFIED — the receive loop got a BSM and `secure_receive` passed every check.

6.5.3 Try This Yourself (mini-experiment)

Start Vehicle B *first*, before Vehicle A exists.

Predict before you run. B's server starts, then B tries to connect to A. A is not there, so `connect_to_peer` fails attempt 1 and sleeps 2 s, fails attempt 2 and sleeps 4 s, and so on — you should see (attempt 1/5), (attempt 2/5) ... with growing pauses. Now start Vehicle A within those 20 seconds: the next retry *succeeds* and the handshake runs. This demonstrates exponential backoff making startup order not matter.

6.6 Attack Relevance

S6 does not invent a new defence — it is the layer that *deploys* every earlier defence and makes the system react to attacks at runtime.

The key behaviour: the receive loop wraps `secure_receive` in `try/except`. A tampered or replayed BSM raises an exception, which the loop *catches*, *logs as an alert*, and then *continues*. An attacker therefore cannot crash a vehicle by feeding it bad messages — the node simply rejects them and keeps going.

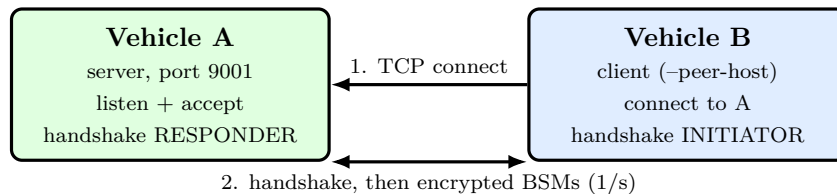
Honest scope note. S6 carries the security in the *payload*, but the TCP connection itself is plain. An attacker on the network can still see *that* two cars are talking, how often, and packet sizes (this is *metadata*). They cannot read or forge the contents, but the project does not hide traffic patterns. Also, the server does `listen(1)` and `_accept_loop` handles exactly *one* peer — this is a two-car simulation, not a scalable many-vehicle network.

Attack	S6's role in stopping it	Where
Tamper	The receive loop catches the <code>TamperError</code> from <code>secure_receive</code> and logs a TAMPER ALERT — without crashing.	<code>_receive_loop</code> .
Replay	The receive loop catches <code>SequenceError</code> / <code>ReplayError</code> and logs a REPLAY ALERT.	<code>_receive_loop</code> .
MITM / impersonation	A failed handshake raises <code>AuthError</code> ; the driver closes the connection so no session is formed.	<code>_run_handshake_*</code> .
Stream desync	Length-prefix framing means a manipulated stream cannot quietly shift message boundaries.	<code>recv_framed</code> .

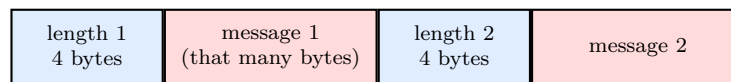
Table 6.1: S6 turns the library defences into a running, attack-resistant program.

6.7 Illustrations

6.7.1 Diagram 1 — the two-node network



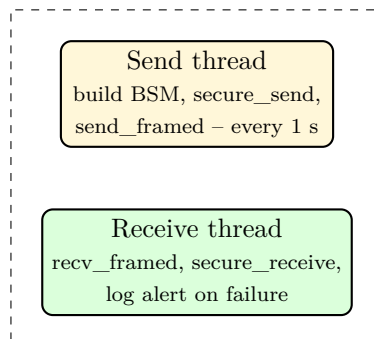
6.7.2 Diagram 2 — length-prefix framing



recv_framed reads the 4-byte length first, then exactly that many bytes — so the byte stream is split cleanly into whole messages.

6.7.3 Diagram 3 — the two communication threads

One TCP connection, two daemon threads



6.8 Presentation Pack

6.8.1 Slide Outline

Slide 1 — From Library to Program.

- S2–S5 were libraries; S6 is the program you run.
- One file, `vehicle_node.py`, launched once per car.
- It loads identity, runs the handshake, exchanges BSMs.
- It also starts the monitoring dashboard.

Slide 2 — TCP, Sockets and Framing.

- TCP = a reliable, ordered byte stream between two programs.
- But a stream has no message boundaries.
- Fix: a 4-byte length header before every message.
- `recv_framed` reads the length, then exactly that many bytes.

Slide 3 — Server, Client and the Handshake.

- One car listens (server); the other connects (client).
- Client connects with up to 5 retries and exponential backoff.
- Client = handshake initiator; server = responder.
- A failed handshake (`AuthError`) → connection closed.

Slide 4 — Two Threads: Send and Receive.

- Sending and receiving must happen at the same time.
- Send thread: one encrypted BSM every second.
- Receive thread: decrypt + verify every incoming BSM.
- Both are daemon threads — they stop when the program stops.

Slide 5 — Reacting to Attacks at Runtime.

- The receive loop wraps `secure_receive` in try/except.
- `TamperError` → TAMPER ALERT, logged, keep running.
- `SequenceError` → REPLAY ALERT, logged, keep running.
- An attacker cannot crash the node with bad messages.

6.8.2 Speaker Talking Points (about 30–60 seconds each)

Slide 1. “Everything we have built so far has been a library — code that other code calls. Section six is different: it is the actual program. You launch `vehicle_node.py` once for each car. It loads that car’s certificate and keys, opens a network connection to the other car, runs the four-step handshake, and then settles into exchanging encrypted safety messages once a second. It even starts the web dashboard. So this file is the conductor that brings sections two through five together into one running system.”

Slide 2. “The cars talk over TCP. TCP is like a reliable phone line — bytes arrive complete and in order. But TCP has one quirk: it delivers an unbroken stream of bytes with no markers showing where one message stops and the next starts. Imagine someone reading phone numbers aloud with no pauses. Our fix is framing: before every message we write a four-byte number saying how long the message is. The receiver reads those four bytes first, learns the length, then reads exactly that many bytes — one clean message, every time.”

Slide 3. “Someone has to be reachable first, so one car is the server: it opens a port and waits. The other is the client: it dials in. Because the two laptops may start at different times, the client retries up to five times, and waits a little longer before each retry — two seconds, four, six — so-called exponential backoff. Once connected, the client plays the initiator in the handshake and the server plays the responder. If the handshake fails — a bad certificate or a bad signature — the code raises an `AuthError` and simply closes the connection. No session, no messaging.”

Slide 4. “A real conversation needs you to speak and listen at the same time. A single line of code can only do one thing at once, so the node uses two threads. The send thread builds a safety message, secures it, and sends it — once every second, forever. The receive thread waits for incoming messages, decrypts and verifies each one. They run side by side. Both are daemon threads, which just means they shut down automatically when the main program exits.”

Slide 5. “Here is the security behaviour that matters at runtime. The receive loop calls `secure_receive` inside a try/except block. If a message has been tampered with, `secure_receive` throws a `TamperError` — the loop catches it, writes a TAMPER ALERT to the log, and carries on. A replayed message throws a `SequenceError` and becomes a REPLAY ALERT. The node never crashes. So an attacker flooding it with bad messages just fills the alert log — they cannot take the vehicle down.”

6.8.3 Live-Demo Script

1. **Open two terminals.** Start Vehicle A (server), then Vehicle B (client) with the commands above.
2. **Point at the handshake line.** `=== AUTHENTICATED with vehicle-a ===` — “the four-step handshake just ran over the network.”
3. **Point at the BSM lines.** `BSM #n sent` and `BSM #n ... VERIFIED` — “send thread and receive thread, both running.”
4. **Show robustness.** Mention that running an attack script against the node produces a TAMPER/REPLAY ALERT line but the node keeps going.
5. **Optional.** Start B before A to show the retry/backoff messages.

6.8.4 Examiner Q&A

1. **Q: What is TCP, and does it provide security here?**
A: TCP is a reliable, ordered byte-stream protocol between two programs. It provides *no* security — the connection is plain. All security is in the SecureBSM payload (S5).
2. **Q: What is the framing problem and how is it solved?**
A: TCP gives a boundary-less byte stream. The fix is a 4-byte big-endian length header before each message; `recv_framed` reads the length then exactly that many bytes.
3. **Q: Why does `_recv_exactly` use a loop?**
A: A single `recv` may return fewer bytes than requested, so the loop keeps reading until exactly `n` bytes are collected.
4. **Q: What is the difference between the server and client?**
A: The server binds a port and waits (`listen/accept`); the client connects to the server’s address. The client is the handshake initiator, the server the responder.
5. **Q: Why are two threads needed?**
A: Sending a BSM every second and receiving BSMs must happen simultaneously; one thread runs the send loop, the other the receive loop. Daemon threads stop automatically when the program exits.

6. Q: What is exponential backoff and why use it?

A: Waiting longer before each retry (2 s, 4 s, 6 s...). It lets a slow-starting server come up without the client hammering the network.

7. Q: What happens when a tampered BSM arrives?

A: `secure_receive` raises `TamperError`; the receive loop catches it, logs a `TAMPER ALERT`, and continues — the node does not crash.

8. Q: Name a limitation of this networking design.

A: The server accepts exactly one peer (`listen(1)`), so it is a two-car simulation, not a scalable network. Also, traffic metadata (timing, sizes, the fact that two cars talk) is visible even though contents are protected.

6.9 Glossary

TCP

Transmission Control Protocol: reliable, ordered byte-stream networking.

Socket

The software end point of a network connection.

Byte stream

A continuous flow of bytes with no message boundaries.

Framing

Adding markers so a stream can be split into messages.

Length-prefix

A length number written just before a message.

Big-endian

Byte order with the most significant byte first.

Server

The side that listens and waits to be contacted.

Client

The side that initiates the connection.

bind / listen / accept

Server socket steps: claim a port, mark ready, take a connection.

connect

The client socket step: dial the server.

Port

A numbered endpoint on a computer that a program listens behind.

0.0.0.0

“All network interfaces” — bind on every local address.

SO_REUSEADDR

A socket option allowing quick re-binding of a port after restart.

Thread

A separate line of execution inside one program.

Daemon thread

A thread that does not keep the program alive on its own.

Concurrency

Doing more than one task in overlapping time.

Exponential backoff

Waiting progressively longer before each retry.

Initiator / Responder

The two handshake roles (client / server here).

VehicleNode

The class representing one running vehicle.

secure_send / secure_receive

The S5 functions the loops call.

TamperError / SequenceError

Exceptions the receive loop catches and logs.

Metadata

Information about traffic (timing, size) rather than its content.

6.10 References and Further Study

- **Python socket module documentation** — <https://docs.python.org/3/library/socket.html>
Read for: bind, listen, accept, connect, recv, sendall.
- **Python socket programming HOWTO** — <https://docs.python.org/3/howto/sockets.html>
Read for: a beginner walk-through of client/server sockets and framing.
- **Python threading module documentation** — <https://docs.python.org/3/library/threading.html>
Read for: Thread, daemon threads, concurrency.
- **Transmission Control Protocol (Wikipedia)** — https://en.wikipedia.org/wiki/Transmission_Control_Protocol
Read for: what TCP guarantees and how it works.
- **Network socket (Wikipedia)** — https://en.wikipedia.org/wiki/Network_socket
Read for: the socket concept explained simply.
- **Exponential backoff (Wikipedia)** — https://en.wikipedia.org/wiki/Exponential_backoff
Read for: why retry delays grow.
- **Python struct module documentation** — <https://docs.python.org/3/library/struct.html>
Read for: the 4-byte big-endian length header ("**>I**").
- **Python argparse documentation** — <https://docs.python.org/3/library/argparse.html>
Read for: how the command-line options are parsed in **main**.
- **Beej's Guide to Network Programming** — <https://beej.us/guide/bgnet/>
Read for: a famous, friendly introduction to sockets and framing.

End of Section S6. Next: S7 — the Flask monitoring dashboard that visualises this running node.

Chapter 7

S7 — The Monitoring Dashboard

7.1 Title and “What You’ll Learn”

Section Title

S7 — The Monitoring Dashboard: a small web page that lets a human *watch* the secure V2V system work in real time.

Plain-English Summary (read this first)

The security work is done by Sections S2–S6. Section S7 adds *eyes*: a web page, served by a tiny **Flask** web application, that shows — live — a vehicle’s certificate status, whether it has authenticated, how many messages it has sent and received, and any security alerts. The browser asks the server “what is happening now?” every three seconds and re-draws the page. It is crucial to understand that the dashboard is a *window*, not a *lock*: it *observes* security, it does not *enforce* it.

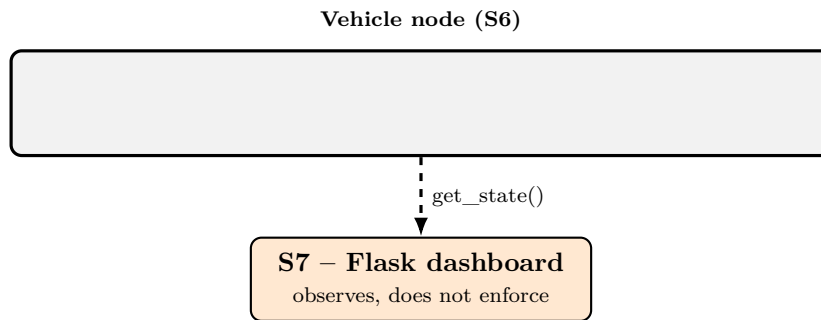
Learning Outcomes

After studying this section you will be able to:

- Explain what **Flask**, a **web server** and an **HTTP route** are.
- Explain a **REST API** and the difference between **GET** and **POST**.
- Explain **JSON** as the language between browser and server.
- Explain **polling** and why the page refreshes every 3 seconds.
- Explain what **CORS** is and why it appears here.
- Walk through all four routes in `app.py` and the JavaScript in `dashboard.html`.
- Explain why the dashboard is *observability*, not a security control.

7.2 Where This Fits

S7 sits *beside* the pipeline, not inside it. The three security stages (Identity, Authentication, Secure Messaging) run inside the vehicle node of S6. S7 attaches a viewing window to that running node: `app.py` reads the node’s state through the `get_state()` method (introduced in S6) and serves it to a browser. If you removed S7 entirely, the cars would still communicate just as securely — you simply would not be able to *see* it.



7.3 The Theory You Must Study

7.3.1 Web Server, Flask and HTTP

Plain definition. A **web server** is a program that answers requests from web browsers. **HTTP** (HyperText Transfer Protocol) is the language of those requests and answers. **Flask** is a small, popular Python library for building web servers.

Real-life analogy. A web server is an information desk. HTTP is the fixed call-and-response format: a visitor asks a question (“request”), the desk gives an answer (“response”). Flask is a starter kit for setting up that desk.

Why it exists. People want to *see* the V2V system in an ordinary web browser. A web server is what lets a browser ask for and receive a page.

How it works here. `app.py` calls `Flask(__name__)` to create the server, then defines a few *routes*. `vehicle_node.py` runs this Flask app in a background daemon thread.

Security property it provides. None — the dashboard is a viewer.

7.3.2 Routes / Endpoints

Plain definition. A **route** (or *endpoint*) is a specific web address the server knows how to answer, paired with the code that produces the answer.

Real-life analogy. The different windows at a post office: “Stamps”, “Parcels”, “Complaints”. Each address goes to a specific window with specific staff.

Why it exists. One server must answer several different questions — “give me the page”, “give me the live data”, “clear the alerts”. Routes keep each question separate.

How it works here. The `@app.route("/...")` decorator above a function says “when a browser asks for this address, run this function”. `app.py` defines four routes.

Security property it provides. None directly.

7.3.3 REST API, GET and POST

Plain definition. An **API** (Application Programming Interface) is a way for one program to ask another for data or actions. A **REST API** over HTTP uses different *methods*: **GET** *reads* data (and changes nothing); **POST** *sends* data or *triggers a change*.

Real-life analogy. GET is *looking at* the noticeboard. POST is *pinning a new note* to it. Looking changes nothing; pinning does.

Why it exists. The browser needs two kinds of interaction: fetch the latest state (read) and clear the alerts (change). Using GET for reads and POST for changes keeps the design clean and predictable.

How it works here. `/api/state` and `/api/alerts` are GET routes (read-only). `/api/clear-alerts` is a POST route (`methods=["POST"]`) because it changes something — it empties the alert list.

Security property it provides. None directly, though using GET only for safe reads is good hygiene.

7.3.4 JSON — the Browser/Server Language

Plain definition. JSON (JavaScript Object Notation) is a simple text format for structured data — names paired with values, in braces.

Real-life analogy. A filled-in form with labelled boxes: `vehicle_id: vehicle-a, total_sent: 42`.

Why it exists. The Python server and the JavaScript in the browser are different languages, but both understand JSON, so it is the neutral format they exchange.

How it works here. Flask’s `jsonify(...)` turns a Python dictionary into a JSON response. In the browser, `await r.json()` turns that JSON text back into a JavaScript object the page can read.

Security property it provides. None — it is a data format.

7.3.5 Polling

Plain definition. Polling means the browser repeatedly asks “anything new?” on a timer, rather than the server pushing updates.

Real-life analogy. Refreshing your email inbox every few minutes instead of waiting for a notification.

Why it exists. It is the *simplest* way to keep a page up to date. The dashboard does not need split-second freshness, so a humble timer is enough — no need for more complex “push” technology.

How it works here. The JavaScript line `setInterval(fetchState, 3000)` runs `fetchState` every 3000 milliseconds (3 seconds). Each call fetches `/api/state` and redraws the page.

Security property it provides. None.

7.3.6 CORS

Plain definition. CORS (Cross-Origin Resource Sharing) is a browser security rule that, by default, stops a page loaded from one web address from reading data from a *different* address. A server can relax this rule with special response headers.

Real-life analogy. A building security rule: a visitor with a “Floor 5” pass cannot wander onto Floor 3. CORS headers are the management explicitly saying “actually, let this pass work on Floor 3 too”.

Why it exists here. Vehicle A’s dashboard runs on port 5001 and Vehicle B’s on port 5002 — the browser treats those as different “origins”. To let a page on one port read the API on another, the server must allow it.

How it works here. An `@app.after_request` hook adds the header `Access-Control-Allow-Origin: *`. The `*` means “any origin may read this API”.

Security property it provides. CORS is itself a security mechanism — but here it is *loosened* to `*`. That is fine for a local classroom demo, but in a real deployment an open `*` on an unauthenticated API would be a weakness. (More on this in the Attack Relevance section.)

7.4 Code Walkthrough

7.4.1 File: `dashboard/app.py` — the Flask server

`create_app()` — build the web application

Purpose: create the Flask app, bind it to a running vehicle node, and define the routes.

```
1 def create_app(node) -> Flask:
2     app = Flask(__name__,
3                 template_folder=str(Path(__file__).parent / "templates"))
4
```

```

5  @app.after_request
6  def _cors(response):
7      response.headers["Access-Control-Allow-Origin"] = "*"
8      response.headers["Access-Control-Allow-Methods"] = "GET, POST, OPTIONS"
9      response.headers["Access-Control-Allow-Headers"] = "Content-Type"
10     return response
11
12     @app.route("/")
13     def index():
14         return render_template("dashboard.html")
15
16     @app.route("/api/state")
17     def api_state():
18         return jsonify(node.get_state())
19
20     @app.route("/api/alerts")
21     def api_alerts():
22         return jsonify({"alerts": node.logger.get_alerts()})
23
24     @app.route("/api/clear-alerts", methods=["POST"])
25     def api_clear_alerts():
26         node.logger.clear_alerts()
27         return jsonify({"status": "cleared"})
28
29     return app

```

Listing 7.1: dashboard/app.py — the whole Flask application.

Step by step:

1. `create_app(node)` takes the running `VehicleNode` (from S6) so the dashboard has a live data source.
2. `Flask(...)` creates the server; `template_folder` tells it where the HTML page lives.
3. The `_cors` function, marked `@app.after_request`, runs *after every response* and adds the CORS headers (see the theory above).
4. Route `/` — the home page. `render_template` loads and returns `dashboard.html`.
5. Route `/api/state` — a GET endpoint. It calls the node's `get_state()` and `jsonifys` the dictionary. This is the endpoint the browser polls every 3 seconds.
6. Route `/api/alerts` — a GET endpoint returning just the alert list.
7. Route `/api/clear-alerts` — a POST endpoint (`methods=["POST"]`) because it *changes* state: it calls `clear_alerts()` and confirms with `{"status": "cleared"}`.

Arguments / return: `create_app` takes `node` and returns the Flask `app`; `vehicle_node.py` then runs it in a thread. **Theory link:** routes, REST (GET vs POST), JSON, CORS. *Analogy:* `app.py` is the information desk; each route is one window.

The data source: `node.get_state()`

`app.py` itself produces no data — it is a thin layer. The real content comes from the vehicle node's `get_state()` method (studied in S6), which returns a dictionary of: `vehicle_id`, certificate status and expiry, authentication status, peer id, whether a session is active, the recent messages, the alert list, and the statistics counters. `/api/state` just wraps that dictionary in JSON.

7.4.2 File: `dashboard.html` — the page and its JavaScript

The page structure (HTML)

The HTML body has four visible regions:

1. A **header** with the title.
2. Three **status cards**: “Certificate Status”, “Authentication Status” and “Security Alerts” (the last one has a *Clear Alerts* button).
3. A **stats row** of four boxes: Messages Sent, Messages Received, Attacks Blocked, Session Active.
4. A **Recent Messages table** with one row per message and a tick/cross for each security property.

Every value on the page starts as a placeholder (– or **LOADING**) and is filled in by JavaScript once the first data arrives.

The CSS (styling) — one security-relevant detail

Most of the CSS is cosmetic (a dark theme, rounded cards). One rule *is* worth noting — the blinking-alert animation:

```

1  /* -- Blinking alert card -- */
2  @keyframes blink-red {
3    0%, 100% { border-color: #f85149; box-shadow: 0 0 15px rgba(248,81,73,0.4); }
4    50%      { border-color: #30363d; box-shadow: none; }
5  }
6  .alert-active { animation: blink-red 1s infinite; }

```

Listing 7.2: dashboard.html — the alert-blink CSS (decorative comment dashes simplified).

When JavaScript adds the class `alert-active` to the alert card, this animation makes it flash red once per second — so a security alert is impossible to miss on screen.

The JavaScript — fetch, update, repeat

```

1  async function fetchState() {
2    try {
3      const r = await fetch('/api/state');    // GET the live state
4      const d = await r.json();              // parse JSON into an object
5      update(d);                             // redraw the page
6    } catch (e) { console.error('Fetch failed', e); }
7  }
8
9  function update(d) {
10   // Certificate card
11   const cb = document.getElementById('cert-badge');
12   cb.textContent = d.cert_status;
13   cb.className = 'badge ' + (d.cert_status === 'VALID' ? 'badge-green' : 'badge-
14     red');
15   // Alert card -- turn the card red and blinking if there are alerts
16   const alerts = d.alerts || [];
17   if (alerts.length > 0) {
18     document.getElementById('alert-card').classList.add('alert-active');
19   } else {
20     document.getElementById('alert-card').classList.remove('alert-active');
21   }
22   // Stats: Attacks Blocked = tamper blocked + replay blocked
23   const s = d.stats || {};
24   document.getElementById('stat-blocked').textContent =
25     (s.tamper_attempts_blocked || 0) + (s.replay_attempts_blocked || 0);
26   // ... fill the message table row by row ...
27 }
28
29 async function clearAlerts() {
30   await fetch('/api/clear-alerts', { method: 'POST' });    // POST

```

```

30   fetchState();
31 }
32
33 setInterval(fetchState, 3000);    // poll every 3 seconds
34 fetchState();                    // and once immediately

```

Listing 7.3: dashboard.html — the polling JavaScript (core parts).

Step by step:

1. `fetchState` is `async` — it sends a GET request to `/api/state`, awaits the reply, and parses the JSON.
2. `update(d)` takes that data object and writes it into the page: it sets the certificate badge text and colour, shows/clears the blinking alert state, fills the four stat boxes (note “Attacks Blocked” is the *sum* of tamper and replay counters), and rebuilds the message table.
3. `clearAlerts` sends a POST to `/api/clear-alerts` (this is the only state-changing call the page makes), then immediately refetches.
4. `setInterval(fetchState, 3000)` repeats the fetch every 3 seconds; the bare `fetchState()` call runs once at page load so the user does not wait 3 seconds for the first data.

Theory link: polling; GET for reads, POST for the change; JSON as the shared format. *Analogy:* a receptionist who phones the back office every 3 seconds and re-writes the whiteboard with the latest news.

7.5 Hands-On / Practical

7.5.1 Open the dashboard

Start a vehicle node as in Section S6 (it launches the dashboard automatically), then open a browser at the dashboard address printed in the log:

```

http://localhost:5001      (Vehicle A)
http://localhost:5002      (Vehicle B)

```

You can also confirm the API directly without a browser, using PowerShell:

```
curl http://localhost:5001/api/state
```

Expected output (sample of the `/api/state` JSON):

```

{
  "vehicle_id": "vehicle-a",
  "cert_status": "VALID",
  "cert_cn": "vehicle-a",
  "cert_expiry": "2027-05-17",
  "auth_status": "AUTHENTICATED",
  "peer_id": "vehicle-b",
  "session_active": true,
  "recent_messages": [ ... ],
  "alerts": [],
  "stats": {"total_sent": 12, "total_received": 11,
            "tamper_attempts_blocked": 0, "replay_attempts_blocked": 0}
}

```

7.5.2 What the dashboard shows

- **Certificate Status card** — VALID (green) if the vehicle’s certificate parsed correctly.
- **Authentication Status card** — AUTHENTICATED (green) once the handshake has completed; shows the peer id.

- **Security Alerts card** — 0 ALERTS (green) normally; if an attack is blocked it turns red and *blinks*.
- **Stats row** — live counts of messages sent, received, attacks blocked, and whether a session is active.
- **Recent Messages table** — the last 20 messages, each with green ticks for Auth / Encrypted / Integrity / Replay-protected.

7.5.3 Illustration placeholder — dashboard screenshot

[ADD SCREENSHOT HERE]

Run a vehicle node, open `http://localhost:5001` in a browser, take a screenshot of the live dashboard, save it as `dashboard.png` in this folder, and replace this box with:

```
\includegraphics[width=0.85\textwidth]{dashboard.png}
```

7.5.4 Try This Yourself (mini-experiment)

With the two vehicle nodes running and the dashboard open, run an attack script against the system (for example `python attacks\tamper_attack.py`, Section S8) — or simply note what *would* happen: when a TAMPER or REPLAY alert is logged, the node’s `get_state()` returns a non-empty `alerts` list.

Predict before you run. On the *next* 3-second poll, the JavaScript `update` function sees `alerts.length > 0`, adds the `alert-active` class to the alert card, and the card starts blinking red; the “Attacks Blocked” number ticks up. Now click *Clear Alerts*: a POST is sent, the alert list empties, and on the next poll the card goes calm green again. As a second experiment, change `setInterval(fetchState, 3000)` to 1000 — predict the page now refreshes once per second instead of every three.

7.6 Attack Relevance

The dashboard defends against *nothing*. This is the single most important point of S7, and an examiner will test it. The dashboard is **observability**: it makes the work of the real defences *visible to a human*. It is the difference between a smoke alarm (a defence-adjacent alert) and the sprinkler system (the actual defence).

What the dashboard *does* contribute:

- It **surfaces** blocked attacks. When S5/S6 reject a tampered or replayed message, the alert is logged; the dashboard turns it into a blinking red card and an “Attacks Blocked” counter — so a human immediately sees the system is under attack and is holding.
- It builds **confidence**: the green ticks per message let an observer confirm every BSM really was authenticated, encrypted, integrity-checked and replay-protected.

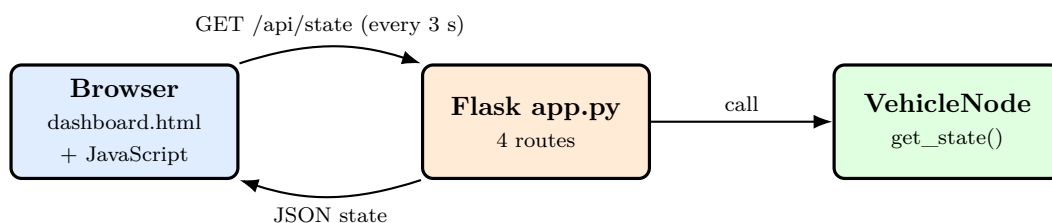
Honest weaknesses of the dashboard itself. If an examiner asks “could the dashboard be attacked?”, the honest answer is yes, in two ways, and neither affects the V2V security itself:

1. **The API is unauthenticated.** Anyone who can reach port 5001 can GET `/api/state` and read the vehicle's status, or POST `/api/clear-alerts` to wipe the alert history. There is no password on the dashboard.
2. **CORS is wide open** (`Access-Control-Allow-Origin: *`), so any web page in the browser could read the API.

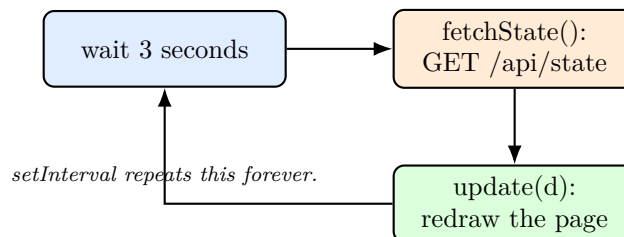
For a local classroom demo this is acceptable and is a deliberate simplification. In a real deployment the monitoring interface would need authentication and a restricted CORS policy. Crucially, even if an attacker cleared the alert log, they would *not* weaken the cars' encryption, signatures or handshake — they would only blind the *viewer*.

7.7 Illustrations

7.7.1 Diagram 1 — browser, Flask and the vehicle node



7.7.2 Diagram 2 — the polling loop



7.8 Presentation Pack

7.8.1 Slide Outline

Slide 1 — What the Dashboard Is.

- A small web page that shows the V2V system live.
- Built with Flask (a Python web framework).
- Shows certificates, authentication, stats, alerts.
- It is a *window*, not a *lock*.

Slide 2 — How the Browser and Server Talk.

- Four routes: the page, state, alerts, clear-alerts.
- GET reads data; POST changes data.
- They exchange JSON — the shared data format.
- CORS headers allow the cross-port setup.

Slide 3 — Polling: Always Up to Date.

- `setInterval(fetchState, 3000)` — ask every 3 seconds.
- `fetchState` GETs `/api/state`, parses JSON.

- **update** redraws cards, stats and the message table.
- Simple timer; no complex push technology needed.

Slide 4 — Seeing an Attack.

- Blocked attacks become alerts in the node’s logger.
- The alert card turns red and *blinks*.
- “Attacks Blocked” = tamper blocked + replay blocked.
- *Clear Alerts* button POSTs to reset the list.

Slide 5 — Observability, Not Enforcement.

- The dashboard defends nothing — S2–S6 do that.
- It makes the defences visible and builds confidence.
- Honest weakness: the API has no password; CORS is open.
- Clearing alerts blinds the viewer — it does not weaken the cars.

7.8.2 Speaker Talking Points (about 30–60 seconds each)

Slide 1. “Sections two through six do the actual security work. Section seven adds eyes. It is a small web page, served by a tiny Python web application built with Flask, that shows the system working in real time — the certificate’s status, whether the cars have authenticated, how many messages have flowed, and any security alerts. The key idea, and I will keep repeating it, is that this dashboard is a window, not a lock. It lets you watch the security; it does not provide the security.”

Slide 2. “The browser and the server talk over HTTP using four routes. One route serves the HTML page itself. One returns the live state. One returns just the alerts. And one clears the alerts. There is a deliberate split here: the read-only routes use the GET method, and the one route that changes something — clearing the alerts — uses POST. Looking at a noticeboard versus pinning a new note. They exchange data as JSON, a text format both Python and JavaScript understand, and a CORS header lets the two dashboards on different ports work together.”

Slide 3. “How does the page stay current? By polling. There is one line of JavaScript — `setInterval, fetchState, three thousand milliseconds` — that says: every three seconds, ask the server for the latest state. Each time, `fetchState` sends a GET request, gets JSON back, and the `update` function rewrites the page: the cards, the statistics, the message table. It is not the most sophisticated technique, but the dashboard does not need split-second freshness, so a simple timer is exactly right.”

Slide 4. “When the security system blocks an attack — a tampered or replayed message — it records an alert. The dashboard turns that alert into something a human cannot miss: the alerts card flips to red and blinks once a second, and the ‘Attacks Blocked’ counter, which is the tamper count plus the replay count, ticks up. There is also a *Clear Alerts* button; pressing it sends a POST request that empties the alert list, and the card calms down on the next poll.”

Slide 5. “Let me be very clear about the dashboard’s role, because it is a favourite exam question. The dashboard enforces nothing. If you deleted it, the cars would be exactly as secure. What it adds is observability — it makes the invisible defences visible, and the green ticks let you confirm every message really was protected. And honestly, the dashboard has its own weak spots: the API has no password, and CORS is wide open. For a classroom demo that is fine. But note: even an attacker who cleared the alert log would only blind the viewer — the encryption, the signatures and the handshake would all still hold.”

7.8.3 Live-Demo Script

1. **Start the system.** Launch two vehicle nodes (S6). Open `http://localhost:5001` and `http://localhost:5002`.
2. **Point at the cards.** Certificate VALID, Authentication AUTHENTICATED — “green means the security stages succeeded.”
3. **Point at the table.** The message rows with green ticks — “every BSM authenticated, encrypted, integrity-checked.”
4. **Show the API.** In a terminal run `curl http://localhost:5001/api/state` — “this raw JSON is what the page polls every 3 seconds.”
5. **Trigger and clear an alert.** Run an attack script (S8); watch the alert card blink red; press *Clear Alerts* and watch it reset.

7.8.4 Examiner Q&A

1. **Q: Does the dashboard make the system more secure?**
A: No. It is observability, not enforcement. It *shows* the security worked; it does not provide it. Removing it leaves the cars equally secure.
2. **Q: What is Flask and what are routes?**
A: Flask is a small Python web framework. A route maps a web address (e.g. `/api/state`) to the function that produces its response.
3. **Q: Why is `/api/clear-alerts` a POST and the others GET?**
A: GET is for safe reads that change nothing; POST is for actions that change state. Clearing the alert list changes state, so it is POST.
4. **Q: How does the page stay up to date?**
A: Polling — `setInterval(fetchState, 3000)` fetches `/api/state` every 3 seconds and the `update` function redraws the page.
5. **Q: What is JSON used for here?**
A: It is the shared data format. Flask’s `jsonify` sends Python data as JSON; the browser’s `r.json()` parses it back.
6. **Q: What is CORS and how is it configured?**
A: A browser rule limiting cross-origin reads. The `after_request` hook sets `Access-Control-Allow-Origin *`, allowing any origin — fine for a demo, too permissive for production.
7. **Q: Where does the dashboard’s data come from?**
A: From the vehicle node’s `get_state()` method (S6); `app.py` only wraps that dictionary as JSON.
8. **Q: Could clearing the alerts help an attacker?**
A: Only cosmetically — it blinds the human viewer. It does not touch the encryption, signatures or handshake, so the cars stay secure.

7.9 Glossary

Dashboard

A web page that displays the live state of the system.

Web server

A program that answers requests from web browsers.

Flask

A small Python web framework used to build the server.

HTTP

HyperText Transfer Protocol: the request/response language of the web.

Route / endpoint

A web address paired with the code that answers it.

API

Application Programming Interface: a way for programs to ask for data/actions.

REST

A style of HTTP API using methods like GET and POST.

GET

An HTTP method for reading data (changes nothing).

POST

An HTTP method for sending data or triggering a change.

JSON

JavaScript Object Notation: a text format for structured data.

jsonify

The Flask function that turns Python data into a JSON response.

render_template

The Flask function that returns an HTML page.

Polling

Repeatedly asking the server for updates on a timer.

setInterval

The JavaScript function that runs code on a repeating timer.

fetch

The browser function that sends an HTTP request.

CORS

Cross-Origin Resource Sharing: a browser rule on cross-address reads.

Origin

A web address identity (scheme + host + port).

after_request

A Flask hook that runs after every response is built.

Observability

The ability to see what a system is doing.

Enforcement

Actually preventing/blocking bad behaviour (what S2–S6 do).

get_state()

The VehicleNode method that supplies the dashboard's data.

Alert

A recorded security event (a blocked tamper or replay).

7.10 References and Further Study

- **Flask documentation — Quickstart** — <https://flask.palletsprojects.com/en/latest/quickstart/>
Read for: routes, render_template, jsonify.
- **Flask documentation — application factories** — <https://flask.palletsprojects.com/en/latest/patterns/appfactories/>
Read for: the create_app(node) pattern used here.
- **MDN — An overview of HTTP** — <https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>
Read for: how HTTP requests and responses work.
- **MDN — HTTP request methods (GET, POST)** — <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>
Read for: the difference between GET and POST.
- **MDN — Cross-Origin Resource Sharing (CORS)** — <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>
Read for: what CORS is and why the * header matters.
- **MDN — Using the Fetch API** — https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
Read for: the fetch calls in dashboard.html.
- **MDN — setInterval** — <https://developer.mozilla.org/en-US/docs/Web/API/setInterval>
Read for: how the 3-second polling timer works.
- **JSON (Wikipedia)** — <https://en.wikipedia.org/wiki/JSON>
Read for: the JSON data format.
- **REST (Wikipedia)** — <https://en.wikipedia.org/wiki/REST>
Read for: the REST API style.
- **OWASP API Security Top 10** — <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
Read for: why an unauthenticated, open-CORS API is a real-world weakness.

End of Section S7. Next: S8 — the attack scripts that try to break the system the dashboard is watching.

Chapter 8

S8 — Attack Demonstrations

8.1 Title and “What You’ll Learn”

Section Title

S8 — Attack Demonstrations: three short scripts that play the role of the attacker and prove, on screen, that the defences actually work.

Plain-English Summary (read this first)

A security claim means nothing until you *try to break it*. Section S8 is the project’s “red team”: three Python scripts that each *become the attacker*. `replay_attack.py` captures a real message and re-sends it late. `tamper_attack.py` intercepts a message and flips bits in it. `mitm_attack.py` plays the man-in-the-middle in two ways — a fake certificate and a forged signature. Each script runs the attack against the real project code and prints [BLOCKED] when the defence stops it. S8 does not add any new defence — it *tests* the defences of S2–S5.

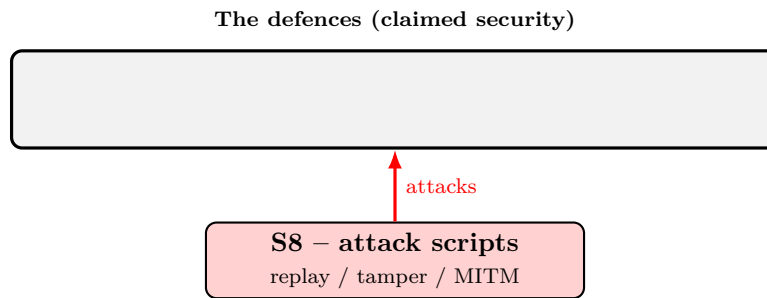
Learning Outcomes

After studying this section you will be able to:

- Explain **adversarial testing** (red-teaming) and why it matters.
- Describe exactly how a **replay**, a **tamper** and a **man-in-the-middle** attack are performed.
- Walk through each attack script step by step.
- Name the precise defence (and which section) that blocks each attack.
- Run all three scripts, read their output, and explain every line.
- Honestly explain the limits of each demonstration.

8.2 Where This Fits

S8 sits *against* the whole pipeline. The earlier sections *claim* six security properties; S8 is where those claims are put on trial. Each script imports the real project modules — the same `crypto.py`, `auth_protocol.py` and `v2v_protocol.py` the vehicles use — and attacks them. If a defence were broken, the script would print [FAIL].



8.3 The Theory You Must Study

8.3.1 Adversarial Testing (Red-Teaming)

Plain definition. *Adversarial testing* means deliberately attacking your own system to check the defences hold. A *red team* is the group (or here, the scripts) that plays the attacker.

Real-life analogy. A locksmith does not just *say* a lock is good — they hire someone to try every trick to pick it. If it opens, the lock is not good.

Why it exists. Designers are optimistic; they trust their own code. An attacker is not bound by the designer’s assumptions. Testing from the attacker’s side finds the gap between “should be secure” and “is secure”.

How it works here. Three scripts each import the genuine project modules and perform a real attack against them, then report [BLOCKED] or [FAIL].

Security property it relates to. It validates *all* of them — S8 is the evidence behind the project’s security claims.

8.3.2 The Replay Attack

Plain definition. The attacker records a genuine, valid encrypted message and sends the *exact same bytes* again later.

Real-life analogy. Secretly recording someone saying “open the garage” and playing the recording back tomorrow.

Why it is dangerous. The replayed message is *perfectly valid* — correct encryption, correct signature. Nothing is forged. The only thing wrong is that it is *old*. Re-sending an old “I am braking!” could trigger needless braking and a crash.

How the script does it. `replay_attack.py` sends a BSM, lets it be received once (which succeeds), waits **6 seconds** — one second past the 5-second freshness window — then re-sends the identical bytes.

The defence that stops it. The `TimestampValidator` (S4/S5): the message’s timestamp is now more than 5 seconds old, so `secure_receive` raises `StaleMessageError`. The sequence-number check is a second line of defence.

8.3.3 The Tamper Attack

Plain definition. The attacker intercepts a message in transit and changes some of its bytes.

Real-life analogy. Steaming open a sealed envelope, editing the letter, and trying to re-seal it — but the seal is mathematical and cannot be re-made.

Why it is dangerous. If tampering went undetected, an attacker could turn “speed 85” into “speed 0” and mislead other cars.

How the script does it. `tamper_attack.py` encrypts a BSM, then flips **3 random bytes** in the packet (deliberately past the header, in the ciphertext region), and tries to receive it.

The defence that stops it. AES-256-GCM’s authentication tag (S3, used by S5): the tag no longer matches the changed ciphertext, `aes_gcm_decrypt` raises `DecryptionError`, and `secure_receive` turns it into a `TamperError`.

8.3.4 The Man-in-the-Middle (MITM) Attack

Plain definition. The attacker secretly positions themselves between the two cars and tries to impersonate one of them.

Real-life analogy. A dishonest interpreter between two people who do not share a language — each trusts the interpreter completely.

Why it is dangerous. A successful MITM could read and rewrite the entire conversation.

How the script does it — two scenarios.

- **Scenario A — fake certificate.** The attacker makes their own *self-signed* certificate that claims the name `vehicle-a`.
- **Scenario B — forged signature.** The attacker lets the real certificates through but tries to answer the handshake challenge by signing the nonces with *their own* private key.

The defences that stop it.

- Scenario A: `verify_certificate_against_ca` (S2/S4) checks the CA’s signature. A self-signed forgery was not signed by the CA, so it raises `CertificateError`.
- Scenario B: `ecdsa_verify` (S3/S4) checks the challenge signature with `vehicle-a`’s *real* public key. A signature made with the attacker’s key does not verify, so it returns `False`.

8.3.5 Self-Signed Forgery vs CA-Signed Trust

A genuine vehicle certificate is signed by the **CA**’s private key. The attacker does not have that key, so the best they can do is a *self-signed* certificate — one signed by their own key, where issuer and subject are the same. It *looks* like a certificate, but the chain of trust check (“did the CA sign this?”) fails immediately. *This is the heart of Scenario A.*

8.4 Code Walkthrough

8.4.1 File: `attacks/replay_attack.py`

One-line purpose: capture a valid BSM, wait past the freshness window, replay it, and show it is rejected.

```
1 def main() -> None:
2     # Setup: shared session key + signing keys
3     priv_a, pub_a = generate_ecdh_keypair()
4     priv_b, pub_b = generate_ecdh_keypair()
5     secret = ecdh_shared_secret(priv_a, pub_b)
6     session_key = derive_session_key(secret, os.urandom(16), os.urandom(16))
7     sign_key = generate_private_key(SECP256R1())
8     verify_key = sign_key.public_key()
9
10    # Step 1: Vehicle A sends a legitimate BSM
11    bsm = BasicSafetyMessage("vehicle-a", 80.0, 45.0, 6.9271, 79.8612,
12                             time.time(), 1, False)
13    raw_bytes = secure_send(bsm, session_key, sign_key, logger)
14
15    # Step 2: First delivery -- should PASS
16    received = secure_receive(raw_bytes, session_key, verify_key,
17                             ReplayCache(), TimestampValidator(5.0), logger2)
18
19    # Step 3: Wait 6 seconds (beyond the 5-second window)
20    time.sleep(6)
```



```

21
22     # Step 4: Replay the exact same bytes
23     try:
24         secure_receive(raw_bytes, session_key, verify_key,
25                        ReplayCache(), TimestampValidator(5.0), logger3, last_seq
26                        =0)
27         print("[FAIL] Replayed message was ACCEPTED")
28     except (StaleMessageError, SequenceError) as e:
29         print(f"[BLOCKED] Replay attempt REJECTED -- Reason: {e}")

```

Listing 8.1: replay_attack.py — the replay attack (key parts).

Step by step:

1. **Setup** — it builds a session key with ECDH+HKDF (simulating a finished handshake) and an ECDSA signing key, so there is a working secure channel to attack.
2. **Step 1** — it creates a BSM and calls `secure_send`; the comment in the script calls this the “[CAPTURED]” message — the bytes an attacker would have recorded.
3. **Step 2** — it delivers the bytes once with `secure_receive`. This *succeeds*, proving the message is genuinely valid.
4. **Step 3** — `time.sleep(6)` waits 6 seconds, one second past the 5-second freshness window.
5. **Step 4** — it calls `secure_receive` again with the *same* bytes. The timestamp is now stale, so a `StaleMessageError` is raised, caught, and reported as [BLOCKED].

Honest note on this demo. Each call uses a *fresh* `ReplayCache`, and recall from S5 that `secure_receive` does not actually consult the nonce cache — so in this isolated script the *timestamp* (and the sequence check) is what blocks the replay. The nonce cache is real, but it is used in the *handshake* (S4), not the BSM path. The script’s final report lists the nonce cache as a defence; that is true of the system as a whole, but it is the timestamp that fires *in this script*. **Theory link:** replay attack vs the timestamp freshness window.

8.4.2 File: attacks/tamper_attack.py

One-line purpose: encrypt a BSM, flip bits in the ciphertext, and show the tampered message is rejected while the original still works.

```

1  def main() -> None:
2      # Setup: session key + signing keys (as in replay_attack)
3      ...
4      # Step 1: create and encrypt a legitimate BSM
5      bsm = BasicSafetyMessage("vehicle-a", 85.3, 90.0, 6.9271, 79.8612,
6                                time.time(), 1, True)
7      raw_bytes = secure_send(bsm, session_key, sign_key, logger1)
8
9      # Step 2: tamper -- flip 3 random bytes in the ciphertext region
10     tampered = bytearray(raw_bytes)
11     safe_start = 30                                # skip the header fields
12     positions = [random.randint(safe_start, len(tampered) - 1) for _ in range(3)]
13     for pos in positions:
14         tampered[pos] ^= 0xFF
15
16     # Step 3: attempt to decrypt the tampered message
17     try:
18         secure_receive(bytes(tampered), session_key, verify_key,
19                        ReplayCache(), TimestampValidator(5.0), logger2)
20         print("[FAIL] Tampered message was ACCEPTED")
21     except TamperError as e:
22         print(f"[BLOCKED] Tampered message REJECTED -- Reason: {e}")
23
24     # Step 4: verify the ORIGINAL (untampered) message still works

```

```

25 received = secure_receive(raw_bytes, session_key, verify_key,
26                           ReplayCache(), TimestampValidator(5.0), logger3)

```

Listing 8.2: `tamper_attack.py` — the tamper attack (key parts).

Step by step:

1. **Setup + Step 1** — build a session key and a signed, encrypted BSM, exactly as a sender would.
2. **Step 2** — copy the wire bytes into a mutable `bytearray`, pick 3 random positions *after byte 30* (so the flips land in the ciphertext, not the header), and flip each with `^= 0xFF` (XOR with all-ones inverts every bit of that byte).
3. **Step 3** — feed the tampered bytes to `secure_receive`. AES-GCM recomputes the authentication tag, finds a mismatch, and `secure_receive` raises `TamperError` — reported as `[BLOCKED]`.
4. **Step 4** — it decrypts the *original* (untouched) bytes to prove the channel itself is fine; only the tampered copy fails. This is an important control: it shows the rejection is caused by the tampering, not by a broken setup.

Theory link: AES-256-GCM authenticated encryption — the 16-byte tag makes the message tamper-evident.

8.4.3 File: `attacks/mitm_attack.py`

One-line purpose: run a man-in-the-middle attack two ways — a fake certificate and a forged signature — and show both are rejected.

`__make_fake_cert()` — forge a self-signed certificate

```

1 def __make_fake_cert(common_name="vehicle-a"):
2     fake_key = generate_private_key(SECP256R1())
3     subject = x509.Name([
4         x509.NameAttribute(NameOID.COMMON_NAME, common_name),
5         x509.NameAttribute(NameOID.ORGANIZATION_NAME, "V2V-Security-Lab"),
6     ])
7     cert = (x509.CertificateBuilder()
8             .subject_name(subject)
9             .issuer_name(subject)                # self-signed -- NOT signed by the CA
10            .public_key(fake_key.public_key())
11            .serial_number(x509.random_serial_number())
12            .not_valid_before(datetime.utcnow())
13            .not_valid_after(datetime.utcnow() + timedelta(days=365))
14            .add_extension(x509.BasicConstraints(ca=False, path_length=None),
15                           critical=True)
16            .sign(private_key=fake_key, algorithm=hashes.SHA256()))
17     return cert.public_bytes(serialization.Encoding.PEM), fake_key

```

Listing 8.3: `mitm_attack.py` — building a fake (self-signed) certificate.

Step by step: the attacker generates their own key, builds a certificate that *names* `vehicle-a`, and — crucially — sets `issuer_name` equal to `subject_name` and signs it with their own key. It is *self-signed*. It looks like a certificate but carries the attacker's signature, not the CA's.

`main()` — the two scenarios

```

1 # SCENARIO A: certificate replacement
2 fake_cert_pem, fake_key = __make_fake_cert("vehicle-a")
3 try:

```

```

4     verify_certificate_against_ca(fake_cert_pem.decode(), ca_pem)
5     print("[FAIL] Fake certificate was ACCEPTED")
6 except CertificateError as e:
7     print(f"[BLOCKED] Fake certificate REJECTED -- Reason: {e}")
8
9 # SCENARIO B: signature forgery
10 auth_a = AuthProtocol("vehicle-a", a_key, a_cert, ca_pem)
11 hello = auth_a.build_hello()
12 auth_b = AuthProtocol("vehicle-b", b_key, b_cert, ca_pem)
13 challenge = auth_b.process_hello(hello)
14
15 attacker_key = generate_private_key(SECP256R1()) # NOT vehicle-a's key
16 nonce_data = bytes.fromhex(hello.nonce) + bytes.fromhex(challenge.nonce)
17 forged_sig = ecdsa_sign(attacker_key, nonce_data)
18
19 a_pub_key = verify_certificate_against_ca(a_cert.decode(), ca_pem)
20 result = ecdsa_verify(a_pub_key, nonce_data, forged_sig)
21 if result:
22     print("[FAIL] Forged signature was ACCEPTED")
23 else:
24     print("[BLOCKED] Forged signature REJECTED")

```

Listing 8.4: mitm_attack.py — the two MITM scenarios (key parts).

Step by step:

1. **Scenario A.** The script makes a fake certificate and passes it to `verify_certificate_against_ca`. That function tries to verify the CA's signature on it — but the CA never signed it, so it raises `CertificateError`. Reported [BLOCKED].
2. **Scenario B.** The script runs a genuine partial handshake (real A sends HELLO, real B replies with a CHALLENGE). Then the attacker, who does *not* have `vehicle-a`'s private key, signs the nonce pair with *their own* `attacker_key`.
3. It then fetches `vehicle-a`'s *real* public key from the real certificate and calls `ecdsa_verify` on the forged signature. Because the signature was made with the wrong key, verification returns `False`. Reported [BLOCKED].

Theory link: Scenario A tests the chain of trust / CA-signature check (S2); Scenario B tests challenge–response signature verification (S3/S4). *Analogy:* A is a forged passport caught by the missing government stamp; B is an impostor who cannot copy your handwriting.

8.5 Hands-On / Practical

8.5.1 Run the three attack scripts

`replay_attack.py` and `tamper_attack.py` need no certificate files. `mitm_attack.py` *does* — run the S2 commands first.

```

python attacks\replay_attack.py
python attacks\tamper_attack.py
python attacks\mitm_attack.py

```

Expected output — `replay_attack.py` (sample):

```

=====
      REPLAY ATTACK DEMONSTRATION
=====

[CAPTURED] BSM at timestamp=1747476000.123, seq=1
           Payload size: 338 bytes

[OK] First delivery ACCEPTED

```

```

    Speed: 80.0 km/h, Seq: 1

[WAITING] 6 seconds... (beyond 5-second replay window)
    6... 5... 4... 3... 2... 1...

[REPLAY] Sending captured bytes again...
[BLOCKED] Replay attempt REJECTED
    Reason: Message is 6.0s old (max 5.0s)

=====
    RESULT: REPLAY ATTACK BLOCKED
=====

```

Expected output — `tamper_attack.py` (sample):

```

=====
    TAMPER ATTACK DEMONSTRATION
=====

[ORIGINAL] BSM encrypted: 338 bytes
[TAMPER] Flipping 3 bytes in the ciphertext:
    Position 142: 0x9c -> 0x63
    Position 201: 0x07 -> 0xf8
    Position 95:  0x3a -> 0xc5
[DECRYPT] Attempting to decrypt tampered message...
[BLOCKED] Tampered message REJECTED
    Reason: Message tampered -- AES-GCM auth tag mismatch
[VERIFY] Decrypting the ORIGINAL (untampered) message...
[OK] Original message decrypted: speed=85.3 km/h

    RESULT: TAMPER ATTACK BLOCKED

```

Expected output — `mitm_attack.py` (sample):

```

=====
    MAN-IN-THE-MIDDLE ATTACK DEMONSTRATION
=====

--- SCENARIO A: Fake Certificate ---
[ATTACK] Created fake certificate for 'vehicle-a' (self-signed)
[CHECK] Vehicle B verifies fake cert against CA...
[BLOCKED] Fake certificate REJECTED
    Reason: Certificate verification failed: ...

--- SCENARIO B: Forged Signature ---
[HELLO] Vehicle A sends authentic HELLO (nonce=3f1a9c2e...)
[CHALLENGE] Vehicle B sends CHALLENGE (nonce=8b4d77e1...)
[ATTACK] Attacker signs nonces with THEIR key (not vehicle-a's key)
[BLOCKED] Forged signature REJECTED
    Reason: Signature does not match vehicle-a's public key

    RESULT: BOTH ATTACK SCENARIOS BLOCKED

```

8.5.2 What the output lines mean

- [CAPTURED] / [ORIGINAL] — the genuine message the attacker starts from.
- [OK] First delivery ACCEPTED — proof the message really is valid before the attack.
- [BLOCKED] ... REJECTED — the defence fired; the **Reason** line names the exact error.
- RESULT: ... BLOCKED — the script's verdict: the defence held.
- In `mitm`, *two* [BLOCKED] lines — one per scenario.

8.5.3 Try This Yourself (mini-experiment)

Open `replay_attack.py` and change the wait from 6 seconds to 3:

```
wait = 6      ->      wait = 3
```

Predict before you run. 3 seconds is *inside* the 5-second freshness window, so the timestamp check will *pass*. The sequence check also passes (`seq=1 > last_seq=0`), and `secure_receive` does not consult the nonce cache. Prediction: the replayed message is *accepted* and the script prints `[FAIL] Replayed message was ACCEPTED`. This is an honest, valuable result — it shows that in this isolated script the *timestamp window* is the mechanism doing the blocking, and that a replay *within* the window would need the nonce-cache layer (which the full handshake uses) to be caught. **Change it back to 6 afterwards.**

8.6 Attack Relevance

S8 *is* the attacks — so here the question is reversed: *which defence does each script test?*

Script	Defence it tests	Section that provides it
<code>replay_attack.py</code>	Timestamp freshness window; sequence numbers.	S4 (<code>TimestampValidator</code>), S5 (sequence check).
<code>tamper_attack.py</code>	AES-256-GCM authentication tag.	S3 (<code>aes_gcm_decrypt</code>), used by S5.
<code>mitm_attack.py</code> (A)	CA-signature / chain-of-trust check.	S2 / S4 (<code>verify_certificate_against_ca</code>).
<code>mitm_attack.py</code> (B)	Challenge-response signature verification.	S3 / S4 (<code>ecdsa_verify</code>).

Table 8.1: Each S8 script is a test harness for a specific defence.

Why the attacker always loses.

- *Replay* — the attacker cannot make an old message young; the timestamp betrays it.
- *Tamper* — the attacker cannot recompute the AES-GCM tag without the session key; any edit is detected.
- *MITM A* — the attacker cannot sign a certificate as the CA without the CA’s private key.
- *MITM B* — the attacker cannot sign the challenge as `vehicle-a` without `vehicle-a`’s private key.

Every attack fails for the same underlying reason: **the attacker does not possess a required secret** (the session key, the CA key, or the vehicle’s private key).

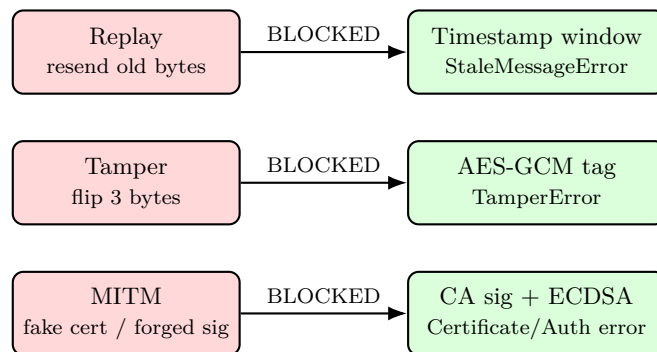
Honest limitations of these demos.

1. They run *in one process* against the library functions — they do not attack a live TCP connection. They prove the *cryptographic* defences, not the networking.
2. The replay demo uses fresh `ReplayCache` objects and the BSM path does not consult the nonce cache, so the timestamp does the blocking — a replay *within* 5 seconds would not be caught *by this script* (see the experiment above).
3. They demonstrate the *specific* attacks chosen; they are not an exhaustive proof of security.

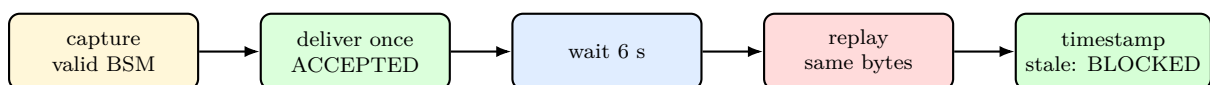
Stating these limits is itself good security practice — it is honest about what the evidence does and does not show.

8.7 Illustrations

8.7.1 Diagram 1 — the three attacks and their defences



8.7.2 Diagram 2 — the replay attack timeline



8.8 Presentation Pack

8.8.1 Slide Outline

Slide 1 — Why Attack Your Own System?

- A security claim is worthless until tested.
- S8 is the project's red team — three attack scripts.
- Each becomes the attacker against the real code.
- [BLOCKED] = the defence held; [FAIL] = it broke.

Slide 2 — The Replay Attack.

- Capture a valid message, wait 6 seconds, re-send it.
- The message is perfectly valid — just *old*.
- Timestamp check: $6\text{ s} > 5\text{ s} \rightarrow \text{StaleMessageError}$.
- Result: replay BLOCKED.

Slide 3 — The Tamper Attack.

- Encrypt a BSM, flip 3 random bytes in the ciphertext.
- AES-GCM tag no longer matches $\rightarrow \text{TamperError}$.
- The original, untouched message still decrypts fine.
- Result: tamper BLOCKED.

Slide 4 — The Man-in-the-Middle Attack.

- Scenario A: fake self-signed certificate $\rightarrow \text{CertificateError}$.
- Scenario B: forged challenge signature $\rightarrow \text{ecdsa_verify False}$.
- Attacker lacks the CA key and the vehicle's private key.
- Result: both scenarios BLOCKED.

Slide 5 — Why the Attacker Always Loses.

- Every attack fails for one reason: a missing secret.

- No session key, no CA key, no vehicle private key.
- Honest limits: in-process tests, not a live network attack.
- The demos prove the crypto, not an exhaustive security proof.

8.8.2 Speaker Talking Points (about 30–60 seconds each)

Slide 1. “It is easy to *say* a system is secure. It means nothing until someone genuinely tries to break it. Section eight is exactly that — the project’s red team. There are three scripts, and each one becomes the attacker. They do not use a watered-down copy of the code; they import the real project modules and attack them. If a defence failed, the script would print FAIL in capital letters. Every time, it prints BLOCKED.”

Slide 2. “First, the replay attack. The attacker records a genuine, valid, encrypted message. Notice — nothing is forged here; the message is completely legitimate. The script delivers it once, and it is accepted, proving it is valid. Then it waits six seconds and sends the exact same bytes again. Now the message’s timestamp is six seconds old, and our freshness window is five. `secure_receive` raises a `StaleMessageError`. The replay is blocked — not because the message is fake, but because it is old.”

Slide 3. “Second, the tamper attack. The script encrypts a safety message, then plays the attacker: it flips three random bytes in the middle of the packet, in the ciphertext region. Then it tries to receive it. AES-GCM recomputes its authentication tag, the tag does not match the changed bytes, and `secure_receive` throws a `TamperError`. To prove the rejection is really caused by the tampering, the script then decrypts the *original* untouched message — and that one works perfectly. Only the tampered copy fails.”

Slide 4. “Third, the man-in-the-middle, in two flavours. In Scenario A the attacker builds their own certificate claiming to be vehicle-a. But they cannot sign it as the CA — they do not have the CA’s key — so it is self-signed, and our certificate check raises a `CertificateError`. In Scenario B the attacker lets the real certificates through but tries to answer the handshake challenge by signing the nonces with their own key. When we verify that signature with vehicle-a’s real public key, it returns False. Both scenarios blocked.”

Slide 5. “Step back and notice the pattern. Every single attack fails for the same root reason: the attacker is missing a secret. They do not have the session key, so they cannot tamper. They do not have the CA’s key, so they cannot forge a certificate. They do not have the vehicle’s private key, so they cannot forge a signature. And to be honest about the limits: these scripts run in one process against the library functions, so they prove the cryptography is sound, not that the networking is flawless, and they test the specific attacks we chose rather than every possible attack.”

8.8.3 Live-Demo Script

1. **Replay.** Run `python attacks\replay_attack.py`. Point at First delivery ACCEPTED (valid), the 6-second countdown, then [BLOCKED] ... Message is 6.0s old.
2. **Tamper.** Run `python attacks\tamper_attack.py`. Point at the three flipped byte positions, then [BLOCKED] ... auth tag mismatch, then the original still decrypts.
3. **MITM.** Run `python attacks\mitm_attack.py` (certificates must exist). Point at the two [BLOCKED] lines — one for the fake certificate, one for the forged signature.
4. **Optional break-it.** Change the replay wait to 3 seconds and show the message is now accepted — explaining the timestamp window.

8.8.4 Examiner Q&A

1. **Q: Why does the project include attack scripts at all?**

A: To *prove* the defences by adversarial testing. A claimed defence is only credible once a real attack against the real code is shown to fail.

2. **Q: In the replay attack, is the message forged?**

A: No — it is a genuine, validly encrypted and signed message. The only thing wrong is that it is old; the timestamp window catches it.

3. **Q: How exactly does the tamper attack get blocked?**

A: Flipping bytes changes the ciphertext; AES-GCM's recomputed authentication tag no longer matches, `aes_gcm_decrypt` raises `DecryptionError`, and `secure_receive` raises `TamperError`.

4. **Q: Why does the tamper script also decrypt the original?**

A: As a control — it proves the channel works and that the rejection was caused specifically by the tampering, not a setup error.

5. **Q: What are the two MITM scenarios and what blocks each?**

A: (A) A fake self-signed certificate, blocked by `verify_certificate_against_ca` (no CA signature). (B) A forged challenge signature, blocked by `ecdsa_verify` with the real public key.

6. **Q: What single thing does the attacker always lack?**

A: A required secret — the session key, the CA's private key, or the vehicle's private key. Without it, no attack succeeds.

7. **Q: A replay within 5 seconds — would this script catch it?**

A: Not by itself: the timestamp would pass and `secure_receive` does not consult the nonce cache. Catching that needs the nonce-cache layer, which the handshake (S4) uses.

8. **Q: Name a limitation of these demonstrations.**

A: They run in one process against library functions, not a live TCP connection, and they test chosen attacks — so they evidence the cryptographic defences rather than constitute an exhaustive proof.

8.9 Glossary

Adversarial testing

Deliberately attacking your own system to test it.

Red team

The people (or scripts) that play the attacker.

Replay attack

Re-sending a previously captured valid message.

Tamper attack

Changing a message's bytes in transit.

MITM

Man-in-the-Middle: an attacker secretly between two parties.

Self-signed certificate

A certificate signed with its own key, not the CA's.

Forged signature

A signature made with the wrong (attacker's) private key.

Freshness window

The 5-second age limit on a valid message.

Authentication tag

The 16-byte AES-GCM value that detects tampering.

Sequence number

A per-sender counter that must always increase.

StaleMessageError

Exception for a message older than the window.

TamperError

Exception for a packet that failed an integrity check.

CertificateError

Exception when certificate verification fails.

XOR / ^=

The bitwise operation used to flip bits when tampering.

bytearray

A mutable sequence of bytes (so it can be edited).

Session key

The shared 32-byte AES key (here simulated for the demos).

CA

Certificate Authority — whose private key the attacker lacks.

Chain of trust

The signature link from a certificate to the trusted CA.

ecdsa__verify

The function that returns False for a forged signature.

verify__certificate__against__ca

The function that rejects a non-CA-signed cert.

8.10 References and Further Study

- **Replay attack (Wikipedia)** — https://en.wikipedia.org/wiki/Replay_attack
Read for: how replay works and standard defences.
- **Man-in-the-middle attack (Wikipedia)** — https://en.wikipedia.org/wiki/Man-in-the-middle_attack
Read for: the MITM concept behind both scenarios.
- **Tampering / message integrity (Wikipedia: Data integrity)** — https://en.wikipedia.org/wiki/Data_integrity
Read for: why detecting modification matters.
- **Red team (Wikipedia)** — https://en.wikipedia.org/wiki/Red_team
Read for: the idea of adversarial testing.
- **Penetration test (Wikipedia)** — https://en.wikipedia.org/wiki/Penetration_test
Read for: professional security testing of which these scripts are a tiny version.
- **OWASP Web Security Testing Guide** — <https://owasp.org/www-project-web-security-testing>
Read for: how real-world security testing is structured.

- **Galois/Counter Mode (Wikipedia)** — https://en.wikipedia.org/wiki/Galois/Counter_Mode
Read for: why a flipped byte breaks the AES-GCM tag.
- **X.509 (Wikipedia)** — <https://en.wikipedia.org/wiki/X.509>
Read for: why a self-signed certificate fails the CA check.
- **Python cryptography — exceptions** — <https://cryptography.io/en/latest/exceptions/>
Read for: `InvalidSignature` and `InvalidTag`, the errors behind the rejections.
- **MITRE ATT&CK — Adversary-in-the-Middle** — <https://attack.mitre.org/techniques/T1557/>
Read for: how MITM is catalogued as a real-world technique.

End of Section S8. Next: S9 — the security evaluation that measures how well these defences perform.

Chapter 9

S9 — Security Evaluation

9.1 Title and “What You’ll Learn”

Section Title

S9 — Security Evaluation: a script that *measures* the project — how fast the cryptography runs and how reliably the defences catch attacks — and prints a pass/fail report.

Plain-English Summary (read this first)

Section S8 *proved* the defences work by attacking them once each. Section S9 goes further: it *measures* them, many times over, and writes the numbers down. One script, `evaluate_security.py`, runs six tests. Three are **speed** tests — can the encryption, decryption and signing keep up with a car sending messages in real time? Three are **correctness** tests — do the replay and tamper defences catch *100%* of attacks, and are the random nonces truly unique? Every test has a numeric **target**; the script prints a table of PASS / FAIL and saves it to a report file. S9 is the *evidence* behind the project’s security claims.

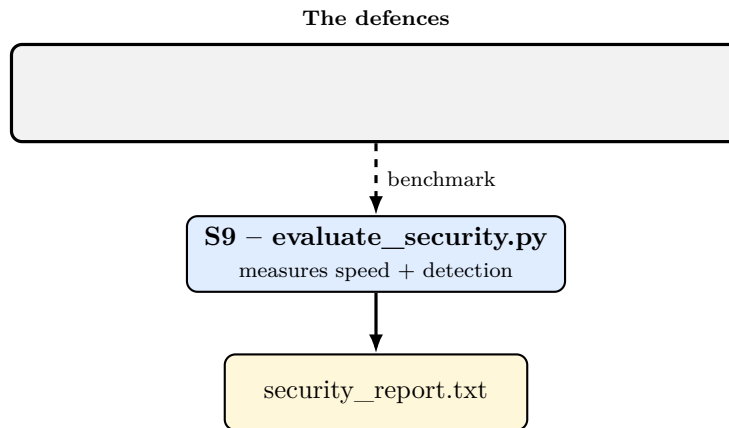
Learning Outcomes

After studying this section you will be able to:

- Explain **benchmarking** and what **throughput** (ops/second) means.
- Explain why a **high-resolution timer** is used for measurement.
- Explain a **detection rate** and why 100% is the target.
- Explain **nonce uniqueness** and, briefly, the *birthday problem*.
- Walk through all six test functions and the report formatter.
- Run the evaluation, read the table, and explain every row.
- Explain the difference between *measuring* and *proving* security.

9.2 Where This Fits

S9 sits *above* the whole pipeline as its report card. The earlier sections *build* the defences and S8 *attacks* them once; S9 *quantifies* them. It imports the real crypto functions (S3) and the real `ReplayCache` (S4) and runs them thousands of times to produce hard numbers — both for *performance justification* (“ECDSA and AES are fast enough for V2V”) and for *security evidence* (“replay and tamper detection are 100%”).



9.3 The Theory You Must Study

9.3.1 Benchmarking and Throughput

Plain definition. *Benchmarking* means measuring how fast a piece of code runs, under controlled, repeatable conditions. *Throughput* is the amount of work done per unit time — here, **operations per second** (ops/s).

Real-life analogy. Timing how many envelopes a person can seal in one minute. Do it once and it is a fluke; do it over a thousand envelopes and you have a trustworthy rate.

Why it exists. A real car sends safety messages many times a second. If encrypting or signing one message were slow, the security would make the car *unsafe* by lagging. Benchmarking answers: “is the cryptography fast enough for real time?”

How it works here. Each speed test runs an operation in a loop many times (1000 for encryption/decryption, 100 for signing), measures the total time, and computes `ops = iterations / elapsed`.

Security property it relates to. Indirectly all of them — it shows the defences are *practical*, not just correct.

9.3.2 High-Resolution Timing (`time.perf_counter`)

Plain definition. `time.perf_counter()` is a clock designed for measuring short *durations* very precisely.

Real-life analogy. A stopwatch, not a wall clock. A wall clock tells you the time of day; a stopwatch is built to measure how long something *took*.

Why it exists. An ordinary clock can jump (e.g. when the operating system adjusts the time) and is too coarse for sub-millisecond work. `perf_counter` is steady and fine-grained, so a benchmark is accurate.

How it works here. The pattern is: `start = time.perf_counter()`, run the loop, then `elapsed = time.perf_counter() - start`.

Security property it relates to. None directly — it makes the measurement trustworthy.

9.3.3 Pass/Fail Thresholds

Plain definition. A *threshold* is a fixed target a measurement must beat to count as a pass.

Real-life analogy. A driving test: parallel-park within a set time or you fail. The number is decided in advance, so the result is objective.

Why it exists. “Fast” and “reliable” are vague. A threshold turns a measurement into a clear yes/no, and makes the report honest — you cannot move the goalposts after seeing the result.

How it works here. Encryption and decryption must exceed **500 ops/s**; signing must exceed **100 ops/s**; detection tests must be **100%**; nonce collisions must be **0**. Each test sets `passed = (measurement meets target)`.

Security property it relates to. It makes every other property *measurable* rather than asserted.

9.3.4 Detection Rate

Plain definition. A *detection rate* is the fraction of attacks a defence catches: *caught divided by total*.

Real-life analogy. A metal detector at an airport: if 100 weapons go through and it catches 100, the detection rate is 100%. Catching 99 is not good enough.

Why it exists. Catching an attack “usually” is a false comfort — a single missed tampered message could cause a crash. The only acceptable target for these defences is **100%**.

How it works here. `test_replay_detection` adds 100 nonces to a `ReplayCache`, then re-adds each and counts how many raise `ReplayError`; it must be 100/100. `test_tamper_detection` encrypts 50 messages, flips a byte in each, and counts how many decryptions raise `DecryptionError`; it must be 50/50.

Security property it relates to. **Replay prevention** and **integrity**.

9.3.5 Nonce Uniqueness and the Birthday Problem

Plain definition. A *nonce* must never repeat. *Nonce uniqueness* testing checks that a batch of randomly generated nonces contains no duplicates (*collisions*). The *birthday problem* is the surprising fact that collisions become likely much sooner than intuition suggests — but only when the pool of possible values is small.

Real-life analogy. In a room of just 23 people, two probably share a birthday — because there are only 365 possible days. But if “birthdays” could be any of 2^{256} values, you would need an unimaginable crowd before a clash.

Why it exists. If two messages reused a nonce, AES-GCM’s security would break. The test gives empirical confidence that `os.urandom` is producing fresh values.

How it works here. `test_nonce_uniqueness` generates 10,000 nonces of 32 bytes (256 bits) each, puts them in a `set` (which discards duplicates), and checks `collisions = count - len(set)` is 0. With 256-bit values, 10,000 draws colliding is so unlikely it is effectively impossible — so a result of 0 is expected every run.

Security property it relates to. It underpins **confidentiality** (AES-GCM relies on unique nonces).

9.3.6 Measuring vs Proving Security

An important distinction. S9 *measures*; it does not mathematically *prove*. A 100% detection rate over 100 trials is strong *evidence*, but it is not a proof that the defence can *never* fail. Real security assurance combines testing (like S9), code review, and reasoning about the underlying mathematics. Being clear about this is part of doing evaluation honestly — the script produces a *report*, not a *theorem*.

9.4 Code Walkthrough

`evaluate_security.py` has a header helper, six test functions, a table formatter, and a `main` that runs everything and saves a report.

9.4.1 The three speed tests

```
1 def test_encryption_speed(iterations: int = 1000) -> dict:
2     """Benchmark AES-256-GCM encryption throughput."""
3     key = os.urandom(32)
4     payload = b"test BSM payload " * 10          # ~170 bytes (realistic BSM)
5
6     start = time.perf_counter()
7     for _ in range(iterations):
8         aes_gcm_encrypt(key, payload)
9     elapsed = time.perf_counter() - start
10
11     ops = iterations / elapsed
12     passed = ops > 500
13     return {"name": "Encryption Speed", "result": f"{ops:.0f} ops/s",
14            "target": ">500", "passed": passed}
```

Listing 9.1: evaluate_security.py — a representative speed benchmark.

Step by step:

1. Make a 32-byte AES key and a 170-byte payload (about the size of a real BSM).
2. Read the high-resolution clock into `start`.
3. Run `aes_gcm_encrypt` 1000 times in a loop.
4. Read the clock again; `elapsed` is the total time.
5. `ops = iterations / elapsed` is the throughput in operations per second.
6. `passed` is `True` if `ops` beats the 500 ops/s threshold.
7. Return a small dictionary with the name, result, target and pass flag.

`test_decryption_speed` is identical but times `aes_gcm_decrypt` (it encrypts once first to get something to decrypt). `test_signature_speed` times `ecdsa_sign` over 100 iterations with a target of 100 ops/s — signing is slower than symmetric encryption, so it has a lower bar. **Theory link:** benchmarking; throughput; `perf_counter`; thresholds.

9.4.2 test_replay_detection() — 100% replay catching

```
1 def test_replay_detection(count: int = 100) -> dict:
2     cache = ReplayCache(window_seconds=300)
3     nonces = [os.urandom(32).hex() for _ in range(count)]
4     for n in nonces:
5         cache.check_and_add(n)          # first time: accepted
6
7     caught = 0
8     for n in nonces:
9         try:
10             cache.check_and_add(n)      # second time: must be rejected
11         except ReplayError:
12             caught += 1
13
14     passed = caught == count
15     return {"name": "Replay Detection Rate", "result": f"{caught}/{count}",
16            "target": "100%", "passed": passed}
```

Listing 9.2: evaluate_security.py — the replay detection test.

Step by step:

1. Create a `ReplayCache` and generate 100 unique random nonces.
2. Add each nonce once — all are new, so all are accepted.

3. Try to add every nonce a *second* time. Each repeat *must* raise `ReplayError`; caught counts how many do.
4. The test passes only if `caught == 100` — a 100% detection rate.

Theory link: detection rate; the `ReplayCache` of S4. This is the quantified version of the S8 replay attack — not one replay, but 100.

9.4.3 `test_tamper_detection()` — 100% tamper catching

```

1 def test_tamper_detection(count: int = 50) -> dict:
2     key = os.urandom(32)
3     caught = 0
4     for _ in range(count):
5         ct, nonce, tag = aes_gcm_encrypt(key, b"test BSM payload " * 10)
6         tampered = bytearray(ct)
7         if tampered:
8             tampered[len(tampered) // 2] ^= 0x01          # flip one bit
9         try:
10            aes_gcm_decrypt(key, bytes(tampered), nonce, tag)
11        except DecryptionError:
12            caught += 1
13    passed = caught == count
14    return {"name": "Tamper Detection Rate", "result": f"{caught}/{count}",
15           "target": "100%", "passed": passed}

```

Listing 9.3: `evaluate_security.py` — the tamper detection test.

Step by step:

1. For each of 50 rounds: encrypt a payload, then flip *one bit* ($\wedge = 0x01$) in the middle of the ciphertext.
2. Try to decrypt the tampered ciphertext. AES-GCM's tag will not match, so `aes_gcm_decrypt` raises `DecryptionError`; `caught` is incremented.
3. The test passes only if all 50 tampered messages were caught — 50/50.

Theory link: detection rate; AES-GCM integrity (S3). Note it flips just *one bit* — the smallest possible change — and even that is always caught.

9.4.4 `test_nonce_uniqueness()` — no collisions

```

1 def test_nonce_uniqueness(count: int = 10000) -> dict:
2     nonces = set()
3     for _ in range(count):
4         nonces.add(os.urandom(32).hex())
5     collisions = count - len(nonces)
6     passed = collisions == 0
7     return {"name": f"Nonce Uniqueness ({count//1000}k)",
8           "result": f"{collisions} collisions", "target": "0", "passed": passed}

```

Listing 9.4: `evaluate_security.py` — the nonce uniqueness test.

Step by step:

1. Generate 10,000 random 32-byte nonces, adding each to a `set`.
2. A `set` automatically discards duplicates, so if any nonce repeated, `len(nonces)` would be less than `count`.
3. `collisions = count - len(nonces)` — expected to be 0.
4. The test passes if there were zero collisions.

Theory link: nonce uniqueness; the birthday problem. With 256-bit nonces, 10,000 draws colliding is astronomically unlikely.

9.4.5 `format_table()` and `main()`

`format_table` takes the list of result dictionaries and draws a neat ASCII table with columns *Test*, *Result*, *Target*, *Status*, then an “Overall: X/Y tests PASSED” line. `main` calls all six test functions, prints the table, and writes it (with a timestamp) to `tests/security_report.txt` so there is a saved record.

9.5 Hands-On / Practical

9.5.1 Run the evaluation

```
python tests\evaluate_security.py
```

Expected output (sample — speed numbers vary by computer):

```
=====
      V2V SECURITY EVALUATION REPORT
=====

+-----+-----+-----+-----+
| Test                | Result          | Target  | Status  |
+-----+-----+-----+-----+
| Encryption Speed    | 121840 ops/s    | >500    | PASS    |
| Decryption Speed    | 118905 ops/s    | >500    | PASS    |
| Signature Speed     | 6730 ops/s      | >100    | PASS    |
| Replay Detection Rate | 100/100         | 100%    | PASS    |
| Tamper Detection Rate | 50/50           | 100%    | PASS    |
| Nonce Uniqueness (10k) | 0 collisions    | 0       | PASS    |
+-----+-----+-----+-----+

Overall: 6/6 tests PASSED

Report saved to: tests/security_report.txt
```

9.5.2 What each row means

- **Encryption / Decryption Speed** — tens of thousands of operations per second, far above the 500 needed; AES-256-GCM is comfortably fast enough for a car sending one BSM per second.
- **Signature Speed** — a few thousand signings per second; ECDSA is slower than symmetric encryption (it is public-key maths) but still well past its 100 ops/s bar.
- **Replay Detection Rate 100/100** — every one of 100 repeated nonces was caught by the ReplayCache.
- **Tamper Detection Rate 50/50** — every one-bit change in 50 messages was caught by AES-GCM.
- **Nonce Uniqueness 0 collisions** — 10,000 random nonces, all distinct.
- **Overall 6/6** — every test met its target.

9.5.3 Try This Yourself (mini-experiment)

Open `evaluate_security.py` and make the encryption threshold deliberately impossible:

```
passed = ops > 500          ->          passed = ops > 999999999
```

Predict before you run. No ordinary computer encrypts 100 million times a second, so `ops` will not beat the new bar. Prediction: the Encryption Speed row now shows **FAIL**, and the summary changes to **Overall: 5/6 tests PASSED**. This shows the threshold is what turns a raw number into a pass/fail verdict — and why thresholds must be chosen *before* seeing the result, not after. **Restore the line to `> 500` afterwards.**

9.6 Attack Relevance

S9 does not block any attack itself — it *quantifies* how well the other sections block attacks, turning the project’s security claims into measured evidence.

S9 test	Attack it relates to	What it evidences
Replay Detection Rate	Replay attack (S8).	The <code>ReplayCache</code> of S4 catches 100% of repeated nonces.
Tamper Detection Rate	Tamper attack (S8).	AES-256-GCM (S3) catches 100% of one-bit changes.
Nonce Uniqueness	Nonce-reuse weakening of AES-GCM.	<code>os.urandom</code> produces no duplicate nonces.
Speed tests	(Not an attack.)	The defences are fast enough for real-time V2V — security does not make the car unsafe.

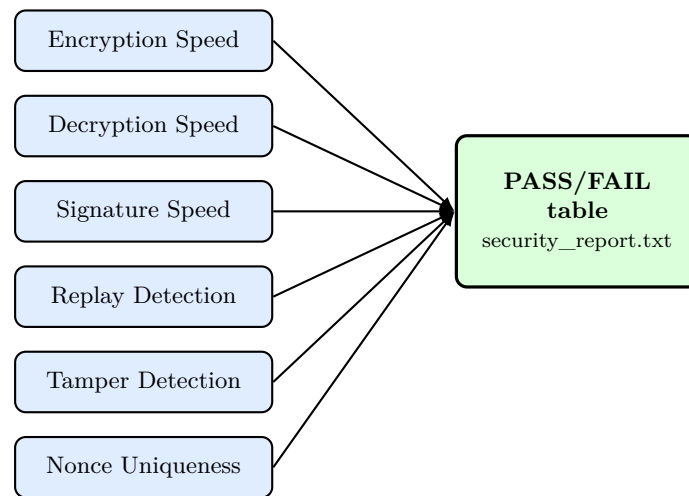
Table 9.1: S9 turns S8’s one-off demonstrations into measured, repeatable evidence.

Why the speed tests are a security matter too. If signing or encrypting a message were slow, a real car could not keep up with the safety message rate — the security layer would itself create a hazard. By showing the cryptography runs thousands to tens of thousands of times per second, S9 demonstrates the defences are *deployable*, not merely correct.

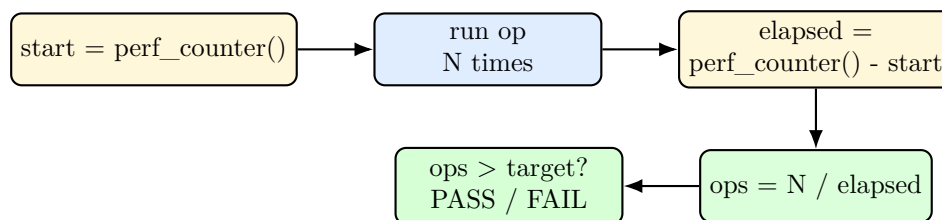
Honest scope note. S9 is *measurement*, not *proof*. A 100% detection rate over 100 or 50 trials is strong evidence but not a guarantee for every conceivable input. The tests also exercise the cryptographic functions in isolation — they do not benchmark the full networked system under load. And the thresholds (500, 100) are project-chosen reference points, not formal V2V standards. A complete security assurance would add code review and mathematical reasoning to this measurement.

9.7 Illustrations

9.7.1 Diagram 1 — the six tests feeding one report



9.7.2 Diagram 2 — the benchmark loop



9.8 Presentation Pack

9.8.1 Slide Outline

Slide 1 — Why Measure?

- S8 proved the defences once; S9 measures them at scale.
- One script, `evaluate_security.py`, six tests.
- Three speed tests, three correctness tests.
- Output: a PASS/FAIL table saved to a report file.

Slide 2 — The Speed Tests.

- Encryption, decryption, signing — measured in ops/second.
- A high-resolution timer (`perf_counter`) for accuracy.
- Targets: >500 ops/s for AES, >100 ops/s for ECDSA.
- Question answered: “fast enough for real-time V2V?”

Slide 3 — The Detection Tests.

- Replay: re-add 100 nonces — all 100 must be rejected.
- Tamper: flip one bit in 50 messages — all 50 must fail to decrypt.
- Target for both: 100%. Anything less is a fail.
- These quantify the S3 and S4 defences.

Slide 4 — Nonce Uniqueness.

- Generate 10,000 random 256-bit nonces.
- Put them in a set; count duplicates.
- Target: 0 collisions — AES-GCM needs unique nonces.
- 256-bit values make a collision effectively impossible.

Slide 5 — Measuring is Not Proving.

- 6/6 PASS is strong evidence the design holds.
- But a test is a sample, not a mathematical proof.
- Honest assurance also needs review and reasoning.
- S9 produces a *report card*, not a *theorem*.

9.8.2 Speaker Talking Points (about 30–60 seconds each)

Slide 1. “In the previous section we proved each defence works by attacking it once. Section nine is the next step: instead of one attack, we *measure*. One script, `evaluate_security.py`, runs six tests. Three of them check speed — is the cryptography fast enough? — and three check correctness — do the defences catch every attack, and are the random values truly unique? At the end it prints a clean pass-or-fail table and even saves it to a report file, so there is a permanent record.”

Slide 2. “The three speed tests run an operation in a tight loop — encrypting a thousand times, signing a hundred times — and measure how long it took with a high-resolution stopwatch called `perf_counter`. Dividing the count by the time gives operations per second. Why does this matter for security? Because a real car sends safety messages constantly. If encryption were slow, the security layer would make the car lag, which is itself dangerous. We set targets in advance — five hundred operations a second for encryption, a hundred for signing — and the real numbers beat them easily.”

Slide 3. “The detection tests quantify the defences. The replay test adds a hundred nonces to the cache, then tries to add every one again — and all one hundred repeats must be rejected. The tamper test encrypts fifty messages, flips a single bit in each, and every one of the fifty must fail to decrypt. The target for both is one hundred percent, and that is deliberate. Catching ninety-nine attacks out of a hundred is not good enough when the one you miss could be a false ‘I am braking’ that causes a crash.”

Slide 4. “The sixth test checks nonce uniqueness. Remember a nonce must never repeat, or AES-GCM’s security breaks. The test generates ten thousand random nonces and drops them into a set, which automatically removes duplicates. If the set still has ten thousand items, there were no collisions. And because each nonce is 256 bits long, the chance of a clash is astronomically small — this is the birthday problem, but with so many possible values that a collision is effectively impossible.”

Slide 5. “One honest closing point. A result of six out of six is genuinely strong evidence that the design holds up. But a test runs a sample of inputs — it is not a mathematical proof that the defence can never fail under any input. Real-world security assurance combines testing like this with careful code review and reasoning about the underlying cryptographic mathematics. So think of section nine as the project’s report card — convincing evidence — rather than a final theorem.”

9.8.3 Live-Demo Script

1. **Run it.** Execute `python tests\evaluate_security.py`.
2. **Point at the speed rows.** “Tens of thousands of operations a second — far past the target. The cryptography is fast enough.”

3. **Point at the detection rows.** 100/100 and 50/50 — “every single attack caught.”
4. **Point at the summary.** Overall: 6/6 tests PASSED.
5. **Show the saved report.** Open `tests\security_report.txt` — “a permanent, timestamped record.”
6. **Optional break-it.** Raise the encryption threshold absurdly high and re-run to show a FAIL and 5/6.

9.8.4 Examiner Q&A

1. **Q: What is the purpose of `evaluate_security.py`?**
A: To measure the project — the speed of the cryptography and the reliability of the defences — and produce a pass/fail evidence report.
2. **Q: What is throughput and why is it measured here?**
A: Operations per second. It shows the cryptography is fast enough for a car sending safety messages in real time, so security does not cause dangerous lag.
3. **Q: Why use `time.perf_counter` rather than an ordinary clock?**
A: `perf_counter` is a high-resolution, steady stopwatch built for measuring short durations; a wall clock can jump and is too coarse.
4. **Q: Why must the detection rate target be 100%?**
A: A single missed tampered or replayed safety message could cause a crash. For these defences, anything below 100% is unacceptable.
5. **Q: How does the tamper test work?**
A: It encrypts 50 messages, flips one bit of ciphertext in each, and counts how many decryptions raise `DecryptionError`. All 50 must.
6. **Q: What is the nonce uniqueness test checking, and why does it pass?**
A: That 10,000 random nonces contain no duplicates. Because each nonce is 256 bits, a collision is astronomically unlikely (the birthday problem with a vast value space), so 0 collisions is expected.
7. **Q: Where do the thresholds 500 and 100 come from?**
A: They are project-chosen reference targets, set in advance so the verdict is objective. They are not formal V2V standards.
8. **Q: Does a 6/6 pass *prove* the system is secure?**
A: No. It is strong *evidence* from a sample of inputs, not a mathematical proof. Full assurance also needs code review and reasoning about the cryptography.

9.9 Glossary

Security evaluation

Measuring how well a system’s defences perform.

Benchmarking

Measuring code speed under controlled, repeatable conditions.

Throughput

Work done per unit time — here operations per second (ops/s).

Operation

One run of a function being measured (one encrypt, one sign...).

time.perf_counter

A high-resolution timer for measuring durations.

Iteration

One pass of a benchmark loop.

Threshold / target

The fixed value a measurement must beat to pass.

Pass / Fail

The verdict from comparing a measurement to its threshold.

Detection rate

Fraction of attacks a defence catches (caught / total).

Replay detection

Catching repeated nonces (tested via `ReplayCache`).

Tamper detection

Catching modified ciphertext (tested via AES-GCM).

Nonce

A value that must be used only once.

Collision

Two generated values turning out to be identical.

Nonce uniqueness

The property that generated nonces never repeat.

Birthday problem

Why collisions appear sooner than expected — but only with a small value space.

set

A Python collection that automatically discards duplicates.

ReplayCache

The S4 class storing seen nonces; tested here.

DecryptionError

The exception AES-GCM raises on a tampered message.

ReplayError

The exception the `ReplayCache` raises on a repeated nonce.

security_report.txt

The saved file holding the evaluation results.

Evidence vs proof

Tests give evidence; only mathematics gives proof.

9.10 References and Further Study

- **Python time module — perf_counter** — https://docs.python.org/3/library/time.html#time.perf_counter
Read for: the high-resolution timer used in every speed test.
- **Python set type documentation** — <https://docs.python.org/3/library/stdtypes.html#set>
Read for: how the nonce-uniqueness test discards duplicates.

- **Benchmark (computing) (Wikipedia)** — [https://en.wikipedia.org/wiki/Benchmark_\(computing\)](https://en.wikipedia.org/wiki/Benchmark_(computing))
Read for: what benchmarking is and how to do it fairly.
- **Throughput (Wikipedia)** — <https://en.wikipedia.org/wiki/Throughput>
Read for: the operations-per-second idea.
- **Birthday problem (Wikipedia)** — https://en.wikipedia.org/wiki/Birthday_problem
Read for: why collisions appear sooner than intuition suggests.
- **Birthday attack (Wikipedia)** — https://en.wikipedia.org/wiki/Birthday_attack
Read for: the cryptographic version — why nonce/hash sizes matter.
- **Cryptographic nonce (Wikipedia)** — https://en.wikipedia.org/wiki/Cryptographic_nonce
Read for: why nonce uniqueness is essential to AES-GCM.
- **NIST SP 800-38D — AES-GCM** — <https://csrc.nist.gov/pubs/sp/800/38/d/final>
Read for: the official nonce rule the uniqueness test supports.
- **Python os.urandom documentation** — <https://docs.python.org/3/library/os.html#os.urandom>
Read for: the source of the random nonces being tested.
- **Software testing (Wikipedia)** — https://en.wikipedia.org/wiki/Software_testing
Read for: the broader idea that testing gives evidence, not proof.

End of Section S9 — and of the V2V study pack (S1–S9). The evaluation report is the project’s final evidence that all six security properties hold.